

ZLang HDL

Language Reference

Community Release

v0.1.0a23

Installation, language semantics, tooling, RTL generation, and verification

Apache-2.0 Community distribution

Contents

1 Installation, WSL2, and external tools **4**
1.1 Recommended installation with uv . . . 4
1.2 Conventional venv and pip alternative . . . 5
1.3 Windows through WSL2 . . . 5
1.4 Check and run ZLang . . . 6
1.5 Choose which external tools you need . . . 6
1.6 Recommended: OSS CAD Suite . . . 7
1.7 Alternative: install components separately . . . 7
1.8 Verify the installation . . . 7
1.9 Verification architecture and troubleshooting . . . 8

2 Native simulation **8**

3 Getting started **9**
3.1 Install for development . . . 9
3.2 A first module . . . 10
3.3 Check named verification goals . . . 10
3.4 Source files . . . 11
3.5 Reading common notation . . . 11
3.6 Where to continue . . . 12

4 Types, numerics, and packed representation **12**
4.1 Characters, fixed strings, and tuples . . . 12
4.2 Nominal enums . . . 12
4.3 Nominal tagged unions . . . 13
4.4 Scalar families . . . 13
4.5 Integer width rules . . . 14
4.6 Fixed point . . . 15
4.7 Vectors and structs . . . 16
4.8 Slicing, concatenation, and representation . . . 16
4.9 Current boundaries . . . 18

5 Expressions, functions, and generics **18**
5.1 Operators and selection . . . 18
5.2 Aggregate value ergonomics . . . 19
5.3 Pure functions . . . 20
5.4 Compile-time functions and selection . . . 21
5.5 Functional datapath . . . 21
5.6 Typed bit and collection composition . . . 23
5.7 Generic structs, functions, and operators . . . 23

6 Sequential state, rules, pipelines, and storage **24**
6.1 Clock and reset . . . 24
6.2 Registers and next state . . . 25
6.3 Delays and fixed pipelines . . . 27
6.4 Exact module timing contracts . . . 27
6.5 Guarded atomic actions . . . 28
6.6 FIFOs . . . 29
6.7 Writable memories . . . 30
6.8 Initialized synchronous ROMs . . . 34
6.9 Stateful hierarchy . . . 34

7 Physical clocks and resets **34**
7.1 Recommended asynchronous reset . . . 35
7.2 Backend lowering . . . 36
7.3 Formal applicability . . . 36
7.4 Boundaries . . . 36

8 Hierarchy, protocols, and explicit CDC **37**

8.1 Modules and instances . . . 37
8.2 Clock and reset boundary . . . 39
8.3 Public top ABI . . . 39
8.4 Ready/valid . . . 39
8.5 Credit and request/response . . . 39
8.6 Aggregate protocols and the standard library . . . 40
8.7 Packets, virtual channels, and arbitration . . . 40
8.8 CSR blocks . . . 40
8.9 Clock-domain crossings . . . 42

9 Named module interfaces 42
9.1 Exact conformance . . . 43
9.2 Aggregate protocol signatures . . . 43
9.3 Selection and backend identity . . . 44
9.4 Current boundaries . . . 44

10 Tagged unions 44
10.1 Type and layout . . . 45
10.2 Construction and matching . . . 45
10.3 Deliberate boundaries . . . 45

11 Parameterized aggregate protocols 45

12 Standard buses 46
12.1 Bounded ZTPU AXI burst subset . . . 47
12.2 AXI4-Lite, APB, and RegBus . . . 47

13 Standard library 48
13.1 Fixed-point math . . . 49
13.2 Complex values, streams, storage, and coding . . . 49
13.3 Target and architecture descriptions . . . 50

14 Projects and dependencies 51
14.1 Project manifest . . . 51
14.2 Lock update . . . 51
14.3 Offline compilation . . . 52
14.4 Identity and artifacts . . . 52
14.5 Deliberate boundaries . . . 53

15 Optimization and formal verification 53
15.1 Canonical and selected IR . . . 53
15.2 One implementation-policy path . . . 53
15.3 First-class verification goals and contracts . . . 57
15.4 Formal layers . . . 58
15.5 Reports . . . 60

16 E-graph and optimization responsibilities 60

17 Implementation profiles 60
17.1 Semantic regions . . . 61
17.2 Normalization and conflicts . . . 61
17.3 Reading the backend selection report . . . 61

18 Target platform descriptions 62
18.1 Compiler-shipped source descriptions . . . 62
18.2 Manual selection . . . 62
18.3 The selected graph . . . 63
18.4 Backend and manifest . . . 63
18.5 Boundaries . . . 63

19 Target resources 63
19.1 Implemented model . . . 64

19.2 DSP48E1 pipeline binding . . . 64
19.3 Signed-product cascade . . . 64
19.4 RAMB36 validation . . . 64
19.5 Intel and ASIC status . . . 65
19.6 Clock-resource blocker . . . 65
19.7 Implementation artifact contents . . . 65

20 Target-aware architecture and pipeline planning 65
20.1 User contract . . . 65
20.2 Candidate generation and legality . . . 66
20.3 Timing DAG . . . 66
20.4 How target evidence affects selection . . . 66
20.5 Current boundary . . . 67

21 Platform constraints 67
21.1 Clock constraints . . . 67

22 Direct-SV 68
22.1 Supported composition subset . . . 68
22.2 Simulation-only architectural state access . . . 69
22.3 Formal boundary . . . 70
22.4 Explicitly unsupported . . . 70
22.5 Example and structural coverage . . . 70

23 Compiler tooling API 70
23.1 Request-scoped analysis . . . 71
23.2 Diagnostic edits . . . 71
23.3 Generated RTL navigation audit . . . 71
23.4 Document symbols . . . 71
23.5 Semantic hover . . . 72
23.6 Definitions . . . 72
23.7 References . . . 72
23.8 Rename . . . 72
23.9 Completion . . . 73
23.10 Signature help . . . 73
23.11 Semantic tokens . . . 73
23.12 Reusable tooling sessions . . . 73

24 Language Server Protocol and VS Code 74
24.1 Supported now . . . 74
24.2 Cache configuration . . . 76
24.3 Not implemented . . . 76

25 Structured diagnostics 76

26 Generated source maps 77
26.1 Exactness boundary . . . 77
26.2 Generated-RTL navigation boundary . . . 78

27 Generated navigation bundles 78
27.1 Why generated navigation uses a separate bundle . . . 78
27.2 Layout and schema . . . 78
27.3 Publication and validation . . . 79
27.4 Source freshness . . . 79
27.5 Relocatability and limitations . . . 79

28 Whole-build manifests and evidence 79
28.1 Compiler and implementation identities . . . 79
28.2 Published files and companions . . . 80
28.3 Exact evidence meanings . . . 80
28.4 Harness generation is not proof execution . . . 81
28.5 Counterexample and source attribution . . . 82

28.6 CLI publication . . . 82
28.7 Current limitations . . . 82

29 Storage-owning instance arrays **82**
29.1 Supported boundary . . . 82

30 Source and product identity **83**

31 Language support matrix **83**
31.1 Language and execution . . . 84
31.2 Formal coverage . . . 85

32 Known limitations **86**
32.1 Language and backend boundaries . . . 86
32.2 Verification boundaries . . . 86

A ZLang HDL concise source-authoring reference **87**
A.1 Non-negotiable rules . . . 87
A.2 Smallest valid modules . . . 87
A.3 Types, literals, and exact conversions . . . 88
A.4 Aggregates and functions . . . 88
A.5 State, hierarchy, and protocols . . . 88
A.6 Functional datapath and implementation intent . . . 89
A.7 Verification . . . 89
A.8 Commands and completion check . . . 89

Development reference toward 0.1.0a23 (unreleased)

This is the complete user-facing reference for the ZLang HDL compiler in the Community distribution. The backend-independent typed IR defines language semantics, and Direct SystemVerilog (Direct-SV) is the supported production RTL path. Unsupported combinations fail closed rather than publishing guessed hardware.

This reference covers installation, language semantics, compilation, editor integration, implementation planning, generated RTL, and verification. The companion quick reference is optimized for daily authoring. The PDF edition appends that quick reference to this document.

1 Installation, WSL2, and external tools

ZLang HDL supports CPython `>=3.12,<3.13`. The source compiler is release-tested on Linux x86-64. Native-simulation wheels are supplied for Linux x86-64 (including WSL2). macOS native wheels are not part of this release. You do not need to find a distribution package for that exact Python runtime: the recommended `uv` workflow can download and manage it independently of the system Python. Parsing, semantic checking and Direct-SV generation need only the Python package. Verilator, Yosys, SymbiYosys and a solver are external programs used only when their corresponding lint, synthesis or formal flow is requested.

The exact versions used by release acceptance are recorded in `release/status.json`. Other versions may work, but they are not evidence for the published release.

1.1 Recommended installation with uv

Install `uv` using its official platform instructions. Confirm that the executable is visible in the same terminal that will run ZLang:

```
uv --version
```

For ordinary use, install the wheel downloaded from the matching GitHub release as an isolated command-line tool. `uv` provisions the requested interpreter if it is not already present:

```
uv tool install --python '>=3.12,<3.13' \
  --with /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
zlang lock --version
```

```
zlang verify --version
zlang lsp --version
```

If `uv` reports that its tool directory is not on `PATH`, run `uv tool update-shell`, then start a new shell. `uv tool dir --bin` prints the command directory for manual PATH configuration. To replace an installation made from a downloaded wheel, repeat `uv tool install --force ...` with the new wheel.

For development from a Git checkout, keep the environment local to the repository:

```
git clone https://github.com/postoroniy/zlang-hdl.git
cd zlang-hdl
uv venv --python '>=3.12,<3.13'
uv pip install -e '.[test]'
source .venv/bin/activate
zlang --version
```

The commands below use `zlang` directly: it is on `PATH` after `uv tool install`, or after activating the checkout's virtual environment with `source .venv/bin/activate` in a Bash/Zsh terminal. Alternatively, in the checkout use `uv run --no-sync zlang ...` without activating it. Open a new terminal and activate the environment again when returning to the checkout.

1.2 Conventional venv and pip alternative

If a compatible Python is already available, use whatever executable name the host provides; the example deliberately avoids assuming a version-suffixed command:

```
python3 -c 'import sys; assert (3, 12) <= sys.version_info < (3, 13), sys.version'
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install \
  /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
```

For an editable development checkout, replace the two-wheel installation command with:

```
python -m pip install -e '.[test]'
```

This fallback needs a compatible interpreter and `venv` support from the host. Do not replace or upgrade the operating system's own Python merely to install ZLang; use `uv` or an isolated virtual environment.

1.3 Windows through WSL2

The current release is tested as a Linux application, not as a native Windows Python package. On Windows 10/11, install WSL2 with an Ubuntu distribution, open the Ubuntu shell, and perform the Linux installation entirely inside it. For example:

```
sudo apt-get update
sudo apt-get install -y curl git build-essential
```

Then install `uv` using its official Linux instructions and follow the `uv` steps above. Keep active projects under the WSL Linux filesystem, for example `~/src/zlang-hdl`, rather than `/mnt/c/...`; this generally gives much better compiler, Git, pytest and EDA-tool filesystem performance. Start VS Code through its WSL extension so the editor, `zlang-lsp`, Python environment and EDA executables all see the same Linux paths.

Inside a development checkout, these forms are equivalent:

```
source .venv/bin/activate
zlang examples/add.zhl --check
uv run --no-sync zlang examples/add.zhl --check
```

`uv run --no-sync` uses the already-created project environment without changing dependencies. In scripts, either activate the environment first or use `uv run --no-sync`; do not depend on a user's interactive shell state.

1.4 Check and run ZLang

Check the installed commands:

```
zlang --version
zlang lock --version
zlang verify --version
zlang lsp --version
```

`zlang-lsp` is a JSON-RPC stdio server and is normally started by the bundled VS Code extension; it is not an interactive shell command. Point the extension at the environment's executable when automatic discovery is not appropriate:

```
{
  "zlang.lsp.path": "/absolute/path/to/.venv/bin/zlang-lsp"
}
```

The extension path remains the standalone `zlang-lsp` compatibility executable because a VS Code executable setting cannot contain subcommand arguments. For manual launches and other clients, `zlang lsp` is the canonical command.

Check a design and emit production Direct-SV without installing any external EDA program:

```
zlang examples/add.zhl --check
mkdir -p build
zlang examples/add.zhl --systemverilog build/Add.sv
```

1.5 Choose which external tools you need

Only a compatible Python runtime and the `zlang-hdl` package are mandatory for checking source and emitting SystemVerilog. `uv` is an optional installer: it can obtain that Python runtime, but ZLang does not need `uv` after installation. Install other tools only for the workflows you actually use:

Workflow	Additional requirement	If it is missing
`zlang SOURCE --check` and SystemVerilog emission	None beyond Python and `zlang-hdl`	Neither Verilator nor Yosys is needed; a missing compiler package or incompatible Python prevents `zlang` from starting.
`zlang sim` (native simulation)	Matching `zlang-native-sim` wheel for the host	Simulation fails explicitly; compiler checking and RTL emission remain available.
VS Code diagnostics and navigation	Bundled extension plus `zlang-lsp` from the base package	Compiler CLI still works, but editor integration does not start.
Strict RTL lint or Verilator execution	`verilator`	RTL can still be emitted; lint and Verilator simulation cannot run.
Generic synthesis	`yosys`	RTL can still be emitted; synthesis cannot run.
Formal BMC, proof, or cover	`yosys`, `sby`, `yosys-smtbmc`, and one selected solver executable	Checking and RTL emission still work; requested formal jobs cannot complete.
Icarus-based test or simulation flow	`iverilog` and `vvp`	Only that selected flow is unavailable.
`zlang lock update` with Git dependencies	`git` and access to the pinned revision during update	Path-only projects and locked offline compilation still work; fetching a Git dependency does not.

`yosys-smtbmc` is the Yosys SMT model-checking driver. It is the executable sometimes informally shortened to "BMC"; there is no ZLang dependency named `bmct`. The default recorded formal route invokes the external `z3`

executable through this driver. Bundle replay may instead select another supported SMTBMC solver, such as `boolector`, `bitwuzla`, `yices`, or `cvc5`, with `zlang verify BUNDLE --solver NAME`. Installing only a Python solver module is not a substitute for putting the corresponding executable on `PATH`.

1.6 Recommended: OSS CAD Suite

The simplest reproducible route is the official YosysHQ OSS CAD Suite. Its release archive provides Yosys, SymbiYosys, solvers and related open-source EDA tools together. Download the archive for the host platform from the OSS CAD Suite releases, extract it, then either source its environment file for each shell:

```
source /absolute/path/to/oss-cad-suite/environment
```

or add its `bin` directory to `PATH`:

```
export PATH="/absolute/path/to/oss-cad-suite/bin:$PATH"
```

Use an absolute installation path. Do not copy individual binaries out of the suite because Yosys and SymbiYosys also use adjacent data and support files. See the upstream OSS CAD Suite release page linked above for supported archives and platform details.

Install Verilator separately when it is not supplied by the selected suite. A distribution package is convenient:

```
sudo apt-get update
sudo apt-get install verilator
```

Distribution versions vary. Follow the upstream Verilator installation guide when the release-tested version or a source build is required.

1.7 Alternative: install components separately

Component installation is useful for distribution packaging or when a pinned tool is maintained independently:

- install Yosys using the upstream Yosys build/install instructions;
- install SymbiYosys using the upstream SymbiYosys documentation;
- install the `z3` command from the official Z3 releases or a suitable distribution package;
- install Verilator using its official guide linked above;
- install Icarus Verilog from the host distribution when the selected test or simulation flow requires `iverilog` /`vvp`.

Keep all selected tool `bin` directories on the same `PATH` used to start ZLang, the terminal, CI job or VS Code. Mixed host/container installations are a common reason for a tool being visible interactively but unavailable to the compiler.

1.8 Verify the installation

```
verilator --version
yosys -V
sby --version
command -v yosys-smtbmc
z3 --version
boolector --version
bitwuzla --version
iverilog -V
```

The release-tested external-tool versions are recorded only in the release status under `eda_toolchain`; the supported Python range is recorded under `platform.python`. These are evidence for that release, not universal tool-version requirements. Other versions may work; validate them for the flow you intend to use. Finding an executable on `PATH` is not, by itself, evidence that a formal or synthesis result is valid.

Run a real formal smoke test:

```
zlang examples/verification/bounded_counter.zhl \
  --verify --verify-require proven \
```

```
--formal-depth 16 --formal-timeout 45 \
--verification-bundle build/verify/counter \
--verification-work-dir build/verify-work/counter \
--verification-report build/verify/counter-result.txt
```

Expected exit status is zero. The `capacity` assertion should be `proven`, the `reaches_capacity` cover should be `witnessed`, and `exceeds_capacity` should be `bounded_unreached`. A bounded cover miss is not a proof of unreachability. See the runnable formal examples for a deliberate counterexample, scoped assumptions, immutable bundle replay and ready/valid checking.

The hosted solver-matrix gate replays the same immutable bundle through Z3, Boolector and Bitwuzla, with Yices and cvc5 as corroborating routes. Each run retains its own solver, tool versions, result identity and counterexample. An agreement report is a consistency check, not a stronger proof status: a BMC result remains bounded, and one solver's `unknown`, timeout, or malformed result never becomes success because another solver completed. All of these executions consume the same compiler/formal lowering, so solver agreement cannot detect every defect shared by that front end or lowering.

Nightly qualification additionally replays the same immutable safety bundles through ABC PDR and bounded Pono/btormc BTOR routes. These separate engine records are qualification-only: the public CLI and required-formal policy continue to expose only the accepted SMTBMC route. Avy/AIGER is recorded as an observation rather than a gate while its pinned-suite crash remains reproducible. No agreement report creates a new proof status.

1.9 Verification architecture and troubleshooting

Formal execution is opt-in. `--check`, normal compilation and SystemVerilog emission do not invoke a solver. `bounded_pass` is bounded evidence and is never reported as `proven`; unavailable tools or bindings remain explicit `unknown`/`skipped` according to the requested policy.

If a tool is not found, run the version commands above in the same environment that starts ZLang. If a proof is `unknown`, inspect the retained report and solver logs under `--verification-work-dir`; increasing depth or timeout does not turn an unsupported binding or reset model into an executable property.

2 Native simulation

The simulation engine compiles a bounded primitive bit-vector plan to native machine code with Cranelift. It does not generate SystemVerilog or invoke an external HDL simulator during ordinary execution. There is one executable simulation semantics path:

```
from zlang import sim

program = sim.compile("examples/counter.zhl", top="Counter")
instance = program.create()
instance.tick("clk")
print(instance.get("y"))
instance.close()
```

The command-line interface uses the same native runtime directly; there is no simulation-engine selector:

```
zlang sim examples/counter.zhl --top Counter \
  --clock clk --cycles 100 --json
zlang sim examples/multi_clock_stateful.zhl --top MultiClockStateful \
  --events events.jsonl --trace build/trace.vcd
zlang sim examples/add.zhl --top Add --set a=200 --set b=55 \
  --compare-with iverilog --compare-artifacts build/sim-compare --json
```

Each non-empty `--events` JSONL line is one simulation event. An event may add `"repeat": N` to apply the same updates, resets and edges for `N` consecutive events; `N` must be a positive integer and the expanded schedule is bounded to 1,000,000 events. The CLI retains one parsed mapping and repeat count per JSONL record rather than copying its nested payload `N` times. This keeps long idle or backpressure intervals declarative without requiring a project-specific event generator.

`--compare-with iverilog` and `--compare-with verilator` emit the normal direct SystemVerilog artifact, execute it in the selected external simulator, and compare every physical output after every event. A mismatch, X/Z value, missing sample, tool failure, or timeout fails closed. Comparison artifacts are retained only when an empty `--compare-artifacts` directory is supplied. Sequential RTL must receive the same explicit reset events needed by physical hardware; use `--events` when comparing a resettable design.

One instance is not thread-safe; independent instances may run concurrently. `eval()` changes no architectural state. `edge_many()` computes all next-state values from one pre-edge snapshot and commits atomically. Synchronous resets act on their domain edge, while supported asynchronous assertion acts immediately. Values are exact-width and are never silently truncated.

The following are implementation limits of this compiler release's native simulator, not ZLang language semantics. Native simulation accepts at most:

- 32,768 compiled simulation operations;
- a cumulative packed-value work budget of 32,768 machine words;
- 16 MiB of compiled simulation data;
- packed values up to 8,192 bits;
- arithmetic operands up to 512 bits; and
- memory elements up to 8,192 bits.

Generated functional work is also bounded to 8 nested regions, 1,000,000 total iterations, and an estimated runtime-work budget of 8,000,000 operations. Compilation fails with a diagnostic when a limit is exceeded.

There is no silent fallback from native execution to an RTL simulator. Retired compatibility selector values produce deterministic migration diagnostics. Unsupported external stateful/protocol models remain unsupported instead of acquiring simulator-specific behavior.

The native executor is distributed only as audited `cp312-abi3` binary wheels:

- `manylinux_2_28_x86_64`, used by Linux x86-64 and WSL2.

The public source archive contains no Rust source, Cargo manifest/lockfile, Rust toolchain file, or native build output. The binary wheels carry Apache-2.0 license and notice files, an exact locked third-party inventory, and a CycloneDX SBOM. The public compiler retains the versioned primitive-plan/packing/ABI contract. Native behavior is validated against independently generated direct SystemVerilog through Icarus and Verilator rather than a second Python engine.

3 Getting started

The representative executable language tour is `examples/all_syntax.zhl`. It intentionally does not enumerate every legal composition or backend boundary. The syntax support matrix and compiler-owned capability registry distinguish supported, bounded, and deferred forms.

Physical ZLang HDL source files use the canonical `.zhl` suffix and MIME type `text/x-zlang-hdl`. Logical imports remain extension-independent. The former `.zl` spelling is intentionally rejected so these sources cannot be confused with the unrelated language that already owns that extension.

3.1 Install for development

This alpha release requires CPython `>=3.12,<3.13`. The recommended `uv` workflow does not require a suitable Python interpreter to be installed first; `uv` can download and manage the verified runtime. From the repository root:

```
uv venv --python '>=3.12,<3.13'
uv pip install -e '.[test]'
```

If a compatible interpreter is already available, the complete installation guide also documents a conventional `venv`/pip path without relying on a version-specific executable name. Windows users should run the Linux flow inside WSL2 rather than installing the compiler into native Windows Python.

The compiler and LSP do not require an EDA installation. Verilator, Yosys, SymbiYosys, `yosys-smtbmc`, Z3 and Icarus are optional external programs for the corresponding lint, synthesis, formal and simulation flows. See Installing ZLang HDL and the external EDA toolchain for the recommended OSS CAD Suite route, separate component installation, release-tested versions and a real formal smoke test.

Check a source file without creating backend artifacts:

```
zlang examples/all_syntax.zhl --check
```

Without `--top`, `--check` validates every declared module. With `--top NAME`, it validates only that elaboration root. This is a semantic-only demand: it does not run implementation planning, formal execution, report rendering, or the SystemVerilog backend.

3.2 A first module

```
module Add {
    in  a : u8
    in  b : u8
    out y : u9

    y = a + b
}
```

Hardware assignments are concurrent. Source order does not turn `=` into software-style sequencing. `=` drives a combinational value; `<-` schedules a register's next value at its clock edge.

Generate Direct-SV and lint it:

```
zlang examples/extended_add.zhl \
  --systemverilog build/ExtendedAdd.sv
verilator --lint-only --top-module ExtendedAdd build/ExtendedAdd.sv
```

An explicit artifact path keeps stdout empty. Use `--verbose` for success messages on stderr. A bare invocation emits the production Direct-SV artifact to stdout; use `--check` for semantic validation without emission.

3.3 Check named verification goals

Clocked modules can carry same-cycle safety and bounded-reachability goals:

```
assert count_within @ clk { count <= DEPTH }
cover reaches_full @ clk { count == DEPTH }
```

Run the applicable verification jobs and source goals:

```
zlang design.zhl --top Top --verify \
  --formal-jobs 4 \
  --verification-report build/verification.txt \
  --verification-work-dir build/verification-work
```

Or publish a hash-validated bundle and replay it without recompiling source:

```
zlang design.zhl --top Top \
  --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
  --work-dir build/verify-work
```

`bounded_pass` means only that no counterexample was found through the stated depth. A cover reports `witnessed` or `bounded_unreached`; a cover miss is not a safety failure. A `prove` request runs BMC first and starts proof only when every safety job is `bounded_pass`. Covers run once and are not rerun; an unrelated cover does not block proof, while an unwitnessed feasibility cover makes its dependent safety result vacuous/unknown. Solver configurations, logs, and VCDs are retained in the external work directory; the immutable bundle is not modified. `--formal-jobs` parallelizes independent bundled safety/cover jobs and independent selected-candidate sites. Stages inside one candidate site remain ordered; plans, evidence, and report order remain deterministic. Candidate semantic-reference equivalence jobs use deterministic subdirectories of that same external root; an exact in-session formal-aware selection reuse reports its retained root when one exists. Persistent proof-cache hits do not fabricate old workspace paths.

Each verification goal is routed against its own declared clock/reset pair. Goals in two supported synchronous domains can execute independently; an unsupported domain skips only goals in that domain. This does not add a cross-domain temporal property or change the general backend domain boundary.

The formal triggers are deliberately distinct:

- a non-`off` `--formal-policy` without `--verify` runs only the formal-aware semantic-reference equivalence gate used during selection;
- `--verify` with policy `off` runs safety verification/source safety and covers;
- `--verification-bundle` publishes the base safety/cover bundle and linking plan but does not prepare or execute selected-candidate semantic-reference equivalence;
- `--verify` with a non-`off` policy additionally executes the compatible selected-candidate Direct-SV semantic-reference equivalence route.

3.4 Source files

A file may contain imports, type aliases, structs, pure functions, operator overloads, rewrite declarations, target/resource declarations, and one or more modules:

```
import std.math.complex

type Byte = u8

struct Pair<type T> {
    left  : T
    right : T
}

fn widen(x : Byte) -> u9 { extend<9>(x) }

module Example {
    in  x : Byte
    out y : u9 = widen(x)
}
```

Select a top explicitly when a file contains several modules:

```
zlang design.zhl --top Example --check
```

Names are case-sensitive. There are no semicolons. Line comments use `//`. Non-nested block comments use `/* ... */`; both forms are lexical whitespace. Decimal, hexadecimal (`0x`), and binary (`0b`/`0B`) integer literals may contain valid underscore separators.

3.5 Reading common notation

Form	Meaning
`name = expression`	Drive a combinational value or resource control.
`state <- expression`	Schedule next-state at the active edge.
`source -> destination`	Connect typed endpoints or bind a command.
`key => expression`	Select a `switch`/choice arm.
`thing @ clk`	Associate a declaration with a clock domain.
`field @ 7:4`	Place a CSR field.
`object.field`	Select an aggregate, protocol, or resource member.
`values[index]`	Read a range-proven vector element; the selector may be runtime.
`raw[index]` for a packed scalar	Read one LSB-zero packed bit; a runtime selector needs a compiler-proven in-range unsigned interval.
`raw[offset +: WIDTH]`	Read a fixed-width LSB-based packed window; `WIDTH` is compile-time and the runtime offset must be provably in range.

ZLang requires exact canonical types at assignment boundaries. It never silently truncates, extends, performs a numeric signed/unsigned conversion, or rescales fixed-point values. Equal-width raw boundaries involving

`bits<N>` or flat `vec<N,bit>` are the documented representation-only exception; use `bitcast<T>` explicitly when the boundary is not already typed.

3.6 Where to continue

- Types and numerics
- Expressions, functions, and generics
- Sequential logic, rules, and storage
- Hierarchy, protocols, and composition
- Optimization and formal verification
- Direct-SV, CLI, and tooling
- Standard library
- Reproducible projects and dependencies
- Named module interfaces
- Validated real designs

4 Types, numerics, and packed representation

ZLang types describe hardware representation exactly. Width, signedness, fixed-point scale, vector length, struct identity, and fixed-point overflow policy participate in typing and canonical identity.

4.1 Characters, fixed strings, and tuples

`char` is a source alias for canonical `u8`; it is intentionally not a separate nominal type. `string<N>` is a source alias for the corresponding fixed hardware collection and normalizes to canonical `vec<N,u8>`:

```
letter : char = 'A'
tag : string<4> = "OFDM"
joined = concat("802.", "11a")
```

Literals contain printable ASCII directly and may use `\0`, `\n`, `\r`, `\t`, `\\`, `\'`, `\"`, or an exact `\xNN` byte. Strings have no hidden terminator or length field and cannot be empty. They inherit vector indexing, equality, `length`, `generate`/`map`, homogeneous concatenation, pure functions, generic specialization, and typed compile-time constant parameters. Registers, FIFOs, synchronous memories, and ROMs store them with the same semantics as `vec<N,u8>`. When packed, the first character is the least-significant byte (`pack("AB") == 0x4241`). Unicode, interpolation, formatting, padding, and dynamic-length strings are not implicit operations.

Tuples are structural ordered aggregates. Tuple types use `(T,U)` and tuple values use the same positional spelling:

```
pair : (u8, bit) = (data, last)
payload = pair[0]
(next_data, next_last) = pair
```

Tuple arity is 2 through 8. Projection is a zero-based integer literal and flat destructuring introduces exhaustive immutable bindings. Nested tuple values are legal, but nested/wildcard patterns, runtime tuple selection, tuple update, arithmetic, and ordering are not. Exact `==`/`!=` is structural. Component zero occupies the least-significant packed region; `pack`/`bitcast` require every component to satisfy the ordinary non-enum packing rules.

`char` is indistinguishable from `u8` during overload resolution, and `string<N>` is indistinguishable from `vec<N,u8>`. Tuple identity, by contrast, is the recursive ordered sequence of component types.

4.2 Nominal enums

Enums give control state a nominal hardware type without exposing a numeric encoding in source:

```
enum State { Idle Header Payload Done }

reg state : State = State.Idle
active = state == State.Payload
```

Members use qualified names. Their declaration-order ordinals are encoded in `max(1, ceil_log2(member_count))` bits, so a one-member enum still occupies one bit. The encoding is a stable storage/backend ABI, but enum values are not integers: arithmetic, ordering, resizing, and cross-enum comparison are errors. Equality and inequality require the same nominal enum declaration.

When an external encoding is part of the hardware contract, the representation can instead be declared explicitly:

```
enum WifiRate : bits<3> {
    Continue = 0
    Bpsk6 = 1
    Qpsk12 = 2
    Qam16_24 = 4
}
```

Every member then requires one unique code fitting the exact `bits<W>` backing type. Declaration order still controls exhaustive source selection, while the declared sparse code is the physical representation. Mixing explicit and implicit members is an error. The ordinal form above remains unchanged.

Raw boundaries use total, typed operations rather than enum casts:

```
raw     : bits<3> = enum_encode(rate)
valid   : bit = enum_valid<WifiRate>(raw)
decoded : WifiRate =
    enum_decode<WifiRate>(raw, WifiRate.Continue)
```

`enum_encode` is lossless. `enum_valid<T>` recognizes exactly the declared codes. `enum_decode<T>` returns its required same-enum fallback for every hole in the encoding, so invalid raw bits never become an invalid nominal value. These operations perform no resize and accept no signed/integer substitute.

Enums may appear in locals, outputs, registers/rules, structs, and vectors. A top-level enum input is rejected because an external invalid bit pattern would violate the nominal value invariant. General `unpack<Enum>` is likewise deferred. Exhaustive selection is described under operators and selection.

4.3 Nominal tagged unions

A tagged union represents one of several named payload shapes without exposing raw tag arithmetic:

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }
result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}
```

The nominal declaration identity is part of the type. The source-order tag is stored in the most-significant bits; fields occupy the maximum-width payload in source order and shorter payloads receive zero low-order padding. A fieldless constructor is written `Message.Idle`. Every pure `match` is exhaustive, names each variant once, and binds exactly the declared fields. See the complete bounded surface.

4.4 Scalar families

Spelling	Meaning
`bit`	A one-bit control value, distinct from integers and raw bits.
`uN`, `uint<N>`	An `N`-bit unsigned integer.
`sN`, `sint<N>`	An `N`-bit two's-complement signed integer.
`bits<N>`	An `N`-bit raw vector without integer arithmetic.

Spelling	Meaning
`fixed<W,F>`	Signed fixed point, `W` total bits and `F` fractional bits, wrapping target narrowing.
`ufixed<W,F>`	Unsigned fixed point with wrapping target narrowing.
`fixed_sat<W,F>`	Signed fixed point with saturating target narrowing.
`ufixed_sat<W,F>`	Unsigned fixed point with saturating target narrowing.

Widths are positive compile-time integers. Fixed-point types additionally require `0 <= F < W`. The families are distinct: no implicit signed/unsigned, integer/raw-bit, or integer/fixed conversion is performed.

Aliases disappear during typing:

```
type Address = uint<32>
type Payload = bits<64>
```

Alias cycles, duplicate declarations, and redefinition of built-ins are errors.

4.5 Integer width rules

An unsuffixed integer literal has the smallest exact hardware type implied by its value. Non-negative literals are unsigned; syntactic negative literals are signed two's-complement values:

Literal	Inferred type
`0`, `1`	`u1`
`2`, `3`	`u2`
`255`	`u8`
`-1`	`s1`
`-2`	`s2`
`-3`, `-4`	`s3`
`-123`	`s8`

Decimal, hexadecimal, and binary spellings with the same value infer the same type; leading zeros do not request storage width. A direct literal at an explicitly typed assignment, field, argument, or return boundary instead uses that target type when the value fits exactly. An out-of-range literal is an error. Context never recursively resizes a compound expression, so `concat(0,1)` and `0 + 1` retain the types derived by their own operators.

Unary `-` applied to a non-literal remains ordinary typed arithmetic. It does not reinterpret an unsigned signal as a signed one.

Expression	Result
`uN + uM`	`u(max(N,M)+1)`
`sN + sM`	`s(max(N,M)+1)`
`uN - uM`	`u(max(N,M))`, modular underflow
`sN - sM`	`s(max(N,M)+1)`
same-family `N * M`	Width `N+M`
`&`, `|`, `^`	Matching family, widest operand width
shifts	The left operand's exact type and width
comparisons	`bit`

Addition and multiplication preserve carry bits. A left shift discards bits beyond its fixed result width. Signed right shift is arithmetic; unsigned and raw-bit right shift is logical.

Use explicit resizing:

```
wide = extend<17>(value)
low  = truncate<8>(wide)
```

`extend<N>` preserves signedness behavior and `truncate<N>` keeps low bits. Neither changes the type family.

At an explicitly typed output, local, register write, storage write, child binding, struct field, or declared function-return boundary, the width may be obtained from that boundary while the conversion remains explicit:

```
low : u8 = truncate(wide)
wide_again : u17 = extend(low)
```

The contextual and explicit-width forms normalize to the same typed resize. An inferred local, a nested arithmetic operand, or a mismatched type family has no such context and must keep `truncate<N>` or `extend<N>`.

4.6 Fixed point

Concrete spellings expose integer and fractional widths directly:

Spelling	Canonical type	Narrowing policy
`SF8.8`	`fixed<16,8>`	wrap
`SF_Sat8.8`	`fixed_sat<16,8>`	saturate
`UF8.8`	`ufixed<16,8>`	wrap
`UF_Sat8.8`	`ufixed_sat<16,8>`	saturate

For signed values the integer part includes the sign bit. `SF8.8` therefore represents `-128.0` through `127.99609375` in steps of `2^-8`.

Arithmetic is widened before any storage conversion. Fixed addition and subtraction require equal fractional widths; multiplication returns total width `W1+W2` and fractional width `F1+F2`. Target overflow policy applies only at an explicit or contextual storage conversion, not after every intermediate node.

Direct fixed-point literals must be exactly representable and in range, even for a saturating target. Loss is explicit:

```
y = quantize<SF_Sat8.8>(value) {
    round nearest_even
    overflow saturate
}
```

The rounding mode is mandatory; there is no implicit rounding default.

`round` mode	Discarded fractional part
`toward_zero`	Truncate toward zero.
`floor`	Round toward negative infinity.
`away_zero`	Round away from zero if any fractional part remains.
`nearest_even`	Round to nearest; on an exact tie choose the even stored integer.

Form	Target and overflow behavior
`quantize(value, MODE)`	Target comes from an unambiguous fixed-point assignment/return context; overflow defaults to that target type's policy.
`quantize(value) { round MODE overflow POLICY }`	Contextual target; both clauses are required.
`quantize<T>(value) { round MODE overflow POLICY }`	Explicit fixed-point target; both clauses are required.

`POLICY` is `wrap` (modular truncation to the target bit width) or `saturate` (clamp to the target's minimum/maximum). Ordinary `fixed`/`ufixed` targets default to `wrap`; `fixed_sat`/`ufixed_sat` targets default to `saturate` only in the short two-argument form. The block's explicit policy takes precedence. Conversion order is exact rescale, rounding, then overflow. `fixed_raw(0x0C22)` creates a target-context raw pattern; `fixed_to_raw` exposes the stored integer.

An exact fixed-point `dot(a,b)` retains the full accumulator. If the destination reduces fractional precision, give one post-accumulation rounding mode:

```
out y : SF_Sat8.8
y = dot(coefficients, samples, floor)
```

4.7 Vectors and structs

`vec<N,T>` has a compile-time length:

```
in samples : vec<4,u8>
```

An explicitly typed boundary can use a non-empty vector literal or contextual replication:

```
pair          : vec<2,u8> = [left, right]
zero_values   : vec<4,u8> = repeat(0)
zero_values_2 : vec<4,u8> = repeat<4>(0)
```

Every element must have the exact contextual element type. `repeat(value)` requires the destination vector to provide its length; `repeat<N>(value)` makes the length explicit. Both forms retain ordinary vector sequence order and lower to the existing typed generation representation.

Constant indexes are checked during elaboration. Runtime reads are accepted only when conservative range analysis proves every encoded value in range; ZLang does not silently mask or clamp. A rule may update one range-proven element of a one-dimensional `reg vec<N,T>` atomically. Nested paths, multiple dynamic writes to the same register action group, and runtime-selected instances remain unsupported; see Sequential logic, rules, and storage.

Structs are nominal products:

```
struct Packet {
    data : u64
    last : bit
}

packet = Packet { data = payload last }
```

Constructors are complete and may use same-name field punning. An immutable update rebuilds the same nominal struct and preserves fields not named in the update:

```
completed = packet with { last = 1 }
```

Unknown or duplicate fields and values of the wrong exact field type are rejected. This is a pure value expression, not mutation. Recursive layouts are still rejected. Generic structs are covered in Expressions, functions, and generics.

Structural `==` and `!=` are available for exact matching struct and vector types. Comparison is recursive over every field/element and returns `bit`; it does not perform conversion, resizing, or nominal coercion.

An immutable nominal struct can also be destructured exhaustively:

```
Packet { data, last } = packet
```

Every declared field must appear exactly once, under its declared name, and the new immutable bindings may not shadow another symbol. Destructuring lowers to ordinary typed field values; it does not add tuple patterns, partial patterns, renaming, mutation, or runtime matching.

4.8 Slicing, concatenation, and representation

An inclusive, compile-time bit slice returns raw bits:

```
in word : u16
out byte : bits<8>
byte = word[15:8]
```

Both `MSB` and `LSB` are compile-time integer expressions. The result is `bits<MSB-LSB+1>`. Reversed, negative, or out-of-range bounds are errors; slicing never resizes, reinterprets, or clamps its operand.

A `bits<N>` value also supports concise single-bit selection:

```
in raw : bits<24>
out lsb : bit = raw[0]
out msb : bit = raw[23]
```

Packed indices use conventional bit numbering: index zero is the least-significant bit, exactly as `[0:0]`. A runtime unsigned selector is legal only when its compiler-proven interval lies entirely within the packed width; there is no implicit modulo, clamp, or out-of-range hardware behavior. A fixed-width runtime window uses LSB-based `+:` spelling:

```
in raw    : bits<16>
in offset : u3
out byte  : bits<8>
byte = raw[offset +: 8]
```

`WIDTH` is a positive compile-time integer. The compiler requires `offset + WIDTH <= packed_width` for every possible offset. Vector element zero likewise occupies the least-significant packed element region.

`concat(a,b,...)` accepts at least two operands. When every operand is a vector with the exact same element type it returns one longer vector, preserving source and element order:

```
in left  : vec<2,u8>
in right : vec<3,u8>
out all  : vec<5,u8>
all = concat(left, right) // left[0], left[1], right[0], right[1], right[2]
```

Otherwise no operand may be a vector and all operands must be recursively bit-packable. The result is raw `bits<N>` and the first operand occupies the most-significant bits:

```
joined = concat(high, low) // high is the MSB portion
```

Each scalar operand is typed independently, before the result width is known. For example, `concat(0,1)` is `bits<2>` and `concat(3,0)` is `bits<3>`. Neither an assignment target nor a function caller may widen an operand or the completed concatenation implicitly.

Exact packed fill constants are available without manufacturing a numeric literal of the desired width:

```
clear_mask : bits<24> = zeros<24>
set_mask   : bits<24> = ones<24>
```

The width is a positive compile-time integer expression. Both forms produce exact `bits<N>` values and lower to ordinary constants; they are not vector replication. The separate `zero<x>`/`ones<x>` spellings inside an `equiv` declaration remain typed pattern constants rather than hardware expressions.

Vector/scalar mixtures and different vector element types are errors. Convert a vector explicitly with `bitcast<bits<W>>` when raw bit assembly is intended.

`reshape(value)` obtains a vector target shape from an explicitly typed context; `reshape<T>(value)` supplies it directly. Reshape recursively enumerates leaves outer-to-inner, then rebuilds the requested shape without packing them:

```
in matrix : vec<2,vec<4,u8>>
out flat  : vec<8,u8>
flat = reshape(matrix)
matrix_again = reshape<vec<2,vec<4,u8>>>(flat)
```

Source and target must be vectors with the same leaf count and exact same leaf type. Reshape has no endianness and performs no numerical conversion.

`bitcast<T>(value)` exposes or reconstructs the exact stored representation when source and target have equal packed width and both are recursively bit-packable and non-enum. It never resizes, sign-extends, truncates,

rescales, rounds, or saturates:

```
raw    : bits<8> = bitcast<bits<8>>(signed_value)
signed : s8      = bitcast<s8>(raw)
```

`pack(value)` and `unpack<T>(raw)` remain supported low-level compatibility spellings and normalize to the same typed `Bitcast` operation. Signed integers and signed fixed-point values use their stored two's-complement pattern.

Aggregate layout is deterministic and backend-independent:

- struct field zero (declaration order) is at the MSB;
- vector element zero is at the LSB;
- tuple component `item0` is at the LSB;
- nested vectors and tuples apply the LSB-first rule recursively;
- nested structs retain declaration-order/MSB-first field layout.

The initialized `rom<T,N>` image format uses this same layout for each word. Images contain one exact-width binary word per line with address zero first; they do not inherit host byte order. Consequently simulator values and Direct-SV `$readmemb` agree on nested aggregate and fixed-point ROM contents. See Initialized synchronous ROMs.

Scalar numeric/raw types and recursively bit-packable structs, vectors, and structural tuples are accepted. Nominal enums, including an aggregate containing an enum, are intentionally rejected by `bitcast`, `pack`, and `unpack`. Enum ordinal storage remains available through ordinary typed values and `reshape`, but raw construction could create an invalid ordinal; no validity policy for that conversion is defined.

At explicitly typed assignment/storage boundaries, ZLang may insert an equal-width raw bitcast only when either source or target is `bits<N>` or flat `vec<N,bit>`. This makes raw wiring concise while preserving the rule that direct signed-to-unsigned, integer-to-fixed, resizing, and rescaling conversions are never implicit. Raw conversion does not participate in function arguments, operator overloads, generic inference, switch branch unification, or protocol compatibility.

Scalar `reduce(&, value)`, `reduce(|, value)`, and `reduce(^, value)` reduce the stored bits of `bit`, `bits<N>`, `uN`, or `sN` and return `bit`. `parity(value)` is the concise XOR form and also accepts `vec<N,bit>`. Existing vector reduction still reduces elements and returns the element type; fixed-point and struct scalars are not representation-bit reduction operands.

4.9 Current boundaries

Runtime packed-bit selection, runtime/reversed/out-of-range slicing, zero or unresolved packed widths, one-operand or heterogeneous-vector concatenation, runtime reshape, unequal-width bitcast, enum bitcast, and external top enum inputs remain deliberately rejected. These boundaries are diagnosed rather than inferred by a backend. See the syntax support matrix.

5 Expressions, functions, and generics

ZLang expressions are pure typed hardware values unless they explicitly contain a timing construct. Assignment requires an exact canonical target type.

5.1 Operators and selection

From highest to lowest precedence:

1. calls, parentheses, aggregate selection, conversions, functional forms;
2. unary `-` and logical `!`;
3. `*`, and compile-time `/`;
4. `+`, binary `-`;
5. `<<`, `>>`;
6. `&`, then `^`, then `|`;
7. `==`, `!=`, `<`, `<=`, `>`, `>=`;
8. `&&`, then `||` in compile-time/equivalence guards;
9. right-associative `condition ? true_value : false_value`.

Unary - is implemented for numeric scalar/fixed types and may be overloaded for a nominal struct. It is not a general implicit-cast mechanism. Runtime logical ! accepts exactly bit and returns bit; it is equivalent to an exact comparison with zero. /, &&, and || are available only where the compile-time/equivalence sublanguage accepts them; they do not infer runtime divider or general truthiness for multi-bit hardware values.

Two-way runtime selection can use ?: or mux:

```
y = select ? a : b
y = mux(select, a, b)
```

The condition is bit and both arms have one exact type. Numeric switch requires unique fitting keys and an else arm:

```
y = switch opcode {
    0 => a
    1 => b
    else => 0
}
```

A switch over a nominal enum uses qualified members and must cover every member. It needs no catch-all arm, which keeps a newly added state from silently taking an old default path:

```
next = switch state {
    State.Idle => State.Header
    State.Header => State.Payload
    State.Payload => State.Done
    State.Done => State.Idle
}
```

Numeric keys in an enum switch, members from another enum, duplicate members, and incomplete coverage are errors. Numeric switches retain their existing mandatory else behavior.

Nominal tagged unions use the parallel exhaustive match expression. Each arm names its union and variant and binds the exact source-declared payload names; there is no default arm or hidden priority:

```
y = match message {
    Message.Idle => 0
    Message.Data { value } => value
}
```

Typing lowers this surface to an ordinary exact-width switch over the union tag plus typed field projections before canonical optimization and backend emission.

Exact matching structs, vectors, and structural tuples also support == and !=. These comparisons recursively combine the ordinary exact leaf comparisons and return bit; mismatched nominal structs, vector lengths, tuple component order/arity, or element types are errors.

5.2 Aggregate value ergonomics

Vector literals and replication use their explicit destination type:

```
pair        : vec<2,u8> = [left, right]
zero_values : vec<4,u8> = repeat(0)
```

repeat<N>(value) may state the compile-time length explicitly. The contextual and explicit forms lower to the same existing vector-generation semantics. Struct update is likewise a pure expression over an existing nominal value:

```
next = current with { valid = 1 payload = replacement }
```

It preserves omitted fields and lowers to a complete typed struct construction; it does not create mutable records or a backend-specific update operation. Exhaustive Struct { field, ... } = value destructuring similarly lowers to ordinary immutable field bindings. It requires every field exactly once and does not provide partial

patterns, renaming, or runtime pattern matching. Flat tuple destructuring `(first, second) = pair` likewise evaluates one RHS and introduces fresh immutable projections. Nested, wildcard, and rest patterns remain outside this bounded surface.

5.3 Pure functions

Functions are top-level, pure, and combinational. The smallest body is one result expression:

```
fn tap(sample : u8, coefficient : u8) -> u16 {
    sample * coefficient
}
```

They may call other non-recursive pure functions. They cannot capture module ports or contain state, clocks, delays, protocols, or pipelines. Return types may be explicit or inferred for both ordinary and generic functions:

```
fn make_tag(high : bits<2>) {
    concat(high, zeros<2>, ones<2>)
}
```

An inferred body is checked without a caller-provided expected type, and its exact final-expression type becomes the function signature. Forward references are resolved independent of declaration order. Direct or indirect inferred return cycles are rejected; ZLang does not attempt a recursive type fixed point. For ordinary functions, explicit and inferred spellings with the same exact signature lower to the same callable identity. Generic specialization identity continues to include the generic declaration form and its exact specialization arguments.

A body may introduce sequential immutable bindings without `let`, `const`, or `return`; the final expression is the result:

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Each binding has its exact inferred type, may reference parameters and earlier bindings, and cannot shadow or be reassigned. Forward references and module captures are rejected. This syntax is normalized into the same concrete typed expression/call IR as the nested expression spelling.

Source units are declaration-order independent: functions, structs, enums, and modules may be declared after their users, and a shipped library unit may contain declarations without a hardware top. Parameterized modules can state compile-time obligations at specialization:

```
module Queue<type T,D=4>
where D >= 2 && is_power_of_two(D) {
    // ...
}
```

The constraint is checked against exact type/value arguments and creates no runtime hardware.

5.3.1 Typed constants and statically selected functions

Module and generic-function specializations may carry required named compile-time constants and pure function references:

```
fn widen(x : u8) -> u9 { extend<9>(x) }

fn transform_with<
    type A,
    type B,
    operation : fn(A) -> B
>(x : A) {
    operation(x)
```

```
}

module ImageBank<type T,N,IW,image : vec<N,T>> {
    clock clk
    reset rst
    in address : uint<IW>
    out data : T

    rom table : rom<T,N> { read_latency 1 init image }
    table.read_address = address
    data = table.read_data
}
```

These argument kinds are always named: `image=image` and `operation=fn widen`. A generic target may be selected explicitly, for example `operation=fn widen_as<A=u8,B=u9>`. Constants must have the exact declared, recursively bit-packable non-enum type and must evaluate completely during elaboration. A function reference must be pure and match the exact parameter and return types; it is not a runtime function pointer and cannot be stored, returned, compared, or carried through hardware ports.

Type and integer shape parameters resolve before constant/function parameters. The compiler retains a typed binding record containing the canonical constant value or exact callable identity, dependency closure, evaluator schema, and content digest. Source spelling is absent from semantic, artifact, and cache identity. Defaults for constant/function parameters are unsupported.

5.4 Compile-time functions and selection

Compile-time evaluation is bounded and deterministic. Structural intrinsics are compiler-owned, not stdlib overloads:

- `length(vector)`;
- `floor_log2(n)`, `ceil_log2(n)`, `index_width(n)`, `is_power_of_two(n)`;
- `pi()`, `sin(x)`, `cos(x)`, `log2(x)`, `log(base,x)`, `exp(x)`, and `sqrt(x)`.

Real intrinsics use a versioned decimal evaluator rather than host binary floating point. A non-integral real enters hardware only through explicit fixed-point quantization. `1 / sqrt(x)` is the compile-time reciprocal square root form used when generating seed or approximation tables.

`index_width(n)` is the storage width for an index into `n` elements: `max(1, ceil_log2(n))`. Its argument must be a positive compile-time integer expression. Integer parameter defaults are resolved in declaration order and may reference only earlier parameters:

```
module Table<N=8,IW=index_width(N)> {
    // Ports and implementation omitted.
}
```

A forward reference such as `module Bad<IW=index_width(N),N=8>` is rejected.

Specialization-time `if` accepts value parameters and canonical type parameters:

```
fn type_flag<type T>(x : T) {
    if T == u8 { 1 } else { 0 }
}

module Stage<D=1> {
    out y : u8
    if D == 1 { y = 1 } else { y = 2 }
}
```

Only the selected branch is semantically checked. Runtime signals are rejected in compile-time conditions with a suggestion to use `?:`, `mux`, `switch`, or a guarded action.

5.5 Functional datapath

Ranges are half-open and compile-time bounded:

```
products = generate(i in 0..8) a[i] * b[i]
doubled  = map(i in 0..8) { a[i] + a[i] }
y        = reduce(+, products)
z        = sum(i in 0..8) a[i] * b[i]
d        = dot(a, b)
```

`generate` and `map` produce `vec<N,T>`. `reduce` supports `+`, `*`, `&`, `|`, and `^`; `sum` is addition reduction. Reduction order is a deterministic source-order balanced tree, so finite-width intermediate types are stable across backends. `dot` uses full-width products followed by that same reduction.

For a nominal struct element, only `sum`/`reduce(+, ...)` is defined. Every internal node resolves the ordinary exact `operator +` overload for the two types produced by its children; the overload result is the exact type presented to the next node. Small and otherwise ineligible reductions retain that complete typed tree directly. Eligible large pure bounded regions instead retain a `FunctionalRegion` plus an `ExactReductionPlan`; the plan records the same balanced topology, exact intermediate result types, and resolved overload/callee identity without cloning every call body. These are two storage forms for one language contract, not two reduction semantics.

There is no component-wise field rule, implicit conversion, identity insertion, or special case for `Complex`. exact reduction planning and the e-graph do not reassociate the frozen tree, and explicit quantization remains exactly where the source or overload body placed it.

Range bounds may use resolved module value parameters. Runtime ranges, empty value-producing ranges, and unbounded generation are rejected.

Compile-time iterator arithmetic is mathematical integer arithmetic. It does not require hardware `extend`/`truncate` noise:

```
reversed = generate(i in 0..48) bits[47 - i]
```

For a vector, `values[first..past_last]` is a half-open compile-time range and lowers to the same ordered static element references as the corresponding vector literal. It is not a runtime slice and does not change packed bit-slice syntax `raw[MSB:LSB]`.

A functional iterator is a compile-time integer and may feed a generic value specialization directly or through a bounded constant expression:

```
fn coefficient<K>() -> u8 { K }

values = generate(k in 0..4) coefficient<K=k>()
next   = map(k in 0..4) { coefficient<K=k+1>() }
total  = sum(k in 0..4) coefficient<K=k>()
```

The evaluated value, not the binder spelling, participates in specialization identity. Runtime values are not specialization constants, and an iterator binder cannot shadow a module parameter.

5.5.1 Scalable bounded functional elaboration

Large compile-time `generate`/`map` regions with a binder-invariant result type may use the compact functional representation described above. The compiler still type-checks the region with the ordinary exact rules and counts its logical elements against the same bounded generation budget (currently at most 4096); it does not defer type checking to a backend or create runtime loops. Callable specializations remain ordinary monomorphic definitions, and each use retains its own source origin.

The accepted whole-vector IFFT64 reference exercises an outer 64-lane `generate`, 64 exact products per lane, nominal complex reduction, and one final quantization. N=64 is validated through semantic analysis, canonical round-trip, and the simulator; N=8/N=16 exercise Direct-SV and Verilator. This evidence does not claim full 4096-multiplier N=64 RTL or a production streaming architecture. See the 802.11a validation report for the measured boundary and the later streaming implementation.

A fully compile-time generated vector may initialize an immutable ROM without external source generation:

```
fn coefficients<N>() {
    generate(i in 0..N) truncate<8>(i * i)
}
```

```
rom table : rom<u8,N> {
    read_latency 1
    init coefficients<N=N>()
}
```

The complete transitive function/intrinsic dependency identity participates in the ROM and companion hashes. Compile-time functions still have no filesystem, network, process, clock, state, random, or environment access; only the compiler publishes a validated image. The initializer must resolve to exactly the declared vector shape and element type.

5.6 Typed bit and collection composition

Bit slicing and composition are expressions rather than backend spellings:

```
high   : bits<4> = word[7:4]
flag   : bit = word[0]
joined : bits<8> = concat(high, word[3:0])
same   : u8 = bitcast<u8>(joined)
flat   : vec<8,u8> = reshape(matrix)
```

Slices use inclusive compile-time `[MSB:LSB]` bounds. A single packed-bit selection `raw[index]` returns `bit`, uses conventional LSB-zero numbering, and allows a range-proven runtime unsigned index. A fixed-width packed slice `raw[offset +: WIDTH]` returns `bits<WIDTH>` after the same bounds proof. Vector sequence indexing uses the same low-first convention: `vec` element zero occupies the least-significant packed element region. Scalar `concat` places its first operand at the MSB; homogeneous-vector `concat` preserves collection order and returns a vector. `reshape` changes only nested vector shape. `bitcast<T>` uses the canonical layout described in Types and numerics: struct declaration field zero is at the MSB, while vector element zero and tuple `item0` are at the LSB. It requires exact packed width and performs no resize or scale change. A source or target containing a nominal enum is rejected; enum representation is not a public bitcast contract. `pack` and `unpack<T>` remain low-level compatibility spellings.

For representation-bit checks, `reduce(^, scalar)` and `parity(scalar)` operate on the stored bits of `bit`, raw bits, and signed/unsigned integers. Reducing a general vector retains the existing element-wise reduction semantics.

5.7 Generic structs, functions, and operators

Generic parameters have explicit kinds:

```
struct Pair<type T> { left:T right:T }

fn duplicate<type T>(x:T) { x + x }

module Use {
    in x : u8
    out y : u9
    y = duplicate(x)
}
```

Calls support exact call-site inference or explicit specialization. Generic struct constructors infer their specialization from fields where unambiguous. Every accepted call is monomorphized; there is no runtime dispatch.

A direct integer literal may use a concrete formal argument type after explicit generic arguments or non-literal arguments have fixed that type:

```
first  = same<T=u16>(1)
second = combine(1, existing_u16)
third  = combine(existing_u16, 1)
```

If a type is inferable only from literals, each literal first receives its own minimum-width exact type and normal exact unification applies. Argument order does not select a wider type. Compound expressions never receive this literal-only contextual treatment.

Libraries may overload binary `+`, `-`, `*` and unary `-` for nominal structs. Built-in scalar/fixed operations cannot be replaced. Resolution uses exact type unification, prefers an exact concrete overload over a generic one, performs no implicit conversion, and enforces nominal-owner coherence. Specialized bodies are retained once as deterministic monomorphic callable definitions. Each use site is a typed `Call` carrying the exact callee identity, signature, result type, and its own source origin. Analyses that require concrete arithmetic use the shared bounded call-expansion service; nominal reductions remain opaque to exact reduction planning and the frozen e-graph rewrite set. Direct-SV emits one `function automatic` per specialization rather than cloning the body at every call site.

`std.math.complex` is ordinary ZLang source defining `Complex<T>`, arithmetic operators, `Butterfly<S,D>`, and explicit component quantization. For example:

```
Complex<fixed<18,16>> * Complex<fixed<16,14>>
    -> Complex<fixed<35,30>>
```

The scale-30 result cannot be added to a scale-16 value until explicitly quantized. Its `complex_sum(values)` helper is a discoverability alias that returns the same typed `Reduce` as `sum(values)`; it adds no compiler behavior. Generics and nominal reductions do not imply rescaling, rounding, saturation, or casts.

Traits, runtime polymorphism, generic methods, function-name overloading, higher-order functions, recursion extensions, and implicit conversions are unsupported.

6 Sequential state, rules, pipelines, and storage

ZLang keeps combinational values, next-state actions, timing, storage, and physical clock ownership explicit in typed IR. Synchronous active-high reset is the default. Each independently declared clock has one reset contract and may own ordinary state in the same module.

6.1 Clock and reset

Single-domain modules declare one pair:

```
clock clk
reset rst
```

Because newlines are ordinary whitespace, the compact adjacent spelling is equivalent and introduces no combined declaration or inferred domain:

```
clock clk reset rst
```

Clock and reset are declared together. Multiple domains require explicit reset, port, and state ownership wherever inference would be ambiguous:

```
clock source_clock
reset source_reset @source_clock

clock destination_clock
reset destination_reset @destination_clock

reg source_count : u32 @source_clock = 0
reg destination_count : u32 @destination_clock = 0

rule TickSource @source_clock when 1 {
    source_count <- truncate<32>(source_count + 1)
}

rule TickDestination @destination_clock when 1 {
    destination_count <- truncate<32>(destination_count + 2)
}
```

An unannotated stateful declaration remains concise in a single-clock module. In a multi-clock module its domain may be inferred from one unique destination or dynamic operand domain; otherwise analysis reports `ZL`

-DOMAIN-AMBIGUOUS. Constants are domain-neutral. Combining dynamic values from different domains, including through an unannotated local, reports ZL-DOMAIN-CROSSING.

Rules and priorities are scheduled within a domain. Rules in unrelated domains do not acquire cross-clock mutual exclusion or atomicity. FSM-generated state and rules inherit the FSM annotation. Exact pipeline(N) @clk { ... }, FIFO, memory, ROM, and CSR state similarly retain one resolved owner; inserted pipeline and balancing registers never act as CDC.

Implicit clock-domain crossing is always rejected. See Hierarchy and protocols.

The recommended asynchronous spelling asserts immediately and releases only after two active clock edges:

```
clock clk
async reset arst_n @clk { polarity active_low }
```

Normal state transition resumes on the third edge after external deassertion. Use the externally synchronized release contract only when the external source already guarantees synchronous deassertion to this exact clock. It removes the internal 2FF while retaining asynchronous reset for state with a declared reset value. The low-level reset ... { mode asynchronous ... } spelling retains raw native deassertion for compatibility. Exact polarity, falling-edge behavior, hierarchical propagation, and unsupported formal/target routes are documented in the physical clock/reset contract.

6.2 Registers and next state

```
reg count : u8 = 0
count <- truncate<8>(count + 1)
```

The explicit multi-clock spelling places the annotation before the initializer:

```
reg count : u8 @datapath_clk = 0
```

Expressions read committed beginning-of-cycle state. All accepted <- updates become visible together at the active edge. A register without an accepted update holds. Reset restores the declared constant. Multiple unordered writers are rejected.

The reset value is optional. reg retained : u8 has no generated reset branch or reset sensitivity; its hardware value is unspecified until written. Native two-state simulation starts it from a deterministic zero seed for tool parity, which is not a power-up or reset guarantee in generated RTL.

Use zlang sim --logic-state to opt into the per-bit simulation lattice 0/1/U/X. State without an initializer begins U (never refreshed). An accepted state write converts unresolved U bits to X (refreshed but unknown), while a non-write holds U. Exact controlling values still resolve: U | 1 and X | 1 are 1, while U & 0 and X & 0 are 0. --strict-uninitialized implies logic-state execution and turns a typed read of either U or X into an error that identifies which state was observed.

Python callers may use set_logic, get_logic, outputs_logic, and Program.create(initial_registers=...). Packed spellings are MSB-first, case-insensitive 0, 1, u, or x; a single character broadcasts to the full width. Register names are resolved through the compiler-owned state catalog, including explicit child paths such as child.counter; ambiguous or non-register names fail before any state is changed. An edge that fails because of an unknown stateful control or native execution error rolls back logic state, event flags, and trace state atomically. CLI equivalents are --set-logic and --initial-reg. Standard VCD represents both U and X as x; ZLang emits a same-width companion __zlang_meta/*_u_mask whose set bits distinguish U. --trace-signal may be repeated to select the traced surface, and the initial sample is emitted at time zero. The VCD is an event-index visualization: each JSONL event occupies two 1 ns trace ticks, reset changes precede that event's selected clock edges, and falling-edge, active-low, and coincident-domain levels follow the compiled domain contract. It does not reconstruct analog time or simulator delta-cycle regions that are absent from the event schedule.

This is not full Verilog simulation: high impedance (Z), electrical resolution/strength, and Verilog event-region semantics are not modeled. Ordinary simulation remains deterministic two-state simulation, and generated Direct-SV is unchanged by these simulator options.

An ordinary output is a combinational wire. Output-visible state is declared explicitly with out reg:

```
out remaining : s32              // combinational wire
```

```
out reg accepted : s32 = 0       // resettable stored output
out reg retained : u8 @data_clk  // stored output with no reset branch
```

A registered output is one state resource and the public output port at the same time. It is readable inside the module, accepts `<-` updates, and holds when no accepted action updates it. Its optional initializer is its reset value; omitting it has the same unspecified-hardware-reset semantics as an ordinary register without an initializer. The type must be register-storable. Ready/valid, credit, packet, request/response and other protocol endpoints are not `out reg` ports; their individual protocol fields retain their existing ownership and flow-control rules.

ZLang `out reg` names actual architectural storage. It is not the historical Verilog/SystemVerilog `output reg` declaration category: a ZLang registered output has hold behavior, one explicit state owner, and the same clock/reset and atomic-update rules as an ordinary ZLang register.

Control registers may use nominal enums. Reset and next-state values must be members of the exact same declaration, and an exhaustive enum `switch` is the preferred finite-state-machine spelling:

```
enum Phase { Idle Active Done }
reg phase : Phase = Phase.Idle
phase <- switch phase {
    Phase.Idle => start ? Phase.Active : Phase.Idle
    Phase.Active => finish ? Phase.Done : Phase.Active
    Phase.Done => Phase.Idle
}
```

The semantic simulator and Direct-SV share the declaration-order ordinal encoding. The source-authored APB bridge in `stdlib/bus/apb.zhl` uses an enum for its internal phase without exposing it through the external bus ABI.

For control-oriented state, `fsm` is concise syntax for the same enum register and guarded atomic rules:

```
enum TxPhase { Idle Header Payload Done }

out reg remaining : u32 = 0

fsm phase = TxPhase.Idle {
    Idle {
        when start -> Header { remaining <- length }
    }
    Header { when header_sent -> Payload {} }
    Payload {
        priority {
            when abort -> Idle {}
            when last -> Done {}
        }
    }
    Done { -> Idle {} }
}
```

The qualified initial member in `fsm phase = TxPhase.Idle` infers the exact nominal enum type. The explicit compatibility spelling `fsm phase : TxPhase = Idle` remains accepted; neither form infers from guards, actions, or transition targets.

Every member appears exactly once. A state body is `hold`, one transition, or an explicit `priority` block; source order never creates hidden priority. The state update and transition actions form one ordinary atomic rule, reset restores the declared initial member, and direct writes to the generated state register are rejected. `fsm` is lowered before canonical/backend IR and does not add a second scheduler or runtime procedural `if`.

The production Wi-Fi `IeeeIFFTInputStrip` is a second source-authored witness: its `Data`/`Flush` FSM atomically starts and retires a 64-transfer zero-frame flush, holds under backpressure, and returns to `Data` after the last accepted sample. Mid-flush reset restores the declared initial state and discards the incomplete protocol epoch.

6.2.1 Runtime-indexed vector register updates

A one-dimensional vector register supports one range-proven runtime element write in an action group:

```
reg samples : vec<64,u8> = generate(i in 0..64) 0

when capture {
    samples[index] <- value
}
```

This is exactly `samples <- VectorUpdate(samples,index,value)` over the beginning-of-cycle snapshot. The unsigned index must be statically proven in range. A second dynamic or static write to the same vector register conflicts; nested paths, slices, runtime-selected instances, and automatic memory inference remain outside this bounded form.

A rule guard may establish the required bound through a simple unsigned comparison and conjunction:

```
when (index < 64) & capture {
    samples[index] <- value
}
```

That fact is scoped to this rule only. The bounded refinement understands these simple comparisons/conjunctions; `index <= 64`, disjunctions, and uses outside the guarded rule do not prove the required `0..63` range.

6.3 Delays and fixed pipelines

```
delayed = delay<2>(value)
piped = pipeline(3) { expression }
```

`delay<N>` adds exactly `N` zero-reset scalar stages. `pipeline(N)` is an exact visible-latency contract: for a supported pure scalar expression DAG, planning may place real computation boundaries within those `N` cycles and inserts the required reconvergent-path balancing. It never changes the requested latency or II, and `pipeline(1)` remains exactly one cycle. Unsupported nodes fail closed; the emitter does not fall back to an unreported whole-expression output delay. Outside a scheduled pipeline region, non-timeless operands at an operator must still be latency-aligned explicitly.

Compiler-selected latency/resource alternatives use `implement { expression intent { ... } }`. Scalar `pipeline(auto)` is retired; protocol `transform pipeline(auto, ...)` is a separate ready/valid construct.

6.4 Exact module timing contracts

A scalar datapath module may publish one exact contract shared by all of its wire outputs:

```
module TimedChild {
    clock clk
    reset rst
    in x : u8
    out y : u8

    timing {
        latency 4
        ii 1
    }

    y = pipeline(4) { x }
}
```

The contract describes observable behavior; it does not itself insert registers or authorize retiming. Constants are timeless, ordinary input-dependent logic is known at latency zero, and explicit `delay<N>`/`pipeline(N)` adds exactly `N`. A supported pipeline region may use its compiler-owned scheduled graph to place and balance those internal cuts. Timeless values may join a known path; joins outside that graph still require equal known latency.

Registers, rules, FIFO/memory/ROM observations, protocols, and uncontracted child outputs are `unknown` under the current bounded contract. A positive latency requires exactly one clock/reset domain. Only `ii 1` is accepted. A contracted child adds its declared latency to the common known latency of its bound scalar inputs; an unaligned parent join is rejected instead of silently balanced.

This is distinct from `implement` constraints, target estimates, and measured evidence. A planner or profile must preserve an exact module contract as immutable public behavior.

6.5 Guarded atomic actions

The concise action form is:

```
when increment {
    count <- truncate<8>(count + 1)
}
```

`<-` is exclusively a state update. It may target an ordinary register, an `out reg`, or a supported indexed vector-register element. An ordinary `out` wire is never stored and therefore cannot be updated with `<-`. A transient rule/FSM output uses `drive` instead:

```
out fired : bit

rule publish when accept {
    drive fired = 1
}
```

The driven wire is zero when no owning action fires. It participates in the same atomic scheduling, conflicts and explicit priority as the rule's other effects. `drive` cannot target a register or `out reg`, and a registered output cannot also have a combinational assignment. Because changing a wire update to stored state is a semantic choice, the compiler reports both alternatives but does not apply an automatic editor fix.

`drive` is a logical transient output assignment. It does not model a tri-state driver, high impedance (`Z`), electrical or pad drive strength, analog behavior, or Verilog multiple-driver resolution. Its zero default is therefore pulse/event semantics, not retained state.

Form	Cycle semantics	What it is not
`out value : T` with `value = expression`	Continuously defined combinational value	Stored state
`out reg value : T [@clock] [= reset]` with `value <- expression`	Clocked architectural state that holds when not updated	Verilog's historical output declaration category
`drive value = expression` in an action	Scheduled transient wire effect; zero when no owning action fires	Tri-state/electrical drive or implicit storage

Named compatibility spelling remains supported:

```
rule increment_count when increment {
    count <- truncate<8>(count + 1)
}
```

Action blocks may select effects recursively with runtime `when`, `else when`, and `else`:

```
fault_update: when fault {
    when valid_state {
        overflow_state <- 1
    } else {
        valid_state <- 1
        overflow_state <- 0
        sticky_state <- fault_code
    }
} else when clear_valid {
```

```
    valid_state <- 0
    overflow_state <- 0
} else when clear_overflow {
    overflow_state <- 0
}
```

An `else` binds to the nearest unmatched `when`, and an `else when` chain selects its first true predicate. A false `when` without an `else` contributes no effects; later statements in the enclosing action block are still considered, so multiple sibling conditionals can contribute to the same transaction.

Read this as two operations that never feed back into each other: predicates first choose the effects, then the scheduler either commits the complete chosen transaction or commits nothing. Storage readiness and rule conflicts belong only to the second operation; they can suppress a transaction but can never make an `else` branch run.

Every guard and operand reads the same pre-edge snapshot. The complete tree retains one outer `Rule`, one rule-fire identity, and one `ActionGroup`; selected effects and unconditional surrounding effects commit together or not at all. If no effect is active, the rule does not fire. Source order creates neither state visibility nor implicit priority, and reset suppresses the complete group.

Branch choice depends only on the predicates. If the selected branch contains an illegal FIFO or memory action, the whole action group is suppressed; it does not fall back to an `else` branch because another branch's storage action would be ready. The scheduler checks conflicts and storage legality using only active effects. Explicit `drive` output writes are scheduling resources too: a rule that loses a priority conflict on an output cannot commit its otherwise unrelated state effects.

Each runtime action condition must have exact type `bit`. Opposite structured branches may write the same resource, but writes on potentially overlapping paths are rejected; the compiler does not attempt arbitrary Boolean theorem proving. An explicitly empty branch is a valid no-op, while an entire action tree with no possible effect is rejected.

Runtime action selection is distinct from both other selection forms:

- compile-time `if` chooses declarations or values during elaboration and is not an action statement;
- `condition ? a : b`, `mux`, and `switch` select pure combinational values;
- runtime `when` selects effects that participate in one cycle-level atomic transaction.

Rules read beginning-of-cycle state. If the guard is true, all resource actions are legal, and scheduling permits the rule, its actions commit atomically. Reset suppresses all rules. Conflicting writers require explicit priority:

```
priority {
    clear: when clear_request { count <- 0 }
    increment: when increment_request {
        count <- truncate<8>(count + 1)
    }
}
```

Priority suppresses a lower enabled rule only when it conflicts with a fired higher rule. It is not procedural `if`/`else`: nonconflicting enabled rules may fire together.

A simple ordered chain may instead be written as:

```
priority first > second > third
```

This syntax expands to the existing adjacent edges `first > second` and `second > third`. It does not invent extra edges or infer priority from source order; retain the block/pair form for any other priority graph.

6.6 FIFOs

```
fifo queue : fifo<u8,4>
queue.data = rx.payload
queue.push = rx.transfer
queue.pop = tx.transfer
```

Read-only observations include `.front`, `.count`, `.full`, `.empty`, `.ready`, `.valid`, `.overflow`, and `.underflow`. A legal simultaneous pop/push while full preserves occupancy and ordering. An empty pop cannot consume a same-cycle push.

FIFO depth may be a positive compile-time value expression. Backends receive the resolved integer, not a hardware parameter.

Rules can instead own FIFO operations atomically:

```
rule replace when rotate {
    queue.pop()
    queue.push(queue.front)
    accepted <- 1
}
```

Do not mix rule-owned actions with globally driven `.push`/`.pop` controls. An illegal resource action suppresses the whole rule.

Rule scheduling selects the lexicographically highest-priority **legal atomic set**, not each rule independently. Thus a push on a full FIFO may fire with a compatible lower-priority pop on the same edge; a push alone remains blocked. The compiler constructs exact selection regions for empty, interior, and full occupancy without enumerating every count or rule subset. If exact scheduling exceeds its bounded work budget, compilation reports `ZL-STATE-SCHEDULER-LIMIT` rather than emitting an approximate circuit.

6.7 Writable memories

One-read/one-write memories may optionally expose a byte write mask. The existing one-cycle, reset-cleared form remains the default:

```
in write_mask : bits<4>
memory table : mem<u32,256> {
    read_latency 1
    collision write_first
}
table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
table.write_mask = write_mask
```

The mask type is exactly `bits<ceil(element_width / 8)>`. `write_mask[0]` controls the least-significant byte. For a width that is not a multiple of eight, the final mask bit controls only the remaining live most-significant bits; padding bits are never stored. Disabled lanes retain their old bits. For a same-address `write_first` access, the registered read result is the post-mask merged word, not the unmasked input. Omitting `write_mask` preserves the existing full-word write behavior. Consequently, unmasked and masked memory elements may have any positive recursively bit-packable, non-enum width.

Rule-owned memories accept the same operation as an optional third operand:

```
rule store when enable { table.write(address, data, byte_mask) }
```

Within a masked rule-owned memory, the compatible two-operand form `table.write(address, data)` means an all-lanes write. The mask does not add a read enable, another port, a new clock domain, or target-specific BRAM mapping.

`read_latency` counts active edges in the memory's read domain between a captured address and the visible result. These are compiler/simulator/Direct-SV contracts, not promises that a particular FPGA RAM primitive will be inferred:

Storage form	Accepted `read_latency`	Tested behavior and physical boundary
Globally controlled `mem`, including named same-clock ports	`0..16`	`0` is combinational; `1` is registered; `2..16` add exact read-domain register stages. Cycle tests cover `0`, `1`, `2`, and the upper bound `16`; Direct-SV is linted.
Rule-owned legacy `mem`	Exactly `1`	A selected read action captures at an edge and holds its result otherwise; rule/storage simulations and Direct-SV tests cover it.
`async_mem` 1W1R	`1..16`	Reader-domain registered output; cycle tests cover `1`, `2`, `3`, and `16`, including unrelated clocks. Generic Direct-SV supports `old`; stronger cross-clock collision claims need exact target evidence.
Immutable `rom`	Exactly `1`	Synchronous table read; compile, simulation, and Direct-SV tests cover it.
Internal memory of `async_fifo`	Fixed `1`	The FIFO prefetch controller accounts for the registered read; this is not a user-settable FIFO latency.

The current explicit Xilinx 7-Series synchronous-memory physical route accepts only `read_latency 1` and matching reset, mask, port, and collision capability. Other legal latencies can use generic RTL under a preferred policy; a required native route rejects them. `read_latency 0` never claims synchronous BRAM.

6.7.1 Named same-clock ports

Leaving the body without port declarations preserves the legacy controls and their generated behavior. A memory may instead declare up to eight named logical ports:

```
memory table : mem<u32,1024> @clk {
    read_write_port a
    read_write_port b
    init RESET_WORD
    read_latency 1
    collision old
    write_priority a > b
}

table.a.read_enable = a_read
table.a.write_enable = a_write
table.a.address = a_address
table.a.write_data = a_data
a_data_out = table.a.read_data
```

`read_port`, `write_port`, and `read_write_port` state the exact access shape. All named ports of an ordinary `mem` have one resolved clock domain. When more than one port can write, `write_priority` must list every writer exactly once. Priority affects only writes to the same address; simultaneous writes to different addresses both commit. `collision old`, `new`, and `no_change` select the same-clock registered-read result. The older spellings `read_first` and `write_first` remain accepted aliases for `old` and `new`.

Named-port masks retain the arbitrary-bitwidth lane rule above. Mixed port widths and asymmetric depths are rejected. Rule-local memory actions remain a property of the legacy implicit 1R1W form and cannot be mixed with named ports.

An optional `init VALUE` supplies one exact compile-time element value for every cell. It may be a typed module value parameter. With `contents clear`, generic/register-array hardware writes that value to every cell on reset. With `contents preserve`, it is only the deterministic power-up value; FPGA block RAM maps it to bitstream initialization and does not pretend that a runtime reset rewrites the array. Omitting `init` retains the exact zero value used by the existing reset/initialization policies. Full per-address images remain the role of immutable `rom` in the current language contract.

The bounded implementation planner first uses an exact target resource. A `1W+nR` shape otherwise becomes coherent replicated 1R1W storage; other same-clock multiwrite shapes may use a deterministic register array, read muxes, and priority gates when the logical storage is at most 4096 bits. Larger unsupported shapes fail with a diagnostic recommending explicit banking, arbitration, or standard-library composition. No automatic banking is performed because arbitrary same-bank collisions need a protocol contract.

6.7.2 Explicit dual-clock memory

`async_mem<T,N>` is a separate, deliberately narrow semantic resource:

```
memory table : async_mem<u32,1024> {
    write_port wr @write_clk
    read_port rd @read_clk
    init INITIAL_WORD
    read_latency 1
    collision old
    reset { contents preserve read_data clear }
}

table.wr.enable = write_enable
table.wr.address = write_address
table.wr.data = write_data
table.wr.mask = write_mask
table.rd.address = read_address
read_data = table.rd.data
```

It has exactly one write-only port and one read-only port in different explicit domains. Cells change only on the writer edge; `rd.data` changes only on the reader edge and has destination-domain provenance. The contents reset policy belongs to the write domain and the read-result policy to the read domain. Direct use of foreign-domain controls is rejected; a pipeline register is not a CDC primitive.

`async_mem` requires `read_latency` in `1..16`. The generic implementation captures the cell on the reader edge and, for latency greater than one, shifts the captured value through reader-domain output registers. This adds visible latency without changing the contents or crossing domains. `read_latency 0` is invalid for `async_mem`; it would be an unregistered cross-clock read, not a block-RAM read port.

For coincident logical edges the simulator uses a pre-edge snapshot: `old` returns the prior cell, while `new` forwards the coincident write. This is a precise digital model, not an analog metastability or silicon timing guarantee. Portable generic Direct-SV publishes `old`; stricter or other collision claims require an exact target capability. Asynchronous 2RW, mixed-width ports, ECC, and dual-clock ROM remain deferred.

```
memory table : mem<u8,16> {
    read_latency 1
    collision read_first
}

table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
read_data = table.read_data
```

Alternatively, one memory can be owned by atomic rules:

```
rule fetch when read_enable {
    table.read(address)
    accepted <- 1
}

rule store when write_enable {
    table.write(address, write_data)
}
```

`read` registers `table.read_data` one cycle later and it holds when no read fires. `write` commits at the selected edge. One selected read and one selected write may fire together; same-address behavior follows the declared collision mode. Their operands and all other rule effects observe the same pre-edge snapshot. Reset suppresses actions. With the default profile it clears both cells and `read_data`.

Global controls and rule actions cannot be mixed. The bounded scheduled form supports one memory per module, alongside registers and scheduled FIFOs; two reads or two writes conflict and require existing explicit rule priority.

Executable memory elements may be recursively bit-packable non-enum aggregates, as well as scalars; depth is a power of two and at least two. A globally controlled ordinary `mem` accepts `read_latency` in `0..16`: zero is a combinational read; each positive value is an exact number of domain-local read edges from address capture to the visible result. Rule-owned memories remain exactly one-cycle because their read action is selected at an edge. Zero-cycle ordinary `mem` is a generic combinational storage behavior, not a native synchronous BRAM claim. Native target binding requires an explicitly matching clocked-read capability. Until a target mapping proves its internal output-register configuration and any additional fabric stages, a requested latency greater than the advertised native latency uses generic storage under a preferred policy or fails under a required policy.

Cell and visible read-result reset behavior may be selected independently:

```
memory table : mem<u32,256> {
    read_latency 0
    collision read_first
    reset {
        contents preserve
        read_data preserve
    }
}
```

If the `reset` block is present, both directives are required. If it is absent, the exact default is `contents clear` and `read_data clear`. Reset always suppresses writes and scheduled actions. At latency zero, `read_data clear` masks the combinational result while reset is asserted; `read_data preserve` leaves the addressed preserved cell visible. At latency one, the corresponding policy clears or holds the result register. `read_first` observes the old cell before a coincident write edge; `write_first` observes the fully byte-mask-merged write word.

The simulator and generic Direct-SV initialize executable preserved memory state to its declared uniform `init` word, or zero when omitted, but runtime reset does not recreate that initialization under `contents preserve`. The Xilinx same-clock true-dual mapping therefore requires preserved contents; Vivado maps its source initializer to BRAM initialization. Target selection stays fail-closed when a resource does not advertise initialization support. Full/partial writable-memory images, automatic banking, unbounded multiport memory, and asynchronous 2RW are not part of this hardware surface. Bounded named-port selection is described above. Simulation-only tests may instead publish a selected-IR-bound state catalog and Verilator VPI companion with `--simulation-state-bundle`; that tooling neither adds hardware ports nor changes memory reset semantics. The writable-memory contract and public simulation-state access sections define the two distinct boundaries.

6.7.3 Replicated read ports, wrappers, and banking

Named ports now provide a bounded native logical model; source composition is still preferable when banking or arbitration policy is architecturally significant. The validated `ZtpuBankedMemory` uses four banks and two read replicas per bank: both replicas receive the same decoded, byte-masked synchronous write, while each read address selects its own replica and bank. This produces eight distinct physical 1R1W memories, one shared leaf specialization, two combinational read results, and one logical write port.

The concrete witness contains 256 32-bit words, split into four banks of 64 words. Its runtime reset suppresses writes and preserves both cells and combinational read data; executable simulation begins from deterministic zero contents. The semantic simulator and Direct-SV agree on full and per-byte writes, independent reads, same-address `read_first` behavior, reset, and replica coherence. This is source composition, not a new memory semantic or backend name-based rewrite. The ordinary stdlib modules `StorageDualPortMemory`, `Storage2R1WMemory`, and `StorageAsyncMemory1W1R` expose common shapes while lowering through the same memory IR. `StorageAsyncFifo` similarly wraps the existing ready/valid `async_fifo(D)` crossing; a normal `fifo<T,N>` never becomes asynchronous.

The FIFO crossing now decomposes into an internal typed `async_mem` with independent write/read clocks, `read_latency 1`, and a one-slot prefetch controller. It is not a second public memory declaration in the mod-

ule. The selected AMD 7-Series `Xilinx7AsyncFifoRAMB18` or `Xilinx7AsyncFifoRAMB36` architecture binds only this internal FIFO memory to a matching independent-clock 1W1R RAM with `DO_REG=0`; unsupported width/depth/-configuration fails under `required` policy and remains generic RTL under `preferred`. This route does not imply that arbitrary public `async_mem` has a native cross-clock collision guarantee.

Target-memory closure remains separate. In particular, the existing promoted OpenRAM/target macro contract does not yet match this zero-latency, preserve-on-reset profile, so the witness makes no BRAM/SRAM inference, QoR, or physical-macro claim. A byte-addressed external wrapper and target-specific latency/collision adaptation are not currently supported by this mapping.

6.8 Initialized synchronous ROMs

An immutable lookup table is declared separately from writable memory:

```
rom twiddles : rom<Complex<Twiddle>, N / 2> {
    read_latency 1
    init fft_twiddles<T=Twiddle,N=N>()
}

twiddles.read_address = phase
selected_twiddle = twiddles.read_data
```

The initializer must specialize at compile time to exactly `vec<depth,element_type>`. Elements may be bit-packable scalar/fixed values, vectors, or concrete structs; protocol/state values and nominal enums are rejected. Depth is positive and may be one or non-power-of-two. The address is an unsigned `uint<max(1,ceil_log2(depth))>` value whose proven range remains inside the declared depth.

The read is a **one-cycle synchronous read**. During reset the registered read result is zero. Reset never changes the immutable contents; after reset, the first non-reset result corresponds to the preceding accepted non-reset address. There is no hidden second output register.

The production backend consumes one compiler-owned companion image. It contains one exact-width binary word per line, address zero first. Struct declaration field zero occupies the most-significant bits; vector element zero and tuple item zero occupy the least-significant component bits recursively. Fixed-point values retain their raw signed or unsigned bit pattern. Direct-SV `$readmemb` consumes that image. The CLI publishes companions beside the selected output, and artifact metadata retains the initialization dependency, evaluator, content, and file hashes. Missing or colliding companion files fail closed.

Writable or partial initialization, reloadable/asynchronous/multiport ROM, enum elements, and automatic BRAM/storage exploration remain deferred.

6.9 Stateful hierarchy

Single-domain scalar child modules may contain registers and rules. A compile-time indexed instance array may contain bounded scalar sequential or primitive ready/valid children with one matching clock/reset domain; each physical instance has independent state and reset. Aggregate scalar outputs and mixed scalar/ready-valid ports retain their exact typed leaves. Storage-only child arrays may own one FIFO, synchronous memory, or initialized ROM. The scheduled-FIFO profile may also combine FIFO actions with ordinary register writes/rules because both use the same `ResolvedTransition`; legacy globally controlled storage plus user state remains rejected.

One outer array may contain a bounded same-domain scalar or direct-ready-valid hierarchy, including an inner scalar compile-time array. First-level in-order request/response requester/responder arrays are supported with scalar wire peers and explicit indexed connections. Nested storage/CSR/aggregate protocols, nested or out-of-order request/response, other non-RV protocols, CDC, runtime-selected instances, and cross-module atomic scheduling remain fail-closed. See Hierarchy and protocols and storage-owning instance arrays.

7 Physical clocks and resets

ZLang's backend-independent `ClockDomain` records the complete physical contract of one clock and its reset. Concise declarations retain the default reset contract:

```
clock clk
reset rst @clk
```

normalizes to rising-edge clocking, synchronous active-high reset, native release, and unspecified power-up state.

7.1 Recommended asynchronous reset

The concise asynchronous form is safe for ordinary state. In a multi-clock module each annotated domain owns an independent conditioner:

```
clock clk
async reset arst @clk
```

It means asynchronous assertion followed by deassertion synchronized through two registers. Reset remains active on the first two active clock edges after the external pin is deasserted; normal state transition resumes on the third edge. Reassertion at any point immediately restarts that release sequence.

The external pin may be active-low:

```
clock clk
async reset arst_n @clk {
    polarity active_low
}
```

If reset deassertion is already synchronized to this exact clock outside the generated module, state that physical contract explicitly:

```
clock clk
async reset arst_n @clk {
    polarity active_low
    release externally_synchronized
}
```

This retains asynchronous assertion for resettable state but emits no internal two-register release conditioner. The annotation is never inferred: every destination clock domain must receive a release synchronized to that clock. Omitting `release` retains the safe internal 2FF default.

A register participates in reset only when it declares a reset value:

```
reg control : u8 = 0 // asynchronous reset to zero
reg datapath : u8    // no reset branch or reset sensitivity
```

The unreset register continues to accept enabled clocked updates while reset is asserted. Its RTL power-up value is unspecified until the design writes it. Native two-state simulation uses a deterministic zero startup seed, which is not a source-level power-up or reset guarantee. The opt-in logic-state mode represents unrefreshed bits as `U`; strict observation rejects `U` or computed `X` rather than collapsing either to a binary value. See Registers and next state.

A falling-edge clock uses falling edges for the two release cycles as well. Polarity affects the external pin and generated RTL; the simulator still accepts a logical `True` for asserted reset.

The lower-level compatibility spelling remains available for a raw native asynchronous assertion/deassertion contract:

```
clock clk { edge falling }
reset rst_n @clk {
    mode asynchronous
    polarity active_low
    power_up unspecified
}
```

This full block does not insert synchronized release. When either low-level physical block is present, every clause in that block is required. Raw active-low protocol helpers gate transfers with the deasserted high level; they do not reinterpret the physical pin as active-high.

7.2 Backend lowering

Direct-SV emits a deterministic two-register synchronizer marked with `ASYNC_REG` for the unannotated concise form. The top-level domain owns it and routes one conditioned reset through register/rule, FIFO, memory, CSR, protocol, and child state. A hierarchy does not create one synchronizer per sibling. A clocked but completely state-free module publishes the same physical contract without emitting an unused conditioner, because no reset epoch is consumed.

Within a composed artifact, each child receives the top's conditioned net and uses it directly as its asynchronous reset. Children and nested grandchildren never add another release conditioner, so hierarchy depth does not add reset release latency. The same reusable child compiled as a public top owns its own conditioner, because that compilation has a new external reset boundary.

`release externally_synchronized` routes the external reset directly into the asynchronous event control of resettable state and emits no conditioner. Resetless registers use only the active clock event.

The semantic simulator models each input item as one active edge. It therefore models immediate assertion at the sampled boundary and the exact two-edge release hold. Assertion between active edges is additionally checked in RTL simulation.

The generated implementation artifact records the contract identity, external RTL clock/reset paths, edge, assertion mode, polarity, release policy/cycles, power-up policy, and source origin. Component and physical-instance records reference that identity.

7.3 Formal applicability

Executable formal work consumes the exact declared contract. With `power_up unspecified`, the supported combinations are described below.

Formal-only accepted `rule.fire` projections use the effective reset and polarity, not a raw active-high input assumption. Descendants consume the conditioned native reset. Rising and falling active edges, active-high and active-low pins, and synchronous or asynchronous assertion are supported. An asynchronous reset may use native or exactly two active edges of synchronized release. Applicable routes are source safety/cover, bindable recursive safety, same-cycle or fixed-latency II=1 Direct-SV semantic-reference equivalence, and formal-aware selection. Asynchronous formal execution is limited to one physical domain.

Multiple supported synchronous domains may still produce independent goal-local jobs. Asynchronous execution is deliberately single-domain; this does not define a cross-domain reset relation. Same-cycle pure candidate equivalence remains reset-independent, while fixed-latency comparison uses the declared active edge and release window.

The compiler carries the same typed clock/reset contract from the selected design through RTL bindings, formal jobs and their results. Restoration rejects missing, corrupted or mismatched contracts rather than guessing a reset mode.

Prepared-artifact, bundle, harness, result-cache, and evidence recipes therefore separate otherwise identical goals that use different edges, assertion modes, polarities, or release policies. A non-executable goal still retains its exact typed contract. If no compatible artifact exists, its physical-domain identity is absent rather than guessed.

Formal traces distinguish the two reset views. `physical_reset` is the raw external pin with its declared polarity. `trace:reset` and result `reset_state` are the normalized active-high *effective* reset used by property guards, previous-cycle history, feasibility covers, fill masks, and comparison windows. For synchronized release, effective reset remains asserted for both release edges even though the external pin is already deasserted. Failure-cycle and originating-sample attribution therefore describe the real reset epoch, not merely the raw pin level.

7.4 Boundaries

One clock domain has exactly one reset. Declaring both `reset` and `async reset` for that domain is an error. Direct-SV supports multiple independently conditioned asynchronous domains; current formal execution still skips an asynchronous goal in a multi-domain module because no cross-domain reset-epoch relation is claimed. Implicit reset crossings, reset combiners, configurable synchronizer depth, glitch filters, power-on reset, and macro-selected RTL semantics are not supported.

`power_up reset` remains typed but fails closed because no common portable synthesizable initialization mechanism is frozen. DSP48 physical reset pins, elastic or variable-latency `pipeline(auto)` equivalence, CDC/reset-

refinement proofs, and target BRAM reset pins remain fail closed. So do multi-domain asynchronous formal execution, general hierarchical semantic-reference equivalence, and every route with missing or incompatible domain manifests, bindings, assumptions, or observations. formal-aware selection `available` records unavailable evidence without changing eligibility; required policies fail unless the existing exact semantic-reference equivalence route executes at the requested level. No new formal property, observation, or equivalence family is introduced.

8 Hierarchy, protocols, and explicit CDC

ZLang represents modules, endpoints, ownership, clock domains, connections, and physical instances explicitly before backend lowering. Connections are never inferred by guessing generated RTL names.

8.1 Modules and instances

Modules may have compile-time type and value parameters:

```
module Engine<type T, DEPTH=4> {
    in  value : T
    out copy  : T = value
}
```

Instantiate and bind scalar inputs explicitly or by same-name shorthand:

```
inst engine : Engine<T=u8,DEPTH=4> { value }
result = engine.copy
```

Specialization identity is distinct from physical instance identity. A single-domain parent may supply an unambiguous child clock/reset implicitly; multi-domain hierarchy requires explicit compatible domains. Child protocol endpoints use `connect`, not scalar binding syntax.

A reusable component fingerprint covers its typed hardware and the deterministic transitive closure of callables that hardware actually invokes. Functions that are merely visible through a parent/import context do not participate. Thus the same parameterless child reached through different parents is emitted once, while different value/type specializations or a different reachable helper body remain distinct and fail closed on an identity conflict.

When every intermediate child conforms to one exact named interface with one protocol input and one protocol output, an option-free path can be written as:

```
input -> decode -> execute -> output
```

This lowers to the ordinary pairwise typed edges `input -> decode.rx`, `decode.tx -> execute.rx`, and `execute.tx -> output`. Intermediate names are physical instances, not guessed ports. Arrays, unnamed or ambiguous signatures, edge options, adapters, buffering, and crossings remain explicit.

One-dimensional compile-time instance arrays are structurally unrolled:

```
inst lane[N] : Lane<W>
generate(i in 0..N) {
    lane[i].x = values[i]
}
outputs = generate(i in 0..N) lane[i].y
```

Structural generation may be nested and may contain compile-time `if`/`else`. Both are elaborated by the same bounded unroller; they do not create runtime loops or dynamic topology:

```
generate(i in 0..N) {
    generate(j in 0..N) {
        if i == j { lane[i].x = values[j] }
    }
}
```

The existing total expansion limit applies across the complete nested tree. Exceeding it reports a bounded compile-time-generation diagnostic rather than partially elaborating the design.

A runtime selector may project one exact bit-packable wire output. This is a read-only mux over the already elaborated children, never a dynamic physical instance:

```
selected = lane[select].output
```

Every `lane[i]` still exists, receives its compile-time bindings, and advances its own state independently. The selector neither routes child inputs nor gates a child clock, reset, rule, or storage action. The explicit equivalent is `outputs = generate(i in 0..N) lane[i].output` followed by `selected = outputs[select]`; both spellings retain the same physical child identities.

The bounded supported form covers:

- combinational scalar children, including exact aggregate scalar outputs;
- single-domain scalar children with registers/rules;
- primitive ready/valid children, including scalar wire ports beside the endpoint and the existing one-FIFO profile;
- legacy storage-only children owning one FIFO, synchronous memory, or initialized ROM;
- the scheduled-FIFO profile in which FIFO actions and ordinary register writes share one existing `ResolvedTransition`; and
- an outer one-dimensional array whose element contains a bounded same-domain scalar or direct-ready-valid hierarchy. Inner compile-time scalar arrays are retained as distinct physical paths rather than flattened by generated name.

The first-level request/response profile is also bounded and explicit. Each array child has exactly one `ordering in_order` endpoint plus scalar wire ports, and each connection names compile-time indexed requester and responder elements:

```
generate(i in 0..N) {
    requester[i].issue = issue[i]
    responder[i].accept = accept[i]
    connect requester[i].mem -> responder[i].mem
}
```

Every physical element retains its own outstanding ledger and reset epoch. Out-of-order matching, an additional protocol on the same child, nested or transitive request/response arrays, and unindexed endpoint references fail closed. This profile has an executable capability-registry witness and is covered by compiler, simulator, and backend validation.

Legacy globally controlled storage still cannot be combined with user registers/rules. Credit and other non-RV protocols, CSR or storage below a nested array element, CDC arrays, runtime-selected inputs or protocol endpoints, cross-module atomic scheduling, and incomplete bindings are rejected. See storage-owning instance arrays for the exact state and backend evidence.

Concise declarations normalize to the same hierarchy IR when unambiguous:

```
frontend : AXI4LiteToRegBus<32,32>
axi : AXI4Lite<32,32>.slave @clk
axi -> frontend.axi
```

Explicit `inst` remains the escape hatch for namespace ambiguity.

8.1.1 Typed external modules

A bounded scalar-wire component may declare backend-independent behavior with an ordinary pure ZLang model:

```
extern module VendorAdd : AddIfc { model add_model }
```

The concise source form is `extern module VendorAdd : AddIfc { model add_model }`.

The applied interface and model produce one exact semantic signature. Physical SystemVerilog is supplied separately through a hash-validated `ExternalPhysicalMapping`; inline HDL never defines ZLang semantics. The

supported form has non-parameterized, clockless scalar inputs and one scalar output in Direct-SV. State, protocols, arrays, and generic external components fail closed.

8.2 Clock and reset boundary

Legacy `clock clk` plus synchronous active-high `reset rst` works through the production Direct-SV backend. A single-domain parent may supply that exact domain to a child only when the mapping is unambiguous; this is domain inheritance, not implicit CDC. One non-default falling-edge, asynchronous, or active-low physical contract is typed and emitted by Direct-SV. Concise `async reset` adds one root-owned two-edge release conditioner and passes the conditioned reset through the closed child ABI; the raw asynchronous compatibility form remains distinct. Instance arrays require one compatible inherited domain unless they are purely combinational. Multiple independently conditioned asynchronous domains are supported in Direct-SV. Every domain of a stateful child must match exactly one parent physical clock/reset contract; no implicit reset crossing is inserted. Multi-domain asynchronous formal execution, power-on reset, and implicit reset crossing remain fail-closed.

8.3 Public top ABI

The selected top always exposes integration-friendly typed leaves. Struct fields and structural tuple components, including those inside protocol payloads, become individually named ports; tuple paths use `item0`, `item1`, and so on. A vector leaf remains one multidimensional packed SystemVerilog array; for example `vec<8,u8>` becomes `logic [7:0][7:0] samples`. It is neither an anonymous 64-bit public bus nor eight separately named ports. Nested `vec<Struct>` values become one array per struct field, and `vec<Tuple>` values become one array per tuple component.

Direct-SV RTL uses this public `TopPhysicalABI`. The conversion follows the canonical packing order with element zero in the least-significant region: for a descending packed vector dimension, source element zero maps to physical index `0`. There is no source annotation, compiler option, or backend-specific preprocessor branch selecting another public ABI.

8.4 Ready/valid

```
in  rx : rv<u8>
out tx : rv<u8>

tx.payload = rx.payload
tx.valid = rx.valid
rx.ready = tx.ready
```

For an input endpoint, payload/valid enter the module and ready leaves it. Output ownership is reversed. `.transfer` is a read-only protocol property meaning `valid & ready`. A source stalled with `valid=1` and `ready=0` must hold payload and valid stable.

Direct composition names source then sink:

```
connect rx -> tx
connect buffered_rx -> buffered_tx { buffer 2 }
```

A ready/valid buffer is real FIFO state; it does not appear implicitly.

8.5 Credit and request/response

`credit<T,N>` provides pulse-return flow control. `.transfer` is the physical send after credit gating, `.credits` is the current bounded count, and `.return` returns one credit. Counters begin at `N`.

Adapters are explicit:

```
connect rx -> tx { adapter rv_to_credit }
connect crx -> rtx { buffer 2 adapter credit_to_rv }
```

Request/response endpoints declare ordering and outstanding capacity:

```
interface mem : request_response<Request,Response> {
    max_outstanding 2
```

```
    ordering in_order
}
```

The requester owns request payload/valid and response ready; the responder owns request ready and response payload/valid. Request and response channels expose their own read-only `.transfer` events. Directional buffers are independent:

```
connect requester.mem -> responder.mem {
    request_buffer 4
    response_buffer 2
}
```

Buffered but unaccepted requests are distinct from accepted outstanding requests. The ledger increments on responder-side request transfer and decrements on requester-side response transfer. Reset begins a new protocol epoch. `ordering out_of_order` requires a matching scalar ID field; hierarchical out-of-order composition remains outside the current bounded subset.

8.6 Aggregate protocols and the standard library

The compiler-shipped `std` namespace maps to ordinary `.zhl` files below `stdlib/`. The compiler knows generic aggregate schemas, roles, ownership, hierarchy, and domains; it does not hard-code AXI/APB transactions.

Available source-authored profiles include RegBus, AXI4-Lite, APB, AXI-Stream, Wishbone B4 Classic, bridges, and the CSR target:

```
import std.bus.reg
import std.bus.axi_lite

module AxiCsrTop {
    clock clk
    reset rst
    interface axi : AXI4Lite<32,32>.slave @clk
    inst frontend : AXI4LiteToRegBus<32,32>
    inst csr : RegBusCSRTarget<32,32>
    connect axi -> frontend.axi
    connect frontend.regbus -> csr.regbus
}
```

Aggregate pass-through connects each member according to typed ownership, including reverse ready:

```
input -> output
```

Protocol member properties such as `.transfer`, FIFO observations, and request/response channel events are part of typed semantics. They are not arbitrary user-defined fields and cannot be assigned when read-only.

8.7 Packets, virtual channels, and arbitration

`packet<T>` extends ready/valid with `.last`. Arbiters explicitly select source order, `fixed_priority` or `round_robin`, and `beat` or `packet` grant lifetime. `vc_credit<T,V,C>` maintains independent credit counts per virtual channel.

8.8 CSR blocks

CSR source describes address layout, access policy, reset, and hardware binding once. Supported policies are `rw`, `ro`, `wo`, `w1c`, `pulse`, and `reserved`. Hardware status uses `<-`, commands use `->`, and sticky events make simultaneous hardware/software priority explicit. RTL, JSON, and Markdown are derived from the same typed model.

Status and sticky sources may select exact leaves of a typed aggregate input; the compiler retains and type-checks the member expression rather than deriving a flattened name:

```
struct Perf { cycles : u32 }
struct EngineStatus { busy : bit perf : Perf }
```

```
module AggregateStatusBank {
    clock clk reset rst
    in status : EngineStatus
    csr registers @0 {
        STATUS @0 { busy bit @0 ro <- status.busy }
        CYCLES @4 { value u32 @31:0 ro <- status.perf.cycles }
    }
}
```

Aggregate members used by one CSR bank must belong to that bank's clock domain. Command bindings remain scalar output ports.

Omitted bits are implicitly reserved. A storage-independent access event may observe the same bits as their single owning field:

```
module CsrAccessEventExample {
    clock clk reset rst
    in fault_status : bits<32>
    out fault_clear, registered_clear : bits<2>
    csr registers @0 {
        FAULT @0x470 {
            value bits<32> @31:0 ro <- fault_status
            clear bits<2> @1:0 on_write -> fault_clear
            clear_later bits<2> @1:0 on_write post_accept -> registered_clear
        }
    }
}
```

The event is address-qualified by the compiler and owns no field state. Its default `active_transfer` phase is combinational during the qualified access. An explicit `post_accept` phase captures the selected event value at the accepting edge, publishes it for the following cycle, and then clears it when there is no new matching access. This phase does not change CSR readback or the canonical rule that `rdata` is meaningful only for reads. Stored, external-status, and event observations are available through typed child namespaces `state`, `status`, and `events`. JSON and Markdown maps record the selected event phase.

Repeated layouts flatten into one bank and one decoder:

```
module CsrGroupExample {
    clock clk reset rst
    csr group Window { CONTROL @0 { enable bit @0 rw = 0 } }
    csr control @0 { windows : Window[13] @0x210 stride 4 }
}
```

Group counts are bounded to 1..64 and a bank to 256 expanded registers. A logical 64-bit value may retain two independently writable 32-bit software registers:

```
module CsrSplitExample {
    clock clk reset rst
    csr registers @0 {
        INPUT_BASE @0x18 split<32> value u64 rw = 0 order low_first
    }
}
```

The same `split<32>` declaration may be part of a reusable group. Every group instance receives one logical typed projection and two independently writable 32-bit words:

```
module GroupedSplitExample {
    clock clk reset rst
    csr group Window {
        CONTROL @0 { enable bit @0 rw = 0 }
        BASE @4 split<32> value u64 rw = 0 order low_first
    }
```

```
    csr registers @0 { windows : Window[2] @0 stride 12 }
}
```

This does not imply byte strobes or an atomic two-word write. Typed command capture and registered RegBus timing are ordinary source modules: `StorageSnapshot<T,reset_value>` in `std.storage.core` and `RegisteredCSRTarget<AW,DW>` in `std.bus.reg`; the compiler has no special case for either module name.

In a multi-clock module the bank declares its owner after its base address:

```
csr control @0x4000_0000 @apb_clk { /* registers */ }
```

All blocks sharing the canonical access ABI must use that one domain. A runtime CSR value consumed by another domain still requires explicit CDC.

8.9 Clock-domain crossings

Implicit CDC is an error. Existing explicit forms are:

Crossing	Supported endpoint
`sync_level`	Slowly changing `bit` level
`pulse_toggle`	Rate-limited `bit` pulse
`handshake`	One scalar ready/valid payload
`async_fifo(N)`	Ready/valid stream through a power-of-two dual-clock FIFO

```
connect source -> destination { crossing async_fifo(4) }
```

The crossing is a semantic boundary. Its output has destination-domain provenance and may feed destination-domain combinational logic, rules, FSMs, or state normally. The compiler never rewrites, balances, or resource-packs logic through the boundary, and never chooses a crossing automatically.

```
connect enable_a -> enable_b { crossing sync_level }
rule Observe @clk_b when enable_b { seen <- 1 }
```

An aggregate with exactly one forward ready/valid member may use the same `async_fifo` crossing; its complete payload is atomic. Both domains must participate in a coordinated reset episode.

The physical Direct-SV FIFO uses one compiler-owned 1W1R `async_mem` with a one-cycle registered read, two-stage Gray-pointer synchronizers, and one prefetched output beat. A stalled beat holds both payload and `valid`; the read/consumed pointer advances only on transfer, so that beat still occupies capacity. Continuous accepted beats can transfer at II=1 without an extra output bubble. The synchronizers model digital CDC behavior, not analog metastability or an MTBF guarantee.

The ordinary `std.storage.core.StorageAsyncFifo<T,D>` wrapper exposes this same crossing as a reusable module with explicit writer and reader clock/reset ports. It lowers through the existing CDC IR; its module name is not special to the compiler. A normal `fifo<T,N>` never changes into an asynchronous FIFO because its endpoints happen to use different domains.

No automatic protocol adaptation or implicit CDC is performed. Full AXI4 bursts/IDs and bus-specific CDC bridges remain deferred.

9 Named module interfaces

Named module interfaces give a public name to an exact, behavior-free module signature. They let a library state the ABI that several concrete modules must implement without selecting, replacing, or instantiating any implementation.

```
interface FirIfc {
    clock clk
    reset rst
    in  x : fixed<18,16>
    out y : fixed<18,16>
```

```
    timing {
        latency 4
        ii 1
    }
}

module DirectFir : FirIfc {
    // The complete public surface is inherited from FirIfc.
    y = pipeline(4) { x }
}
```

The declaration after `interface` contains only public signature items:

- type and value parameters;
- `clock` and `reset` declarations;
- scalar, wire, ready/valid, credit, packet, and VC-credit `in`/`out` ports;
- source-authored aggregate protocol endpoints and their roles;
- at most one exact `timing { latency N ii 1 }` contract.

It cannot contain assignments, registers, rules, storage, child instances, or other implementation behavior.

9.1 Exact conformance

`module M : Ifc` means that `M` promises to have exactly the applied signature of `Ifc`. Conformance is checked during typed semantic analysis. It is not structural duck typing and does not introduce an implicit conversion.

The following all belong to the contract:

- parameter names, kinds, declaration order, and defaults;
- applied type and value parameter values after specialization;
- port names, declaration order, directions, canonical types, and protocol capacities;
- clock/reset pairs and every port or endpoint domain;
- aggregate protocol specialization, role, member ownership, payload types, and member domains;
- presence and exact value of the module timing contract.

If a conforming module declares no public surface, the complete applied interface surface is inherited: parameters, ports, clock/reset domains, aggregate endpoints, and timing. This is exact signature inheritance, not structural inference. The legacy fully redeclared form remains accepted. A partial redeclaration is never merged with the interface and fails exact conformance.

Missing or additional members in a fully redeclared surface are errors. So are renamed ports, changed directions, width changes, role changes, a different domain, and a timing contract present on only one side.

Named parameters and concrete type parameters are supported:

```
interface TransformIfc<type T, N=4> {
    in  x : vec<N,T>
    out y : vec<N,T>
}

module Transform<type T, N=4> : TransformIfc<T=T, N=N> {
    y = x
}
```

The module parameter declaration itself must match the interface parameter declaration exactly. A concrete specialization records canonical applied values in `ModuleSignature`; concise source spelling does not change that identity.

9.2 Aggregate protocol signatures

An existing source-authored aggregate protocol can be part of a named module interface. Its role and clock domain are explicit:

```
protocol TinyBus<AW=8> {
```

```
    role initiator
    role target
    channel command : rv<bits<AW>> initiator -> target
    member alarm : bit target -> initiator
}

interface TinyTarget<AW=8> {
    clock clk
    reset rst
    interface bus : TinyBus<AW=AW>.target @clk
}

module Target<AW=8> : TinyTarget<AW=AW> {
    clock clk
    reset rst
    interface bus : TinyBus<AW=AW>.target @clk
    // Concrete behavior remains in the module.
}
```

Conformance uses the already typed aggregate schema and ownership model. It does not guess compatibility from endpoint or generated RTL names.

9.3 Selection and backend identity

A named interface never chooses a concrete implementation. Source still instantiates a concrete module, and normal top selection still names a concrete module. There is no automatic substitution, inheritance, refinement search, or runtime dispatch.

The applied `ModuleSignature` is backend-independent semantic/build metadata. It survives canonical round trips, while the implementation body continues through the production Direct-SV path. Merely adding an equivalent interface declaration does not authorize a backend to change RTL or timing.

9.4 Current boundaries

The current bounded surface intentionally does not support:

- `request_response` members in named module interfaces, because their requester/responder role is currently inferred from module behavior rather than declared as an exact signature role;
- interface extension or refinement beyond exact complete-surface inheritance;
- multiple-interface conformance;
- automatic implementation selection or substitution;
- relaxed variance, implicit resizing, protocol adaptation, or CDC insertion.

Use the existing concrete module and protocol forms when one of these boundaries applies. Unsupported or non-conforming signatures fail with a structured interface diagnostic; they never silently weaken the ABI.

10 Tagged unions

ZLang tagged unions are a bounded, backend-independent way to carry one of several named scalar payload shapes. They are values, not procedural control flow and not protocol envelopes.

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }

result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
```

```
    Message.Error { code } => extend<8>(code)
}
```

10.1 Type and layout

The type is nominal: two equal-looking declarations are not interchangeable. Variant order defines ordinal tag codes. The tag width is `max(1, ceil_log2(variant_count))` and occupies the most-significant bits. The payload is as wide as the largest variant. Fields occupy it from most to least significant in declaration order; a shorter payload has zero low-order padding.

The supported field types are flat `bit`, `bits<N>`, `uN`/`uint<N>`, `sN`/`sint<N>`, `fixed`, and `ufixed` fields. A fieldless value uses the concise constructor `Message.Idle`; `Message.Idle {}` remains an equivalent explicit spelling.

10.2 Construction and matching

Constructor fields are named, exact, and complete. Missing, extra, repeated, or wrongly typed fields are compile-time errors. A `match` must contain every variant exactly once. A payload arm binds every field in declaration order; binders cannot rename, omit, repeat, or shadow an existing value.

Matching is pure and zero-latency. Semantic lowering retains a typed `UnionConstruct`, `UnionTag`, and `UnionField`, then expresses selection with the ordinary typed `Switch`. The simulator carries an immutable nominal runtime value. Direct-SV carries the exact frozen packed bits. Registers and internal scalar child ports may use union values.

10.3 Deliberate boundaries

An external top-level union input is rejected because arbitrary raw bits could encode an unused tag. There is no raw decode, `bitcast<Union>`, `unpack<Union>`, generic/recursive union, nested aggregate payload, union operator overload, wildcard/nested pattern, guard, partial match, or protocol inference in this slice. No e-graph rewrite or new formal observation is added.

The runnable source is `examples/tagged_union.zhl`. The exact representation and current exclusions are documented above and in the syntax support matrix.

11 Parameterized aggregate protocols

The generic composition slice expands a parameterized protocol endpoint during semantic elaboration. For example:

```
protocol TinyBus<AW=8> {
    role initiator
    role target
    channel req : rv<uint<AW>> initiator -> target
    channel rsp : rv<uint<AW>> target -> initiator
    member irq : bit target -> initiator
}
```

A module uses this declaration with `interface bus : TinyBus<AW=8>.initiator`. The compiler retains the aggregate identity and specialization, then expands each member into the existing typed leaf endpoint IR. Ready/valid members preserve physical backward-ready propagation; plain members are ordinary scalar wires.

Top-level aggregate endpoints additionally have a backend-independent `TopAggregateABI` projection. It recursively flattens ready/valid payload structs, derives physical direction from source/sink ownership, and keeps aggregate/member identities separate from generated names. Direct-SV consumes that projection without protocol-specific source-name guessing.

`connect left.bus -> right.bus` checks protocol identity, specialization, member names, payload types, roles, and clock domains before producing leaf hierarchical connections. Aggregate buffering, adapters, and CDC are deliberately rejected; they must be expressed on a leaf connection in a later library design.

The Direct-SV backend consumes the same closed child component ABI as other hierarchical protocol children. A child receives every scalar dependency and protocol backward signal explicitly and returns forward values plus

scalar outputs. No aggregate or RTL name is reconstructed by textual substitution. BackendArtifact manifests publish both aggregate identities and their leaf signal bindings, so formal/source attribution remains stable.

Generic value parameters support exact positive width arithmetic (`+`, `-`, `*`, and exact `/`). Type parameters are bound at an aggregate use site. The aggregate connection is structural: it does not implement AXI/APB transaction state machines, adapters, packages, CDC, or automatic protocol conversion.

The TinyBus producer/consumer example (two ready/valid channels plus a reverse scalar member) elaborates into three typed leaf connections and passes the release-tested Direct-SV lint gate.

Stateful children with multiple independent ready/valid channels are supported in Direct-SV. An unconnected aggregate bus on the selected top is projected through the shared `TopPhysicalABI` into typed public leaves. This is generic schema/ownership lowering, not an AXI semantic exception. Arrays, partial aggregate exposure, and unsupported protocol kinds remain fail-closed as listed in the live syntax matrix.

12 Standard buses

The production bus profiles are ordinary ZLang sources. The compiler contains no AHB, AXI, APB, AXI-Stream, Wishbone, or RegBus transaction dispatcher.

Import	Source-owned declarations	Initial profile
`std.bus.reg`	`RegBus`, CSR bank/target	In-order request/response CSR boundary
`std.bus.axi_lite`	`AXI4Lite`, `AXI4LiteToRegBus`	32/32-compatible, independent AW/W buffering
`std.bus.axi_burst`	`AXI4BurstSubset`, read/write views and helpers	No-ID, single-outstanding, full-width incrementing bursts
`std.bus.axi4`	Five-channel `AXI4`/`AXI4WithUser`, bounded read/write managers	IDs, bursts and optional USER payloads; under validation
`std.bus.axi4_subordinate`	Bounded read/write subordinate adapters	Backend transactions and responses; under validation
`std.bus.axi4_pins`	Flat-pin adapters	Source-owned five-channel pin projection
`std.bus.axi4_exclusive`	Reservation monitor and exclusive subordinate	Separate explicit same-edge commit contract; under validation
`std.bus.apb`	`APB`, `APBToRegBus`	APB setup/access sequencing
`std.bus.ahb_lite`	`AHBLite`, `AHBLiteToRegBus`	Single-manager, full-width AHB-Lite to RegBus
`std.bus.axi_stream`	`AXIStream`, `AXIStreamPipe`	Data/keep/strb/last ready/valid stream
`std.bus.wishbone`	`Wishbone`, `WishboneToRegBus`	B4 Classic single-beat, ack/err/stall

Error completion is part of the source profile rather than a backend policy. AXI4-Lite maps a failed RegBus write or read to `SLVERR` (`2'b10`) and holds the complete B/R payload while the corresponding ready signal is low. APB forwards the held RegBus error through `PSLVERR` on the access completion. Wishbone uses mutually exclusive normal `ACK` and abnormal `ERR` termination; either one retires the single outstanding request.

AHB-Lite preserves the protocol's pipelined address/data relationship. The bridge captures an accepted address/control phase, consumes write data in the following data phase, and holds `HREADYOUT` low while its one RegBus request is pending. Local size/alignment errors and RegBus failures both use the standard two-cycle ERROR response: low-ready/high-response followed by high-ready/high-response. The first profile accepts aligned, full-bus-width transfers for power-of-two byte-addressable data widths from 8 through 1024; subword transfer strobes are deliberately deferred.

The bridge owns state but does not impose a fixed physical reset spelling. It inherits the enclosing module's one exact clock/reset contract, and that contract participates in specialization identity and is shared by semantic IR, native simulation, formal products, and Direct-SV. The runnable AHB example uses conventional active-low `hresetn` with asynchronous assertion and the language's two-edge synchronized release; a synchronous active-high parent is also supported. Ambiguous multi-domain inheritance and mismatched connected sequential logic fail closed. In every contract, `HREADYOUT` is high while reset is asserted.

`std` is a logical namespace mapped to the physical `stdlib/` source tree. The resolver discovers `.zhl` modules by convention, resolves a deterministic dependency closure, rejects cycles and unsafe paths, and records every logical identity/content hash in semantic, canonical, and backend artifacts.

The AXI4 source modules include IDs, bounded outstanding slots, five independent channels, and optional USER sidebands. Directed and randomized tests cover stalls, repeated IDs, slot reuse, error responses, native simulation against generated Direct-SV, and direct-SystemVerilog generation. These checks do **not** establish complete external endpoint compliance: bounded formal endpoint proofs remain outstanding. The ordinary subordinate never emits `EXOKAY`; exclusive commit requires the separate `axi4_exclusive` contract and same-edge `exclusive_grant` gating. The narrower `axi_burst` profile remains distinct and unchanged. AXI-Stream ID/dest/user, Wishbone burst/retry, automatic adapters, and CDC are not added by these profiles.

Runnable integrated examples are AXI4-Lite CSR, APB CSR, AHB-Lite CSR, Wishbone CSR, and the streaming packet engine. The bounded burst helpers are exposed by the ZTPU AXI burst witness. Python models in `zlang/standard_bus.py` remain independent test oracles only.

The AHB-Lite contract follows the Arm AMBA 3 AHB-Lite protocol rather than the older ZTPU model's simplified same-cycle "AHB-like" behavior. In particular, write address/control and `HWDATA` are not sampled in the same phase, and ERROR is not shortened to one cycle. The source profile and executable tests freeze that bounded contract; broader AHB features remain explicit exclusions below.

12.1 Bounded ZTPU AXI burst subset

`std.bus.axi_burst` is ordinary source-authoritative ZLang HDL. It defines a combined `AXI4BurstSubset<AW,DW>` with independent AR/R and AW/W/B ready/valid channels, together with read-only and write-only protocol views for ZTPU's separate physical master ports. Address payloads carry byte address, eight-bit beats-minus-one `len`, and three-bit beat `size`; read and write data carry `last`, and R/B carry the exact two-bit response value.

The reusable `AXI4BurstReader<AW,DW,LW>` and `AXI4BurstWriter<AW,DW,LW>` accept aligned 1-256-beat requests and permit one outstanding transaction each. AR and AW payloads are snapshotted and remain stable until accepted. Read consumer backpressure drives RREADY directly. The writer accepts AW before exposing W, while AW, W, and B remain independent handshakes; its ready/valid producer retains W data while stalled. Counted framing owns completion: the reader reports early or missing RLAST but drains the requested beat count, while the writer generates WLAST on its locally counted final beat. Any nonzero RRESP or BRESP sets a sticky boolean error for that transaction epoch.

The concrete `AW=64,DW=32` reader/writer witness passes semantic and canonical round trips, deterministic artifact checks, bounded simulator traces, and Direct-SV/Verilator. Validation includes 1- and 256-beat requests, invalid 0 and 257 lengths, misalignment, independent channel stalls, stable owned payloads, RLAST/RRESP/BRESP failures, reset in active phases, and ignored starts while busy. No compiler, simulator, or backend dispatches on AXI names.

This profile deliberately omits IDs, WSTRB, burst-kind, lock, cache, protection, QoS, region and user fields, multiple outstanding transactions, UB-DMA burst chunking, command-descriptor decoding, fences, CDC, or implicit adaptation. Its accepted semantic boundary is the bounded profile stated above.

12.2 AXI4-Lite, APB, and RegBus

`std.bus.reg`, `std.bus.axi_lite`, and `std.bus.apb` are ordinary imported library sources. The compiler checks structural protocol roles, typed ready/valid members, ownership, and domains; transaction state machines belong to library modules. `RegBus<32,32>` is the typed CSR boundary.

The AXI4-Lite and APB profiles use one clock/reset and one outstanding operation. `Axi4LiteToRegBus` buffers AW and W independently and joins them after both transfers; AR returns one held R response. `ApbToRegBus` implements setup/access phases and holds controls stable while waiting for PREADY. Top-level aggregate endpoints expose typed public leaves (for example, `axi_aw_valid` and `apb_psel`); `connect axi -> frontend.axi` delegates a same-role endpoint to a child.

`RegBusCSRTarget` is ordinary source-authored state with RW, sticky W1C, and one-cycle pulse registers. Its response data and valid bit are held until a response transfer. Formal attachment requires exact published observations: properties needing hidden or unsupported state are explicitly skipped or rejected under the selected policy. Full AXI4 is outside these library profiles; the separate bounded AXI burst subset is described above.

13 Standard library

std is the compiler-shipped logical namespace. It maps to tracked ordinary ZLang HDL sources under stdlib/; it is intentionally shorter than the physical directory name and is not a Python or user-package import.

Import	Purpose
std.math.fixed	Pure exact-width/inferred fixed-point abs, clamp, min/max, MAC and explicit quantization helpers, plus compatibility components
std.math.complex	Generic Complex<T>/Butterfly<S,D>, nominal operators, explicit complex quantization
std.math.complex_fixed_18_16	Compatibility Q2.16 complex multiply profile
std.stream.core	Generic FrameBeat<T,M>, ready/valid register slice, skid buffer, FIFO wrapper, and explicit two-way mux/demux
std.stream.serialization	Bounded power-of-two vector serializer and raw bit collector
std.stream.complex_fixed	Compatibility Q5.32-to-Q2.16 complex stream quantizer
std.dsp.fft	Exact FFT butterfly, bit reversal, and compile-time twiddle helpers
std.storage / std.storage.core	Bounded delay/FIFO plus raw-bit reorder and ordered ping-pong storage wrappers
std.coding / std.coding.core	Parity, bit reversal, checked polynomial-tap LFSR step, and exact convolution helpers
std.bus.reg	RegBus and the source-authoritative CSR target
std.bus.axi_lite	AXI4-Lite protocol and RegBus frontend
std.bus.axi_burst	Bounded no-ID AXI burst profile, read/write views, and single-outstanding burst/single-beat helpers
std.bus.apb	APB protocol and RegBus frontend
std.bus.ahb_lite	Standards-correct bounded AHB-Lite protocol and RegBus frontend
std.bus.axi_stream	AXI4-Stream beat/profile and backpressure-preserving pipe
std.bus.wishbone	Wishbone B4 Classic and RegBus frontend
std.target.generic	Resource-free generic target identity
std.target.asic.generic	Generic ASIC cell/resource capability profile
std.target.asic.sky130	SKY130 sky130_fd_sc_hd standard-cell synthesis profile
std.target.intel.cyclone_v	Bounded Cyclone-V target/resource inventory
std.target.xilinx.series7	Bounded source-described Series-7 DSP48E1 profile
std.target.xilinx.xc7z030	XC7Z030 part and bounded resource inventory
std.arch.xilinx7_fir	Manual four-resource symmetric-FIR cascade template
std.arch.xilinx7_memory	Bounded Series-7 storage/resource mapping descriptions
std.arch.xilinx7_multiply	Series-7 hard-multiplier architecture descriptions
std.arch.xilinx7_signed_product	Signed product-reduction architecture descriptions

Imports are resolved transitively, cycles are rejected, and logical path plus SHA-256 content hash participate in canonical/backend artifact identity. External path/Git packages use the separate pinned zlang.toml/zlang .lock project resolver. Filesystem-relative source imports and implicit network lookup are not part of the compiler-shipped std resolver, and ordinary compilation never fetches dependencies.

std.bus.axi_burst is the source-owned ZTPU interoperability profile, not a compiler-recognized bus. It publishes the combined five-channel AXI4BurstSubset<AW,DW> plus read-only and write-only views matching ZTPU's separate physical masters. AXI4BurstReader and AXI4BurstWriter implement one full-width incrementing transaction at a time, with 1-256-beat counting, independent channel backpressure, checked RLAST, counted WLAST, and deterministic boolean error latching for nonzero RRESP/BRESP. The validated AW=64,DW=32 witness passes semantic/canonical restoration, deterministic backend emission, simulator traces, and Direct-SV/Verilator. IDs, write strobes, burst-kind and other full-AXI sidebands, multiple outstanding transactions, UB-DMA chunking, and fences remain outside this bounded profile.

For one full-width beat, axi_single_beat_address<AW,DW> constructs AxiBurstAddress<AW> with len=0 and the matching size, plus a validity bit for geometry and alignment. AXI4SingleBeatReader preserves the burst reader's one-beat RLAST/RRESP checks; AXI4SingleBeatWriter snapshots AW and W payloads and allows either channel to transfer first. Both retain the one-outstanding busy/done/error convention. See examples/ axi_single_beat.zhl. These helpers do not add IDs, WSTRB, or implicit bus adaptation.

std.bus.ahb_lite follows the AHB-Lite address/data pipeline and two-cycle ERROR response rather than the historical ZTPU model's same-cycle AHB-like shortcut. AHBLiteToRegBus<AW,DW> accepts one aligned full-width

beat at a time for byte-addressable power-of-two DW from 8 through 1024, supplies an all-byte RegBus write mask, and holds the AHB data phase through RegBus request/response stalls. Subword transfers, arbitration, SPLIT/RETRY and compiler/backend AHB dispatch are not part of this profile. Its hresetn example domain uses active-low asynchronous assertion and two-edge synchronized release. The bridge itself inherits the enclosing module's one exact physical clock/reset contract, so the same source also composes with a synchronous active-high system. Ambiguous multi-domain inheritance and mismatched connected state owners remain errors; this is not an implicit reset or CDC conversion.

std.math.complex is ordinary source-authoritative ZLang. It has no bus or stream dependency and uses only generic structs/functions and nominal operator declarations; semantic analysis and the Direct-SV backend have no Complex special case. Mixed fixed-point multiplication retains its full exact width and scale, and the caller places every quantization boundary explicitly. In particular, the core contains no implicit fixed<18,16> butterfly. The historical Q2.16 component and stream profiles live in separate compatibility imports std.math.complex_fixed_18_16 and std.stream.complex_fixed; importing numerical Complex arithmetic does not select either profile or import a bus.

13.1 Fixed-point math

std.math.fixed provides pure generic fixed_abs, fixed_min, fixed_max, fixed_clamp, and exact fixed_mac functions plus explicit rounding/overflow helpers such as fixed_quantize_nearest_even_saturate<T>. fixed_abs widens according to ordinary unary-negation rules, so the most-negative input has an exact positive result; fixed_mac preserves the full product and addition widths. The library also retains the source-authored FixedAbs, FixedClamp, FixedMinMax, FixedMAC, FixedSaturatingAdd, and UFixedSaturatingAdd components. They operate on canonical fixed-point types; concise SF8.8, SF_Sat8.8, UF8.8, and UF_Sat8.8 forms are language aliases rather than library implementations. Encoding and conversion rules are specified in fixed-point-types.md.

13.2 Complex values, streams, storage, and coding

std.math.complex provides the ordinary parameterized value type Complex<T> with re and im fields. butterfly_quantized<T> requires the output numeric profile explicitly; there is no profile-selecting short spelling in the core. Complex remains ordinary source, not a compiler or backend primitive.

std.bus.axi_stream keeps AXIStream<DW> as the byte-lane bus profile and also provides AXIStreamOf<T> for a semantic payload type:

```
input  : AXIStreamOf<Complex<fixed<18,16>>>.sink @clk
output : AXIStreamOf<Complex<fixed<18,16>>>.source @clk
```

AXIStream<32> has data/keep/strb/last; AXIStreamOf<T> transports exactly one typed T per transfer. It does not by itself specify an external AXI TDATA serialization for arbitrary T.

Generic stream composition uses std.stream.core, independently of AXI:

```
import std.stream.core

struct Meta { tag : u2 }
type Beat = FrameBeat<u8,Meta>

module Queue {
    clock clk
    reset rst
    in input : rv<Beat>
    out output : rv<Beat>

    inst storage : RvFifo<T=Beat,D=4>
    connect input -> storage.input
    connect storage.output -> output
}
```

RvRegisterSlice<T>, RvSkidBuffer<T>, and RvFifo<T,D> reuse the exact language FIFO semantics, including simultaneous push/pop and payload stability under stall. RvMux2<T> and RvDemux2<T> provide explicit runtime routing between statically elaborated ready/valid endpoints. They route backpressure only through the selected path, never create dynamic instances or clock gates, and can be composed as a bounded tree for wider routing.

Scalar routing remains ordinary `mux` and explicit valid/enable gating. `std.stream.serialization` currently supports a power-of-two `RvVectorSerializer<T,N,IW>` and a raw `RvBitCollector<N,IW>`, where `IW = floor_log2(N)`. Their declarations enforce `N >= 2`, power-of-two `N`, and exact `IW` through compile-time `where` constraints.

`std.dsp.fft` keeps arithmetic intent explicit. Its butterfly does not quantize, twiddle generation uses compile-time `sin`/`cos` and quantizes only to its caller-selected type, and bit reversal is a deterministic sequence-order operation. Value parameters inferred through a `bits<N>` argument are not implemented yet, so bit-width helpers use explicit calls such as `fft_bit_reverse<N=8>(value)`.

`std.storage.core` supplies `StorageDelay1<T>`, `StorageDelay2<T>`, the generic `StorageQueue<T,D,CW>` (`CW = ceil_log2(D + 1)`), and bounded raw-bit `StorageReorderBits<W,N>`/`StoragePingPongBits<W,N>` banks. The latter use power-of-two depth, exact runtime indices, and explicit bitcast at typed/raw boundaries. Ping-pong publication is ordered: a second `commit` is blocked until the visible read bank retires, while simultaneous retire/commit is legal. The same file provides ordinary `StorageDualPortMemory<T,N,AW>`, `Storage2R1WMemory<T,N,AW>`, and `StorageAsyncMemory1W1R<T,N,AW>` wrappers over the shared typed memory IR. `StorageAsyncFifo<T,D>` wraps the existing explicit ready/valid `async_fifo(D)` crossing. These module names are not compiler intrinsics and do not authorize implicit CDC. It also provides immutable, source-authored generic ROM wrappers:

```
inst direct : StorageRom<T=u8,N=8,IW=3,image=image>
inst generated : StorageGeneratedRom<
    T=u8,N=8,IW=3,producer=fn make_image<T=u8,N=8>
>
```

Both wrappers elaborate to the existing typed `Rom` IR with concrete immutable contents and the deterministic companion image used by Direct-SV. `StorageRom` accepts a fully evaluated exact `vec<N,T>` constant; `StorageGeneratedRom` invokes a statically selected pure zero-argument producer during elaboration. Constants and producers are specialization parameters, not runtime ports or backend callbacks. Fixed, struct, vector, and nested-vector word types retain their exact canonical layout.

The core language currently requires a literal in `delay<N>`, so a generic-depth delay wrapper would be dishonest; longer delays remain explicit source until parameterized delay depth is supported. `std.coding.core` supplies representation-level parity, bit reversal and one-step LFSR operations plus exact dot/convolution and `table_gather` helpers. `table_gather<T,N,IW>` is accepted only when the complete proven `uint<IW>` range fits the source vector. It preserves order but deliberately does not claim bijection: duplicate and omitted elements remain legal. The `CodingLfsrStep<N>` module enforces `N >= 2`; the compatibility pure function cannot carry a `where` clause because function constraints are not yet syntax.

The current language deliberately prevents several tempting but invalid "generic" wrappers. A target-independent `reg vec<N,T>` requires an explicit initializer; `default<T>` is not supported. The shipped reusable reorder/ping-pong banks therefore use an explicit `bits<W>` representation boundary. A generic runtime gather now retains the conservative element-type range through a table-loaded index, but it does not prove that a table is a mathematical permutation. The negative semantic tests retain the bounded gather witness; no backend guesses an initializer or claims a permutation proof. Concrete projects may still use typed ROMs and permutations with known source-generated tables, as the Wi-Fi and FFT sources do.

The separated `std.stream.complex_fixed` compatibility kernel exposes raw `rv<Complex<...>>` ports. The former aggregate AXIStream profile cannot be forwarded transparently yet: hierarchical connections between `AXIStreamOf<Complex<...>>` specializations and member-level child ready/valid ports are rejected by the current aggregate binding model. The kernel is fully backend-validated, but restoring that aggregate ABI requires a generic aggregate-member binding compiler slice rather than bus-specific stdlib code.

Every shipped `.zhl` file is discovered recursively by the `std.*` resolver. Clean-wheel tests compare the complete recursive source tree with wheel contents; adding a library file without packaging it therefore fails the build rather than creating a checkout-only import.

13.3 Target and architecture descriptions

Target libraries use the same safe, hashed `std.*` resolver as bus and math sources. They describe resource capabilities and physical binding locators; they do not add functional primitives. See target-platform-architecture-description.md for the supported declarations, manual selection flow, and current bounded Series-7 implementation.

14 Projects and dependencies

ZLang source imports are logical names. Physical paths, Git URLs, and revisions belong to project metadata, never to `.zhl` source:

```
import acme.dsp.filters
import std.math.complex
```

An import may introduce a source-local qualifier without changing the logical module or dependency identity:

```
import std.math.complex as cx

in sample : cx.Complex<u8>
out total : cx.Complex<u9>
total = cx.complex_sum([sample, sample])
```

The qualifier applies to types, functions, and struct constructors declared by that exact logical module (`cx.Complex { re = ... im = ... }` is valid). It is erased before semantic typing, so the qualified and historical unqualified spellings have the same canonical identity. An aliased import does not expose those declaration names unqualified and does not re-export declarations from a transitive dependency. Aliases are not runtime values or filesystem names; wildcards, member renaming, and re-export remain unsupported.

The compiler-shipped `std` namespace keeps its existing resolver. Other package names are resolved only when the source belongs to a locked project.

14.1 Project manifest

A project is rooted by a versioned `zlang.toml`:

```
schema = 1

[project]
name = "demo"
version = "0.1.0"
source-root = "src"

[dependencies]
acme = { path = "../acme-dsp" }

[dependencies.bus_models]
git = "https://example.invalid/hardware/bus-models.git"
rev = "0123456789abcdef0123456789abcdef01234567"
```

Unlike internal artifact schema versions, the `schema` field above is required user-written project metadata: it tells `zlang lock update` how to interpret the manifest. Use the value shown for the current project format.

Package and module identities contain logical names and content digests, not absolute checkout or cache paths. A file `filters.zhl` directly below package `acme`'s source root is imported as `acme.filters`; nested directories append dotted components.

Path locators are relative to the manifest containing them. Git revisions must be full lowercase 40- or 64-digit hexadecimal object IDs. Branch names, tags, and abbreviated revisions are deliberately rejected because they are mutable.

14.2 Lock update

Dependency resolution and fetching are explicit:

```
zlang lock update --project zlang.toml
```

The command validates the complete transitive graph before publishing a deterministic `zlang.lock`. It records each package manifest, exact source-module index, imports, content digests, and dependency edges. Git content

is populated in the project cache during this command. Path dependencies remain at their manifest-relative location and are checked byte-for-byte against the lock.

Resolution rejects:

- dependency cycles and conflicting package/module declarations;
- source-root traversal or symlink escape;
- a package whose declared identity does not match its dependency key;
- missing, added, removed, or modified locked `.zhl` modules;
- modified dependency-resolution fields in dependency manifests (profile-only edits are intentionally outside the resolution identity);
- missing Git cache content or a revision mismatch.

The lock is written only after the entire graph validates. Failed updates leave the previously accepted lock usable.

Scenario	What to do	Without it
Standalone source or compiler-shipped `std.*` imports	Compile directly; no project lock is needed.	No consequence.
Project with local path packages	Run `zlang lock update` after declaring dependencies and whenever their source or resolution fields change. Keep `zlang.lock` with the project.	Compilation rejects a missing or stale lock; it never silently uses changed dependency files.
Project with pinned Git packages	Run `zlang lock update` while Git/network access is available; retain the exact revision and local cache for later offline builds.	Ordinary compilation does not fetch; missing cache or changed revision fails.
CI or another machine	Supply the project manifest, its lock, exact path sources and cached Git revisions (or update the lock before the offline build).	A lock file alone does not provide Git source bytes; compilation fails closed.

`zlang lock update` resolves dependencies; it is not a package installer or a step required for every single-file invocation.

Bounded scalar `extern module` implementations may also be declared under `[external-mappings.NAME]` and selected by a profile's `external-mappings = ["NAME"]`. The lock records every HDL source path and SHA-256. Paths are project-relative, must remain inside the project, and may not be symlinks. This is a physical DirectSV input: it does not replace the pure ZLang model or enter semantic IR.

14.3 Offline compilation

Ordinary compilation never fetches dependencies and never updates project metadata:

```
zlang src/top.zhl --project zlang.toml --check
zlang src/top.zhl --project zlang.toml --systemverilog build/top.sv
```

`--project` may name a manifest or its directory. Without it, `zlang` searches the source file's parent directories for `zlang.toml`. If no project is found, single-file and compiler-shipped `std.*` compilation retain their existing behavior; arbitrary external imports remain an explicit error.

Compilation checks the current manifest, lock, every path dependency, and every cached Git dependency before semantic analysis. It is read-only: unavailable or dirty content is a lock mismatch, not an invitation to fetch or rewrite files. Build outputs and compiler-owned output/cache directories must remain outside the resolved root, manifest, lock, dependency, and stdlib inputs. The CLI checks the complete physical input set before publication; those host paths are safety guards only and never become semantic or build identity.

14.4 Identity and artifacts

The exact locked dependency closure is carried through semantic and canonical IR and BackendArtifact JSON. It contributes to selected/build identities, formal proof-cache keys, and synthesis-cache keys. Changing dependency

content therefore invalidates results even when emitted RTL text happens to remain identical.

`artifact_hash` keeps its narrower meaning: the hash of emitted backend text. The separate build identity combines that text with the logical root module and locked semantic closure. This distinction permits byte-identical RTL to be recognized while preventing evidence from one dependency closure being reused for another.

14.5 Deliberate boundaries

The current project model has no registry or semantic-version solver, editable global packages, implicit network access, Git submodules/LFS/subdirectories, wildcard imports, member renaming, or re-export. Implementation profiles are a separate compiler-policy layer and do not change dependency resolution identity. Path dependencies declared by a Git package are also deferred in this bounded slice; use another pinned Git dependency instead of reaching outside a fetched checkout.

Ordinary compilation remains offline; resolved module contents and their recorded digests are the authoritative dependency identity.

15 Optimization and formal verification

15.1 Canonical and selected IR

Compilation retains a high-level typed representation and a selected architecture representation. Metadata includes canonical type, width, signedness, latency, initiation interval, domain, purity/effects, source origin, and separate estimated/measured cost evidence.

The pure e-graph layer uses the dependency-pinned egglog engine for a bounded, type-safe scalar rewrite subset. It covers exact integer/fixed arithmetic, bitwise/shift, compare/mux, resize and wiring nodes. Fixed conversion is an opaque quantization boundary. Reassociation of ordinary carry-growing adds, movement across rounding/rescale, state, timing, protocols, storage, rules, and CDC are excluded. Source `equiv` declarations register only accepted exact same-cycle value rules; they are not assertions or temporal equivalence. The current internal responsibility split and fail-closed capability/barrier registry are documented in E-graph and physical optimization architecture.

15.2 One implementation-policy path

The staged flow is explicit; egglog is a typed value-alternative producer, not the pipeline or architecture search engine:

```
typed value IR
    -> optional egglog pure-value alternatives
    -> typed computation DAG
    -> bounded generic/resource covering
    -> target-aware exact-N fixed-latency scheduling
    -> deterministic cost selection
    -> optional formal-aware semantic-reference proof gate
```

Timing alignment validates latency/II for eligible candidates. That validation is not, by itself, a formal proof.

The arithmetic exploration tutorial demonstrates an exact eight-product expression, topology-only selection versus an internally registered pipeline, real latency-aware equivalence/mutations, and independent FPGA timing measurement. Solver success and estimated frequency must not be reported as routed 100 MHz timing closure.

The canonical source form is `implement`; all compiler-selected forms normalize through the same typed implementation-policy/extraction infrastructure:

```
y = implement {
    dot(a, b)
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        minimize lut
    }
}
```

`implement` first performs bounded exact-equivalence exploration for its one pure eligible value root. The source form is always retained. Checked egglog alternatives are deduplicated by a target-neutral structural signature (typed operation DAG, operation counts, depth and fanout), then the limited set of structurally distinct values is presented to the ordinary implementation providers. Constraints still filter complete implementation candidates and the objective still ranks only legal survivors; egglog does not see DSP, FF, latency, II, target or protocol costs.

The default compiler policy permits at most six saturation iterations, 4096 typed e-classes, 16384 e-nodes, sixteen raw extractions, eight retained value structures, 32 provider candidates per retained structure and 64 candidates in the complete local catalog. These are deterministic compiler work bounds, not language syntax or an optimality claim. Crossing an e-graph size bound keeps the independently checked source structure and records the limitation in the exploration report; it never admits an unchecked value or relaxes intent. Projects may lower these bounds, or raise them within compiler hard ceilings, through the selected profile's `intent-exploration` table. No wall-clock timeout participates in normal selection, so the retained subset stays deterministic. Identical exact roots reuse a bounded session/workspace structural recipe, but current origins are rebound and exact certificates are replay-checked on every use. Intent filtering, target selection, formal evidence, and the final chosen implementation remain fresh computations.

`implement` then selects applicable bounded II=1 structural CSA multiplier candidates for exact uniform integer products up to 16 bits per operand, reduction/DSP, and (when a legal clock/reset context exists) fixed-latency pipeline candidates. It does not enable unsafe reassociation, CDC, protocol adaptation, general variable-II sharing, or general retiming implicitly. Those remain compatibility/profile-controlled features with their existing legality checks.

The remaining source forms are:

- `choice(...)` for explicit equivalent arms and bounded cost selection;
- protocol `transform pipeline(auto, ...)` for the existing globally-stalled ready/valid transform.

The scalar spellings `architecture(auto)`, `pipeline(auto)`, and `explore` are retired and fail with migration diagnostics.

`intent` metric	Accepted condition	Meaning / limit
`lut`, `ff`, `dsp`, `bram`	`<=`, `>=`, `==` nonnegative integer	Hard estimated resource bound; not proof of physical usage.
`latency`	`<=`, `>=`, `==` nonnegative integer	Observable sample latency; positive bounds can enable legal clocked pipeline candidates.
`ii`	`<=`, `>=`, `==` positive integer	Initiation interval of an existing candidate. `throughput` is a compatibility alias in implementation profiles.
`fmax` / `fmax_est`	`<=`, `>=`, `==` positive MHz value	Estimated frequency bound; target evidence rules still apply.
`minimize`	`lut`, `ff`, `dsp`, `bram`, `latency`	One objective; omitted objective defaults to `minimize lut`.
`maximize`	`fmax_est` (or `fmax`)	One frequency objective; `maximize` cannot target resource/latency metrics.

An `intent` needs at least one clause; there can be at most one constraint per metric and one objective. Constraints filter candidates, then the objective ranks survivors. Unsupported metrics, contradictory contributions and an empty candidate set are errors, not relaxed requests. `ii == 4` does **not** cause four-cycle sharing of one DSP: current scalar source/pipeline candidates have II=1. `ii <= 4` may therefore select an II=1 candidate without reducing DSP count. To request a smaller DSP estimate, use a `dsp` bound/objective and inspect the selection report. The sole exception is the explicitly flow-controlled foundation described below; ordinary scalar `implement` never gains transaction state or II>1 sharing.

The tested II boundaries are easier to read as three separate contracts:

- **Scalar `implement`.** Structural multiplier, reduction/DSP, and legal fixed pipeline alternatives currently have `ii=1`; selection tests check the reported interval and constraints. This does not imply automatic multi-cycle DSP reuse or a routed Fmax guarantee.

- **Ready/valid `transform pipeline(auto, ...)`.** The supported kernel accepts one beat per unstalled edge after fill, while a stall holds the entire pipeline. Simulation and Verilator cover bubbles and backpressure. This does not promise one beat per edge while `ready=0` or independent per-stage elasticity.
- **Bounded shared arithmetic.** For `transform pipeline(auto) { implement { a*b + c*d intent { ... } } }`, the compiler may choose one non-interleaved multiply resource with `ii=4`, latency `4`, and capacity `1`; output retirement may capture the next input on the same edge. This does not add modulo scheduling, transaction interleaving, arbitrary-DAG scheduling, memory sharing, or general HLS.

An II metric is a candidate contract, not a request to synthesize a new time-multiplexed architecture. For actual resource use and timing, inspect the emitted implementation and downstream synthesis/routed reports.

Hard constraints are never silently relaxed. Estimated cost and measured synthesis evidence stay distinct. Automatic protocol adaptation, CDC insertion, general retiming, variable latency, and II-changing sharing are not inferred.

15.2.1 Explicit choices

```
y = choice(auto, minimize=lut, dsp<=1, latency<=1, ii<=1) {
    mul_add => pipeline(1) { a * b + c }
    dsp_mac => pipeline(1) { a * b + c }
}
```

`dsp_mac` is mapping intent, not evidence that a physical DSP was used.

15.2.2 Exact and selected timing

```
y = implement {
    a * b + c * d + e * f + g * h
    intent { latency <= 3 ii == 1 dsp <= 4 fmax >= 100 }
}
```

`implement` considers pipeline candidates only in a legal clock/reset region when the intent explicitly permits positive latency. `pipeline(3) { expr }` remains exact three-cycle hardware semantics and is never lowered to an implementation preference.

For supported pure scalar DAGs the physical record is one `ScheduledValueGraph`: operations, exact stage assignment, generic or resource bindings, resource-local/fabric cuts, balancing delays, latency, II and cost provenance. Egglog never places these cuts. On Xilinx 7-Series the target planner can cover standalone multiply, MAC/add-sub, preadd-multiply and ordered signed-product cascades with DSP48E1; uncovered operations remain fabric. Unknown target timing cannot satisfy an Fmax constraint, and structural cost is never reported as a measurement.

Only frozen expression shapes are accepted. Candidate timing is checked and the result is a concrete fixed-latency expression. Target-aware fixed-FIR planning is similarly bounded and keeps final fixed-point quantization outside partial products/reduction nodes.

The first protocol-aware form is an explicit ready/valid transform:

```
input -> output {
    transform pipeline(auto, latency<=3, ii==1, fmax>=100) {
        input.payload.a * input.payload.b
          + input.payload.c * input.payload.d
          + input.payload.e * input.payload.f
          + input.payload.g * input.payload.h
    }
}
```

This form **works** for the tested `ElasticPipelineAuto` example: semantic simulation checks first beat, bubbles, reset and global stall, and generated SystemVerilog is exercised with Verilator. It accepts one ready/valid input and output in one synchronous domain and a pure supported product-reduction kernel. All data registers and the valid chain share one global clock enable: they advance only when the output is empty or ready.

Property	Supported contract
Unstalled latency	Selected fixed value `L`.
Unstalled initiation interval	`1` (`ii_no_stall=1`).
Capacity	`L` beats.
Backpressure	Wall-clock latency may grow; output data/valid hold under stall.
Not supported	Independently elastic stages, user registers/rules/storage, CDC, adapters or protocol-control captures inside the kernel.

This is a bounded protocol transformation, **not** a general automatic pipeline/retiming feature. Physical DSP binding additionally needs a target resource compatible with the transform's clock-enable/stall behavior; `preferred` may use generic RTL and `required` rejects an unsupported binding.

15.2.3 Capacity-one temporal sharing foundation

The first II>1 candidate is intentionally available only *inside* a typed ready/valid transform, never for an ordinary wire/scalar `implement`. Its current accepted shape is exactly two uniform integer products followed by one integer addition:

```
input -> output {
    transform pipeline(auto) {
        implement {
            input.payload.a * input.payload.b + input.payload.c * input.payload.d
            intent { latency <= 4 ii <= 4 dsp <= 1 minimize dsp }
        }
    }
}
```

`II` is an accepted-transaction capability, not a decorative `ready` waveform. After `input.valid && input.ready`, this capacity-one implementation blocks admission while its three scheduled operations run and while its result is pending. On a prior `output.valid && output.ready` retirement it may capture the next input on that same edge; under output backpressure, `valid` and payload hold stable and `input.ready` remains low. Latency is the unstalled accepted-input-to-output-valid distance; capacity is separately recorded as one.

The semantic e-graph owns **what** exact value may be computed; the temporal scheduler owns **when**, resource binding owns **where**, and transaction equivalence is separate. A capacity-one transaction miter checks accepted inputs against retired outputs, including same-edge retirement and reload, in bounded formal runs. The route is bound through the selected `BackendArtifact`; `available` records advisory evidence and `required_bmc` requires `bounded_pass`. It is not an unbounded proof: `required_proven` rejects explicitly. Fixed-latency scalar equivalence is never reused for this temporal candidate.

This public-alpha foundation has a deliberately hard boundary.

Supported:

- one ready/valid input and one ready/valid output in the same clock/reset domain;
- the exact integer kernel `a*b + c*d`;
- one shared multiplier, non-interleaved execution, capacity one, latency four, and II four;
- same-edge output retirement and next-input capture under normal ready/valid transfer rules;
- transaction-stream BMC as bounded candidate-selection evidence.

Unsupported:

- scalar/fixed-rate II>1 `implement` regions;
- overlapping transactions, modulo scheduling, arbitrary operation DAGs, or more products;
- memory or CDC sharing, fixed-point temporal sharing, and nested protocols;
- `required_proven` or any claim of unbounded temporal proof.

The checked public example is `examples/temporal_shared_multiply.zhl`.

Target and resource descriptions under `std.target.*` and `std.arch.*` are compiler-shipped source. Functional modules do not name vendor registers or primitives. Manual `required`, `preferred`, and `generic` architecture modes determine failure/fallback behavior; selecting a target alone does not start automatic search.

15.3 First-class verification goals and contracts

Verification declarations are a verification-only overlay over the typed module. Standalone safety and bounded-reachability goals are named:

```
assert count_within @ clk {
    count <= DEPTH
}

cover reaches_done @ clk {
    state == State.Done
}
```

Related requirements and goals can share a contract scope:

```
contract fifo_behavior @ clk {
    require legal_input {
        input_length <= 1500
    }

    assert count_within {
        count <= DEPTH
    }

    ensure output_shape {
        !output.valid | output.payload.ok
    }

    cover full {
        queue.full
    }
}
```

`assert` and `ensure` are same-cycle safety obligations. `cover` is bounded reachability and never constrains the design. `require` is an environment-owned precondition local to its contract; all requirements in one scope are conjoined and gate that scope's goals. An `ensure` must observe at least one implementation-owned public output and cannot depend on hidden state. Goal/clause names are mandatory and unique. `@ clk` may be omitted only when exactly one clock/reset domain exists; reset is inferred from that domain and suppresses sampling.

The existing forms remain source-compatible:

```
assume bounded @clk disable iff rst {
    (a < 8) & (b < 8)
}

guarantee sum_matches @clk disable iff rst {
    y == a + b
}
```

Legacy `assume` normalizes into a module-global requirement and `guarantee` into a module-global assertion. Every body has ordinary ZLang type `bit`. Cross-domain references and unsupported temporal forms are rejected. This slice does not add arbitrary SVA/SMT, liveness, eventual delivery, fairness, or temporal implication syntax.

An `assume` may reference environment-owned leaves only. In particular, a ready/valid or request/response `.transfer` combines an environment input with an implementation output and is therefore not a legal assumption predicate. `guarantee` remains free to observe both sides. Executable declarations lower to typed structured predicates; report strings and RTL names are never parsed to recover their meaning.

The simulation `VerificationMonitor` samples after combinational settle and before edge commit. Failed active `assert`/`ensure` goals are source-attributed DUT failures. Violated `require`/`assume` clauses are recorded as environment violations and gate their dependent goals. Cover records only its first witness cycle; a missing simulation witness is not a failure.

`--contracts-sva` emits supported bindable safety goals through the same structured predicate meaning as safety

verification. Missing observations remain explicitly non-executable rather than falling back to a second expression walker.

Each non-empty assumption set has a feasibility cover. A compile-time false requirement is rejected; a dynamically unwitnessed requirement makes dependent safety success `unknown` with a vacuity diagnostic rather than a pass.

Executable formal routes consume the exact typed physical-domain contract. With `power_up unspecified`, safety verification/source safety and cover, existing bindable recursive safety verification observations, fixed-latency II=1 semantic-reference equivalence, and the corresponding formal-aware selection policies support rising/falling edges, synchronous or raw asynchronous assertion, both polarities, and the existing two-active-edge synchronized release. Asynchronous formal execution remains single-domain; multiple synchronous domains may still produce independent goal-local jobs. Same-cycle pure candidate equivalence remains reset-independent.

The checker normalizes external polarity to one active-high effective reset. For synchronized release, checker history, property guards, feasibility covers, fill masks, and comparison windows stay reset-masked for both release edges after the raw pin deasserts. Formal traces publish both `physical_reset` (the raw external pin) and `trace:reset` (the effective reset); result `reset_state` is the effective view. Unsupported contracts are never coerced to the legacy reset model.

15.3.1 Execution and immutable bundles

Start with the runnable verification tutorial for a proved state invariant, an intentionally rare overflow counterexample, scoped assumptions and RV stall checking. It includes the important case where shallow BMC passes a broken design and deeper replay finds the defect.

```
zlang design.zhl --verify
zlang design.zhl --verification-bundle build/verify
zlang verify build/verify
```

`--verification-report PATH` plus `--verification-format text|json` publishes a structured result. `--verify-require checked|proven` selects the requested safety level. `--formal-jobs N` controls independent safety/cover bundle jobs and independent selected-candidate sites. Dependencies inside one candidate site remain ordered and report ordering remains deterministic. `--verification-work-dir DIR` and replay's `--work-dir DIR` retain generated SBY inputs, logs, and VCDs outside the immutable bundle. A bundle contains a hash-validated manifest, structured verification IR, implementation/formal artifact, source map, separate safety/cover harnesses, and exact ROM companions when needed. Engine, solver, depth, timeout, tool versions, logs, and results belong to replay execution rather than the immutable source bundle identity; the replay tool regenerates execution-specific SBY configuration.

The immutable bundle and separate run report retain deterministic identity, route provenance, staged BMC/prove evidence and per-job work/tool attribution. Joint compiler execution also links candidate semantic-reference equivalence evidence. Retained VCD frames are mapped through the bundle bindings to semantic signal IDs for source-facing witness and counterexample values. Work paths and raw logs remain reproducibility data, not semantic or run identity.

A `proven` request is never a direct shortcut to prove mode. BMC executes first at the requested depth; every safety job must be `bounded_pass` before those safety jobs enter prove mode. Covers execute once and are not rerun. An unrelated cover miss/skip does not block proof, while a feasibility-cover miss first makes its dependent safety job `unknown` and therefore blocks proof.

Safety statuses remain `failed`, `bounded_pass depth=N`, `proven`, `unknown`, and `skipped`. `bounded_pass` is never proof. Cover has a distinct vocabulary: `witnessed cycle=N`, `bounded_unreached depth=N`, `unknown`, and `skipped`. `bounded_unreached` is neither `proven` nor `unreachable`, and an ordinary cover miss does not make the command fail.

Exit status `0` means the requested safety level was satisfied. A safety or source counterexample, or an executed semantic-reference equivalence counterexample, returns `1`. Unknown, skipped, or vacuous verification; tool/configuration failure; or a requested proof with only bounded evidence returns `2`. A cover miss alone is not a failure, and unavailable advisory candidate evidence does not alter formal-aware selection eligibility.

15.4 Formal layers

Variable-latency elastic ready/valid transforms are deliberately outside the semantic-reference equivalence fixed-latency relation. They never enter that route by treating their minimum unstalled latency as wall-clock latency.

The one capacity-one `a*b+c*d` temporal candidate instead has a separate transaction-indexed BMC route. Its proof recipe binds the relation identity, implementation identity, exact direct-SV artifact and bindings, harness, depth, tools, and reset contract. This route may satisfy `required_bmc` only with `bounded_pass`; it cannot satisfy `required_proven`.

Decisive formal-aware selection results are route-bound data, not trusted callback booleans. The property, harness, assumptions, backend route, semantic-reference artifact, implementation artifact, engine, mode, and depth must all match the cache identity before a candidate can become eligible. Exact reuse additionally matches stage policy, timeout, tool snapshot, dependencies, and compiler schema. Failed results require typed counterexample metadata; non-failed results reject it. Decisive semantic-reference equivalence evidence requires a positive depth. Missing `yosys-smtbmc` is reported like any other genuinely unavailable formal tool and never becomes false success. An `unknown` result or timeout in a required mode terminates the selection: the compiler never lets solver runtime become an implicit architecture objective.

`bounded_pass` means no counterexample was found through the stated BMC depth; it is not an unbounded proof. Only `proven` satisfies `required_proven`. `failed`, `unknown`, and `skipped` do not satisfy required policies.

Semantic analysis creates candidate spaces but runs no backend or solver. A compiler-owned candidate-site ledger carries stable site identity and exact rank into the selection phase, where `--formal-policy` executes the formal-aware selection route. Bundle-only publication records one strict compiler-owned linking plan over the exact verification goal plan, candidate-site ledger, and retained formal-aware selection records. Joint `--verify` with a non-off policy enriches that immutable plan with the selected-candidate semantic-reference equivalence plan before execution; candidate results are emitted afterward in the combined report/evidence. Bundle publication may additionally freeze the exact selected-candidate semantic-reference equivalence inputs as hash-validated, path-free companions; `zlang verify` then executes those inputs without source compilation or formal-aware selection reselection. Base bundles contain no candidate replay inputs. The common evidence report validates those links without merging result types. For an executed BMC route it publishes the exact status, route, depth, property or relation identity, solver, known tool versions, and `unbounded: false`. Reset contracts and assumptions are serialized only when the compiler-owned route already represents them; the report does not infer missing context. Capacity-one temporal evidence also records the `capacity_one_transaction_stream` relation, latency 4, II 4, capacity 1, same-edge retire/reload, and that `required_proven` is unsupported. A solver matrix preserves one shared immutable problem identity and independent per-solver run identities/results, with disagreements represented explicitly; agreement never upgrades the underlying result status. Repeated candidate implementations are associated by semantic site and rank, not by candidate identity alone. With policy `off`, the ledger remains visible but there are no formal-aware selection attempt/evidence records. Candidate execution uses deterministic recipe-addressed work roots below the external verification work directory. An exact in-session formal-aware selection reuse retains its recorded work root; persistent formal-aware selection cache data deliberately excludes physical paths, so a cache hit never claims that an old workspace still exists. These paths and the discovered candidate tool snapshot are operational report data, not proof, cache, or compiler-verification-report identity. Parallel sites with an identical proof recipe share the same provider-owned workspace and retain the same complete attribution. `--formal-harness` and `--formal-sby` remain separate safety verification artifact-generation options; writing either file alone is not proof execution. An incomplete or mixed-route compatibility view is rejected with guidance to use a verification bundle rather than silently choosing one backend.

The current bounded applicability also transports observations already required by existing properties: parent request/response outstanding and directional-buffer counts, receiver-credit adapter occupancy, and direct same-domain internal ready/valid guarantees. These are formal-only typed ABI ports or root-monolithic assertions, never inferred RTL names or substituted environment assumptions. Automatic root assumptions receive a separate feasibility cover, so dependent safety evidence is vacuity-checked.

Range-proven runtime vector selection is available to the existing same-cycle predicate IR. One separate compiler-owned helper can derive whole-root same-cycle semantic-reference equivalence evidence for exactly one pure scalar child by following typed hierarchy bindings and producing formal-only Yosys namespaces. It does not flatten production RTL and does not authorize state, protocols, storage, arrays, nesting, or aggregate boundaries. Protocol-valued and aggregate-protocol top boundaries fail closed in the scalar semantic-reference equivalence entry points.

See the language support matrix and Known limitations for the accepted matrix and remaining semantic boundaries.

15.5 Reports

Useful current outputs include high-level and selected optimization IR, saturation, implementation, cost, pipeline, architecture, exploration, synthesis, artifact manifests, and structured formal results. See Check and run ZLang for commands and feature-specific design records for the exact schema.

16 E-graph and optimization responsibilities

Status: implemented.

The compiler keeps four distinct responsibilities:

```
canonical typed value IR
  -> exact typed egglog alternatives
  -> deterministic scalar DAG scheduling
  -> target resource matching
  -> ScheduledValueGraph -> formal-aware selection/semantic-reference equivalence/Direct-SV
```

Egglog changes only pure zero-latency values. It does not insert registers or name FPGA primitives. The scheduler assigns exact cycles and alignment delays, but does not invent algebraic equalities. Resource matchers bind already typed operations or subgraphs to source-described resources. `ScheduledValueGraph` is the authoritative physical bridge consumed by verification and RTL emission.

The production rewrite path has one bounded egglog engine. For an eligible `implement` site it enumerates a limited set of cheapest exact extractions, removes target-neutrally redundant structural spellings, and retains a bounded diverse subset before providers run. Extracted alternatives are admitted only after exact typed validation and a checked value-equivalence certificate. The package dependency declares its pinned version. Internal module filenames are not part of the language contract.

The optimizer is fail-closed. An operation needs an explicit capability for e-graph admission or scalar scheduling. Width, truncation, bit reinterpretation, fixed rescale, rounding, saturation, signedness changes, state and effect nodes are semantic barriers unless one specifically registered exact rule proves the requested transformation. Target resource interest is annotation only and does not change value identity.

Current resource matching supports the accepted Xilinx 7-Series DSP48E1 subset. Other resource families must satisfy the same matcher contract; they do not add vendor tests to the general scheduler or algebraic rules to egglog.

Target-independent architecture-interest labels describe shapes such as multiply-add and preadder-multiply. They neither choose a primitive nor affect semantic identity. `OperationCostModel` estimates an already chosen operation implementation; `ResourceMatcher` decides whether a typed operation fits one source-described resource. Missing timing evidence never becomes zero delay.

Current intentional separations are retained:

- canonical `ExpressionOp` capability and typed-expression class dispatch are connected by one closed mapping, but the two IR layers are not merged;
- formal observability remains binding- and ownership-dependent, so it is not inferred from pure-value capability alone;
- target planner subgraph covering remains separate from scalar operation matching because DSP cascade legality is a graph property, not a node cost.

17 Implementation profiles

Implementation profiles keep physical policy outside portable `.zhl` source. They are named tables in `zlang.toml` and are selected explicitly:

```
[profiles.release]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
allowed-transforms = ["pipeline", "dsp", "reduction"]
avoided-transforms = ["reassociate"]
objective = "maximize fmax"
```

```
architecture = "Xilinx7SymmetricDSPCascade"
architecture-mode = "preferred"
evidence-policy = "measured_required"
formal-policy = "required_bmc"

[profiles.release.constraints]
latency = { maximum = 8 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }
```

`maximize fmax` ranks legal candidates using the strongest compatible frequency evidence available under the selected evidence policy. It does not guarantee achieved FPGA-device or ASIC-silicon frequency. Reports retain the evidence kind and matching target/tool/constraint identity; a structural estimate is never presented as routed timing.

Schematic use inside a project containing the shown `src/fir.zhl` and profile:

```
zlang src/fir.zhl --profile release --systemverilog build/Fir.sv
```

A profile may select pinned scalar external-module implementations with `external-mappings = ["vendor-add "]`. Mapping declarations live in the top-level `external-mappings` tables of `zlang.toml`; their source files and hashes are fixed by `zlang.lock`. They affect Direct-SV artifact text, never the backend-independent model or implementation-policy identity.

Only the selected profile is parsed strictly. This permits a project to carry profiles for newer compilers without making unrelated builds fail. Selecting an unknown profile, selecting a profile without a project, or using an unknown key is an error. Profile data is excluded from dependency resolution and lock identity, but the normalized selected request participates in implementation and later whole-build identities.

17.1 Semantic regions

The compiler reports one stable identity for each public scalar wire-output region. A profile can address an exact subset with `regions = ["DIGEST", ...]`. Generate `--implementation-policy-report` once to obtain the 64-character digests.

Region identity is derived from the logical module specialization, typed public output binding, canonical expression identity, and exact result type. Source paths, spans, physical instances, and generated RTL names are excluded. Stale and duplicate identities fail explicitly. Profile-selected regions are root-module-only and scalar-only; recursive protocol regions are unsupported.

17.2 Normalization and conflicts

`choice(auto)` remains accepted for user-supplied alternatives. Scalar `architecture(auto)`, `pipeline(auto)`, and `explore` are retired and produce migration diagnostics. Canonical `implement` source policy is represented through the same `ImplementationRequest` model as the selected profile and explicit compiler options. Equal normalized contributions are harmless. Different values for the same target, transform set, objective, constraint, evidence policy, formal policy, or architecture are rejected with both origins in the diagnostic.

For an ordinary typed scalar output, profile transforms/constraints/objectives run through bounded exploration. Canonical `implement` regions have already run during semantic analysis, so their retained candidate table is compared but never run a second time. Retained legacy records are handled the same way for replay only; no new transform or equivalence rule is introduced.

An exact module `timing` block is immutable public behavior. Profile bounds may equal or contain that exact latency/II, but cannot weaken or contradict it.

17.3 Reading the backend selection report

`--backend-implementation-report PATH` writes a JSON explanation of what the Direct-SV backend could actually emit for the requested physical implementation. This is **not** another source-language plan or an additional backend. Inspect it when a target/profile request does not produce the resource you expected.

Report status	Meaning
`selected`	Requested physical implementation was selected and emitted.
`generic_fallback`	`preferred` route was unavailable; generic RTL was emitted instead.
`unsupported`	Requested physical route is unsupported; `required` fails compilation.
`not_requested`	No physical implementation route was requested.

Current boundaries are deliberate: one backend is selected per named profile, II is limited to existing semantics, and profiles cannot introduce protocol, CDC, memory, retiming, or formal capabilities that the typed compiler does not already support.

18 Target platform descriptions

ZLang keeps functional behavior, implementation architecture, and physical target data separate. Explicit selection maps the ordinary functional symmetric FIR example to four DSP48E1 resources on `xc7z030ffg676-1`. Without an explicit selection, the same source continues through the generic Direct-SV path. Bounded automatic selection is described under target-aware planning.

18.1 Compiler-shipped source descriptions

Target data is ordinary compiler-shipped ZLang source:

- `std.target.generic` defines the resource-free generic target;
- `std.target.xilinx.series7` defines the bounded DSP48E1 capability;
- `std.target.xilinx.xc7z030` defines the device part, inventory, and cascade capacity;
- `std.arch.xilinx7_fir` defines the one manual symmetric-FIR template;
- `std.target.asic.sky130` describes the real `sky130_fd_sc_hd` standard-cell library boundary and proves that the core records are not FPGA- or Xilinx-specific.

The core parser and IR understand generic resources, typed ports, operation and width limits, register sites, dedicated links, inventories, targets, and architecture requirements. The names `DSP48E1`, `PCIN`, and `PCOUT`, and the physical SystemVerilog binding all originate in the library definitions - not in functional ZLang or semantic branches.

The SKY130 profile intentionally emits ordinary RTL for downstream Liberty-based mapping. It records the documented `dfxtp_1`, `dfrtp_1`, and `dlclkp_1` cell identities, but does not claim a hard multiplier, SRAM, PLL, fixed device inventory, timing closure, or silicon signoff. Those inputs are PDK/project/flow-specific and must be supplied by the implementation flow.

An exact source-level `timing { latency N ii 1 }` block is public module behavior, not target-selection policy. Generic implementation graphs publish their derived `timeless`/`known`/`unknown` timing class and are explicitly backend-independent. A selected physical graph carries its realization backend so a Direct-SV resource plan cannot be attributed to another physical implementation. Target selection must preserve any exact public module latency.

The bounded Series-7 profile describes a signed 25-bit pre-adder, signed 25x18 multiply, 43-bit product, 48-bit P/PCIN/PCOUT path, optional A/B/D/AD/M/C/P register sites, and an adjacent/same-group 48-bit dedicated cascade. The XC7Z030 instance records 400 DSP slices and four locally available cascade positions; this is deliberately not a complete floorplan model.

18.2 Manual selection

```
zlang examples/symmetric_fixed_fir.zhl \
  --target xc7z030ffg676-1 \
  --target-architecture Xilinx7SymmetricDSPCascade \
  --target-architecture-mode required \
  --systemverilog build/SymmetricFixedFIR.sv \
  --implementation-manifest build/SymmetricFixedFIR.manifest.json
```

`required` turns every target, shape, width, inventory, dedicated-link, or backend failure into a diagnostic. `preferred` returns the generic graph on a mapping failure. `generic` and omission of target options preserve the

existing generic implementation. This command performs explicit selection; bounded automatic target-aware selection is documented separately in the Target-aware architecture and pipeline planning.

18.3 The selected graph

The matcher covers typed semantic structure only. It requires eight products, four coefficient expression identities reused at mirrored sample indices, and one final `nearest_even/saturate` conversion to `fixed<16,14>`. It does not use module names, source spelling, runtime coefficient equality, or test vectors.

For `SF2.10` samples and coefficients the legality evidence is:

Quantity	Required	DSP48E1 profile
pair pre-add	13 signed bits	25
coefficient/B	12 signed bits	18
exact product	25 signed bits	43
exact four-pair accumulation	27 signed bits	48

The resulting `ImplementationGraph` has four resource instances and three `DedicatedPhysicalEdge` records:

```
dsp0.PCOUT -> dsp1.PCIN -> dsp2.PCIN -> dsp3.PCIN
```

Each node maps the left/right sample expressions to A/D, the shared coefficient to B, and its predecessor/result identities to PCIN/P/PCOUT. Every internal DSP register site is explicitly zero in this configuration. The ordinary semantic output register provides latency 1; II is 1. The complete 48-bit cascade result then crosses the single original quantization boundary. No intermediate is narrowed.

18.4 Backend and manifest

The Direct-SV emitter consumes the selected graph; it does not rediscover a FIR. Its source-selected physical binding instantiates four Series-7 `DSP48E1` primitives with `INMODE=00100`, `OPMODE=0010101`, `USE_DPORT=TRUE`, direct A/B, and all selected internal register stages disabled. Actual `PCOUT` ports drive the next primitives' actual `PCIN` ports. A distinct behavioral resource model is used only for Verilator simulation.

The versioned implementation manifest retains semantic bindings and target/family/template identities, dependency hashes, selection policy, resource configuration, dedicated edges, latency/II, intended counts, and the emitted artifact hash. Intended resource use is not a vendor measurement.

18.5 Boundaries

The generic Direct-SV implementation remains available when no primitive graph is selected. Existing semantic-reference equivalence can validate the semantic fixed-point region, but its reference emitter does not model vendor primitives. Physical evidence therefore consists of the exact semantic oracle, Verilator execution of the separate resource behavior model, and real Vivado synthesis/place/route of the primitive artifact. No primitive-level formal infrastructure was added.

The automatic planner covers only its documented symmetric-FIR and signed-product regions; general placement/graph covering, DSP48E2, Intel physical emission, automatic storage/clock-resource mapping, and new formal machinery remain outside the current bounded support.

19 Target resources

This low-level layer describes legal resources and physical bindings; it does not itself select a design. The separate accepted high-level target-aware planner automatically selects the bounded symmetric-FIR and signed-product candidates documented there. Bounded ported-memory planning can match an exact advertised memory shape, replicate 1R1W resources for 1W+nR, or use a bounded register/mux fallback. Automatic banking and clock-resource planning remain unsupported.

19.1 Implemented model

Compiler-shipped ZLang source now defines generic, AMD 7-series, Intel Cyclone V, and ASIC-compatible resources. Resource definitions carry generic class, operation, capabilities, typed ports/limits, semantic pipeline sites, legal pipeline configurations, dedicated edges, and an opaque backend binding.

The loader validates duplicate/unknown pipeline sites, configuration latency, II, memory port/width/capacity, clock input/output/output-count requirements, dedicated edges, and target inventory. The same parser accepts project-local resource declarations; resolving project packages is deliberately deferred.

The generic library contains ordinary-RTL descriptions for logic, registers, multiplier/add/MAC, FIFO storage, RAM/ROM, clock/control abstractions, and carry. Selecting `generic`, or compiling without a target, leaves typed semantic IR and Direct-SV semantics unchanged.

19.2 DSP48E1 pipeline binding

The bounded symmetric FIR maps through source-described `DSP48E1` resources. Its semantic sites are:

```
input_preadd -> multiply -> accumulate_output
```

The physical binding maps them to A/B/D registers, MREG, and terminal PREG. Only configurations with compatible typed latency, II, and target capabilities are eligible. Routed measurements, if requested by policy, must match the exact physical graph and constraints. Separate target-resource validation records contain measured configurations; their QoR numbers are not portable language semantics or implicit guarantees for a different design.

19.3 Signed-product cascade

The same source-defined DSP48E1 binding now also covers the ordered `SignedProductReduction` graph used by the FFT scalar real and imaginary components. The Direct-SV emitter consumes the per-resource `accumulator_plus_product/accumulator_minus_product` configuration and emits ALUMODE controls without rewriting the finite-width expression. The final fixed-point conversion remains one typed boundary after the cascade.

Routed evidence, graph identities, and tool invocations belong in separate target-resource validation records. Evidence for another packed ABI, target, constraint, or graph is not reused.

19.4 RAMB36 validation

`examples/target_bram_memory.zhl` is an ordinary 1024x36, one-cycle, read-first synchronous memory with independent read/write addresses. Generic Direct-SV emits ordinary RAM RTL. The manual binding `Xilinx7BRAM36SimpleDualPort` consumes one RAMB36 capability and emits the same semantic template with block-memory physical binding.

The binding requires the resource's exact read-first collision contract; write-first bypass across independent addresses is not silently reinterpreted as a simple-dual-port primitive. Generic RTL may still be selected under an appropriate fallback policy. Resource inference measurements belong to target validation reports rather than the language contract.

Named true-dual selection is a distinct RTL shape. Generic same-clock multiport storage retains one deterministic process, but the selected Xilinx 2RW route requires `contents preserve` and emits one physical process per port. A uniform `init VALUE` is preserved as a power-up/bitstream initialization under that reset policy. Native inference of a resource in a particular synthesis run does not upgrade a cross-clock collision model to an exact vendor guarantee.

The manually selected FIFO-only architectures `Xilinx7AsyncFifoRAMB18` and `Xilinx7AsyncFifoRAMB36` require the compiler-owned registered-read `async_fifo` decomposition, an independent-clock 1W1R port shape, exactly one read cycle and `DO_REG=0`. They do not select a public `async_mem`. Its physical plan owns ordered, width-aware pointer, Gray, full, ready/valid and prefetch equations. The Direct-SV emitter renders those equations and the digital simulator evaluates the same equations. Separate target-resource validation records hold synthesis observations; physical clock/reset and metastability requirements remain implementation responsibilities.

The live resource capability schema now distinguishes `simple_dual`, same-clock `true_dual`, and independent-clock `1W1R`; it also separates same-clock from cross-clock collision guarantees. A legacy `true_dual` flag is not sufficient evidence for an asynchronous collision contract. The current 7-Series data intentionally advertises no

exact cross-clock old/new guarantee, so strict target-required `async_mem` publication fails closed while generic Direct-SV remains an explicitly labelled structural digital model.

19.5 Intel and ASIC status

Cyclone V source data describes variable-precision multiplier modes, 64-bit accumulation, chain connectivity, generic pipeline sites/configurations, M10K port/width modes, ALM/FF distinctions, and a fractional PLL resource. The same generic legality checks used for RAMB/MMCM validate M10K/PLL requests.

Intel primitive emission is unsupported: a selected Intel physical binding fails closed with `unsupported`. The current release publishes no Quartus-backed synthesis or QoR validation for this target. The compiler-shipped `sky130_fd_sc_hd` profile identifies the real high-density standard-cell library plus its documented positive-edge DFF, active-low-reset DFF, and integrated clock-gate cells. Direct-SV remains generic RTL for downstream Liberty mapping. SKY130 SRAMs, PLLs, die-area inventory, PVT corner, timing closure, and physical signoff are deliberately not fabricated by the compiler; project macro libraries and the ASIC implementation flow own them.

Emit technology-neutral RTL carrying the SKY130 target identity with:

```
zlang design.zhl --top Top --target sky130-fd-sc-hd \
  --systemverilog build/Top.sv
```

The downstream synthesis/P&R invocation must still supply the selected `sky130_fd_sc_hd` Liberty, LEF and process configuration.

19.6 Clock-resource blocker

The target libraries can describe clock input/output ranges, output counts, phase capability, and lock/reset metadata. Current functional ZLang clock domains do not express a frequency/phase relationship from which a truthful PLL/MMCM configuration could be derived. Physical clock emission therefore remains unsupported: there is no source-level frequency/phase requirement from which to select a configuration. Raw PLL generics are not exposed in normal modules.

19.7 Implementation artifact contents

Implementation manifests now retain selected pipeline configuration, active semantic sites, physical-binding identities, and separate intended, emitted, and measured resource-count slots plus timing provenance. Existing semantic bindings remain independent. Current compiler-produced artifacts populate intended/emitted counts after successful physical emission; external vendor tools own measured counts and routed timing.

Automatic target-aware planning is enabled only for the reviewed symmetric-FIR and ordered signed-product reduction regions documented in the Target-aware architecture and pipeline planning. General `explore`, BRAM/-clock selection, and arbitrary resource ranking remain disabled. The accepted boundary is documented in the Target-aware architecture and pipeline planning.

20 Target-aware architecture and pipeline planning

The bounded planner is implemented for two exact fixed-point region families on `xc7z030ffg676-1`: the symmetric FIR cascade and ordered signed-product reductions used by complex multiply (`p0 - p1` and `p0 + p1`). It extends the existing architecture alternatives/exact reduction planning/pipeline scheduling/timing alignment/deterministic cost selection/bounded exploration path; it is not a second exploration engine.

20.1 User contract

```
result = implement {
    quantize<fixed<16,14>>(acc) {
        round nearest_even
        overflow saturate
    }
    intent { latency <= 8 ii == 1 fmax >= 100 }
}
```

`ii` and the compatibility spelling `throughput` normalize to one canonical II constraint. Fmax values are MHz. `latency<=N` is a bound. `latency==N` is an exact observable sample latency. `maximize fmax` is an evidence-ranked selection objective, not an achieved-frequency guarantee.

The target is build configuration, for example:

```
zlang examples/symmetric_fixed_fir_auto.zhl \
  --top SymmetricFixedFIRAuto \
  --target xc7z030ffg676-1 \
  --target-evidence-policy measured_required \
  --systemverilog build/SymmetricFixedFIRAuto.sv \
  --implementation-manifest build/SymmetricFixedFIRAuto.json \
  --pipeline-report build/SymmetricFixedFIRAuto.report
```

Functional source never names DSP48E1, MREG or PREG. With no target, or with `--target generic`, the ordinary generic candidate and backends remain active.

20.2 Candidate generation and legality

The symmetric-FIR region is exactly an eight-tap signed fixed reduction where four coefficient semantic values are reused at mirrored sample indices. Runtime coefficient equality is not symmetry. Eight independent coefficients therefore remain generic. The signed-product region is derived from the already typed `SignedProductReduction`: ordered product terms and each add/subtract join are preserved, and the resource must advertise the required accumulator modes. Arbitrary subtraction trees or reassociation are not inferred.

The candidate set is:

1. generic implementation;
2. source-described symmetric cascade with no resource-local site;
3. multiply site selected;
4. multiply and terminal output sites selected;
5. input/pre-add, multiply and terminal output sites selected.

These are the four configurations published by the resource library. Core code does not enumerate primitive register bits. The signed-product family applies the same configuration names to its exact two-resource chain. Before cost extraction the planner validates the typed pattern, exact pre-add/product/accumulator widths, required add/sub capability, inventory, dedicated edges, II, and latency.

The final `FixedConvert` stays after the full 27-bit/F20 accumulation. A mapping requiring hidden narrowing is rejected and the generic candidate remains.

20.3 Timing DAG

Every selected target graph carries a backend-independent timing DAG with stable semantic/implementation identities. It records resource combinational segments, selected resource-local cuts, the three dedicated cascade edges, fabric boundaries, final fixed quantization and output boundary.

Timing alignment produces explicit `alignment` delay objects. Exact external latency produces separate `compensation` objects. Both record cycles, width and fabric FF cost in the graph and manifest. They are never reconstructed from RTL names.

For the selected useful latency-three implementation:

- `latency<=8` adds no delay;
- `latency==8` adds one explicit five-cycle output compensation delay;
- resource-local useful sites remain unchanged.

The Direct-SV emitter consumes this graph. The base output boundary and five compensation cycles form the required output register chain; source `pipeline(N)` latency is not added again.

20.4 How target evidence affects selection

This topic belongs here only as a practical distinction: an estimated resource cost is not a measured FPGA result, and an estimated Fmax is not routed timing.

Evidence kind	What it establishes
`structural_estimate`	Compiler estimate from the selected structure; no vendor run.
`synthesis_measurement`	Result after synthesis, not placed/routed timing closure.
`routed_measurement`	Measured result after implementation/routing for matching target and constraints.

`measured_required` accepts only compatible routed evidence for an Fmax constraint. Structural latency/II remain authoritative and are not erased by that policy. Compatibility includes target/part, architecture, complete graph hash, pipeline configuration, backend, clock constraint and tool/version. `measured_preferred` ranks the strongest compatible evidence cohort before the numeric objective and falls back to estimates only when no legal stronger candidate remains. This prevents a coarse structural Fmax estimate from being compared as though it were an equally authoritative routed measurement.

Packaged routed evidence is bound to the matching part, constraints, physical graph, pipeline configuration, backend and tool recipe; a different graph's measurement cannot be reused as proof of this one. The optional `--implementation-manifest PATH` records the selected configuration, resource and timing decisions and their provenance for inspection or replay. Internal JSON schema numbers are compatibility details, not language features or knobs users should set.

20.5 Current boundary

Automatic target planning is intentionally limited to the symmetric-FIR and signed-product Direct-SV regions above. BRAM, PLL/MMCM, Intel physical planning, arbitrary graph covering, II-changing sharing, and automatic fixed transformations are not enabled. Generic Direct-SV remains the fallback implementation. Existing formal infrastructure is unchanged and no primitive-level proof claim is made.

21 Platform constraints

21.1 Clock constraints

A selected project profile can publish one explicitly named clock period:

```
[profiles.release.platform.clocks.clk]
period-ns = 10.0
```

The clock name must match the one typed `ClockDomain` on the selected top. The period is physical build configuration: it is not semantic IR, is not inferred from `fmax`, and is not merged into `ImplementationRequest`.

The CLI publishes a constraint only together with exactly one backend artifact. The following command assumes a project-local `top.zhl` and the `release` profile shown above:

```
zlang top.zhl --profile release --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc --constraints-sdc build/Top.sdc
```

The compiler resolves the RTL port through the artifact's typed `CLOCK` binding, never through generated-name guessing. XDC and SDC initially contain only:

```
create_clock -name clk -period 10 [get_ports {clk}]
```

Each generated constraint artifact retains its backend hash, selected-IR identity, clock edge, and complete reset mode/polarity/release-cycle/power-up metadata. For a non-default contract, publication also requires an exact physical-domain record in the implementation artifact; stale or mismatched reset semantics are rejected. Whole-build manifests publish the file as a backend companion and include the platform profile and constraint identities. XDC/SDC output is deterministic and atomically published without following symlinks.

No reset false path is emitted: asynchronous-reset recovery/removal timing must not be hidden. This constraint publication supports only the exact single-clock profile shown here. Multiple clock domains are supported in language semantics and Direct-SV emission, but generated clocks, multi-clock XDC/SDC publication, pin/-package/IO properties, input/output delays, CDC constraints, and formal properties are not emitted by this facility.

22 Direct-SV

The backend consumes typed semantic IR. It never dispatches on AXI, APB, or RegBus module names: source-authored standard-bus components use the same aggregate endpoint, hierarchy, register, rule, and connection lowering as user modules.

Ordinary scalar register/rule components share the standalone/composed dispatch and register contributor. Independent rules do not require global scheduler enumeration merely because a module is instantiated as a child. Hierarchy validation is cached only within one emission; protocol/storage effects and conflicting multi-effect rules keep their existing exact paths. Generated private names now use a shared hierarchy-local allocation plan: `cfg_ready`, `lane_0`, `lane_0_value`, `result_pipe_s1`, and short specialized definitions such as `Counter_s1a2b3c4d`. Source names win over generated helpers; actual collisions receive a deterministic suffix. Public top/port ABI and source register names are unchanged, including existing public aggregate-leaf separators. BackendArtifact records the naming schema and complete semantic identities separately from regenerated RTL/VPI locators. Naming does not change circuit semantics or imply QoR gains.

Formal-only `rule.fire` now uses the same effective reset and polarity as production state, including the two-edge synchronized release and conditioned child reset. Real RTL covers both polarities, both active edges and nested hierarchy. Its separate global-guard enumeration scalability limitation remains: large independent-rule formal projections remain outside the scalable supported subset.

Fixed-point ports and expressions use the canonical scaled-integer types behind `fixed`/`ufixed`, concise `SF`/`UF`, and `_Sat` formats. Same-scale target narrowing and explicit rescaling are both emitted from `FixedConvert`; the backend never infers wrap, saturation, or rounding from source spelling.

22.1 Supported composition subset

- scalar and packed struct/vector/tuple datapaths, `char`/`string<N>` aliases, immutable locals, muxes, field-/projection access, range-proven runtime packed-bit indexing/fixed-width packed slices and runtime vector indexing with LSB-zero semantics, exact-width `bitcast`, homogeneous vector `concat`, compile-time `reshape` , reductions, fixed-point arithmetic/conversion, and selected or nested fixed-latency scalar pipelines with II=1;
- registers, next-state assignments, atomic rules, FSM lowering, reset, and priority in one or more explicitly owned domains, including one independent two-register asynchronous-assert/synchronous-release conditioner per applicable domain;
- reusable parameter specializations, distinct scalar physical child instances, and compile-time indexed combinational or single-domain sequential instance arrays, including aggregate scalar outputs and scheduled FIFO state combined with ordinary child registers/rules;
- mixed scalar/state/ready-valid/request-response children;
- mixed scalar plus ready/valid instance-array children, bounded nested same-domain scalar/direct-ready-valid array hierarchy, and first-level indexed in-order request/response requester/responder arrays;
- read-only runtime projection of one exact bit-packable scalar/aggregate wire output from an instance array; every physical child remains independently instantiated and active;
- the bounded single-input/single-output ready/valid `transform pipeline(auto)` with one pure pipeline scheduling product-reduction kernel and a compiler-owned global stall;
- ready/valid connections and finite connection FIFOs, including simultaneous push/pop and backpressure;
- scalar ready/valid `async_fifo(N)` clock crossings and aggregate crossings whose schema has exactly one forward ready/valid member; aggregate payloads cross as one packed atomic value;
- vendor-neutral scalar `sync_level`, `pulse_toggle`, and one-entry ready/valid `handshake` crossings, using the same coordinated-reset and latency semantics as the simulator;
- synchronous memories with the currently typed collision/read-latency model;
- fixed-priority and round-robin packet arbiters with beat- or packet-scoped grants;
- in-order hierarchical request/response channels with independent directional buffers as represented in typed IR;
- one mandatory public top ABI derived from `TopPhysicalABI`: structs are recursively exposed as named field leaves, tuples as deterministic `itemN` leaves, while every `vec<N,T>` leaf is a multidimensional packed SystemVerilog array (`logic [N-1:0][W-1:0] name` for a `W`-bit element); flat packed aggregate representations exist only as deterministic aliases inside the selected top or inside compact child components;
- source-authored AXI4-Lite, APB, AXI-Stream, Wishbone, RegBus, and CSR hierarchy used by the current standard-library and real-design examples;
- hardware-connected CSR `ro`, `pulse`, and sticky-W1C behavior with the frozen software/hardware priority policy;

- ready/valid-to-credit, credit-to-ready/valid, and per-VC credit source state machines from typed protocol/capacity metadata;
- recursive instance/state locators and exact physical-domain contracts for non-default reset modes. Specialization identity is not used as physical instance identity.

SystemVerilog reserved words are deterministically prefixed with `zlang_`. The collision check runs after that physical-name mapping, so two distinct semantic leaves can never silently become one RTL port. Inline-boundary helper signals are allocated outside the public namespace and remain deterministic. Packed dimensions use conventional descending ranges. ZLang element zero maps directly to physical packed index zero and the least-significant private slice. The generated artifact has one unconditional top definition and is accepted by both Yosys and Verilator without backend preprocessor branches. Unresolved locals are eliminated before backend lowering. Unsupported IR shapes raise `SystemVerilogEmissionError`; no artifact is published after a failed emission.

Ordered comparisons preserve typed signedness explicitly: both operands of `<`, `<=`, `>`, and `>=` are rendered through `$signed` or `$unsigned` according to typed IR, independent of whether an operand is a port, local, projection, or register. Equality and inequality remain raw bit comparisons.

Use the stable CLI option:

```
zlang examples/simple_dma.zhl --top SimpleDMA \
  --systemverilog build/SimpleDMA.sv
verilator --lint-only --top-module SimpleDMA build/SimpleDMA.sv
```

The retired hidden `--experimental-systemverilog` option is deliberately rejected. `--systemverilog` is an explicit artifact sink: it writes only the requested SV file and leave stdout empty. A bare `zlang SOURCE` invocation emits the production Direct-SV artifact to stdout. Diagnostics remain on stderr.

Use `--verbose` when an explicit success confirmation is useful:

```
zlang design.zhl --systemverilog design.sv --verbose
```

This keeps stdout artifact-safe and reports the selected top and written paths on stderr. To validate parsing, top selection, and semantic analysis without publishing any backend artifact, use:

```
zlang design.zhl --check
```

`--check` prints a one-line success result and returns zero. Without `--top` it checks every module declared in the source file, preventing an invalid dormant module from being hidden by default-top selection. With `--top`, it checks only that selected elaboration root. Syntax, semantic, and top-selection failures retain their normal nonzero status and diagnostic. As a check-only mode, it cannot be combined with artifact output options.

22.2 Simulation-only architectural state access

A Direct-SV build may publish a separate Verilator VPI companion without adding ports or changing the production RTL:

```
zlang design.zhl \
  --systemverilog build/design.sv \
  --simulation-state-bundle build/state-access
```

The companion manifest covers the selected typed hierarchy and is bound to the exact emitted artifact hash, build identity, state shapes, and emitter-owned physical locators. Publication re-hashes the `.sv` file, so a stale or modified RTL file cannot receive an access manifest. The generated C++ header requires Verilator `--vpi --public-flat-rw` and validates its complete state allow-list before access.

The bounded surface includes bit-packable user registers (including vector registers), writable-memory cells, and the persistent read-result latch of a one-cycle memory. Values up to 64 bits have convenience methods; arbitrary-width scalars and elements use exact 32-bit least-significant-word-first arrays. Vector element zero retains the canonical least-significant packed position. The persistent semantic simulator consumes the same catalog and keeps one state object per physical child instance.

22.3 Formal boundary

Real Z3/SymbiYosys regression targets exercise emitted direct RTL for register transitions and rule priority, ready/valid stability and FIFO accounting, CSR W1C behavior, and request/response acceptance. Arithmetic, state-transition, FIFO-accounting, response-accounting, W1C, and priority mutations all fail with counterexample metadata.

The first-class verification-bundle path also accepts Direct-SV designs with initialized ROM. Exact memory images are published below `implementation/companions/` and listed as hash-validated inputs for every SBY job that consumes them. Generated solver configuration, logs, and VCDs are retained outside the immutable bundle; the run report maps sampled physical VCD signals back to semantic binding IDs for witnesses and counterexamples.

Direct-SV is the executable bundle route. If its formal emission cannot bind a required observation, the goal is explicitly skipped or rejected according to policy. Unsupported aggregate/protocol shapes report a non-executable reason; no generated-name reconstruction or alternate backend substitution is used.

`RegBusCSRTarget` stores response data and holds `response.valid` until `response.transfer`. Source-authored AXI-Lite and APB frontends can therefore complete write transactions through the typed RegBus boundary.

The implementation artifact publishes recursive physical locators only for signals actually present in emitted RTL. Current Direct-SV formal emission publishes the typed accepted-request, directional-buffer occupancy, response-consumption, and parent-owned outstanding-ledger observations used by the supported recursive request/response properties. Register, FIFO, CSR, and rule-fire observations are likewise executable where their complete typed binding sets exist. A `formal_observation_token` remains absent for hidden or unsupported state, and the affected property reports `skipped` rather than fabricating a proof.

22.4 Explicitly unsupported

Legacy globally controlled storage combined with user state, out-of-order or nested request/response arrays, additional protocols on a request/response array child, arbitrary nested storage/CSR/aggregate protocol hierarchy, CDC arrays, and runtime-selected inputs, protocols, or actions remain fail-closed. The backend never silently emits only one element or reconstructs an indexed connection from an RTL name.

Sequential ready/valid modules and role-qualified standalone in-order `request_response` requester/responder modules use the same typed clock/reset, ownership, and accounting semantics as their hierarchical forms. A standalone endpoint owns a local unbuffered ledger: request transfer increments it, response transfer decrements it, and reset starts a new empty epoch. Stateful generic child templates that still require parent specialization remain validated through their concrete parents.

22.5 Example and structural coverage

The release checks every standalone-supported example root through Direct-SV emission and strict Verilator lint. Child/template roots require a concrete specialization or parent. These tests are evidence for the shipped examples, not a language allow-list; unsupported new combinations fail closed. Current corpus counts and tool versions are published in release status.

The streaming FFT and IEEE 802.11a transmitter examples exercise typed hierarchy, ROMs, fixed-point datapaths, protocols and state. Their RTL is built through the same generic backend path as other designs, without module-name dispatch. Selected examples also have independent cycle-level oracle checks.

Generated RTL uses exact packed widths and single-driver combinational assignments. Procedural combinational blocks are used where defaults or branching require them; strict lint treats width, latch, driver and structural warnings as failures. A target binding is claimed only when the selected physical capability proves its clock, latency, collision and reset contract.

23 Compiler tooling API

ZLang HDL exposes a small, read-only Python integration surface in `zlang.tooling`. It is intended for editors, build coordinators and other compiler-aware tools that must not depend on parser, workspace or semantic IR implementation modules.

Consumers must check `TOOLING_API_SCHEMA`; the package version alone is not a compatibility guarantee. The API publishes immutable records for:

- compiler, source-suffix and capability identity;
- source declarations and direct-import resolution;
- the immutable document-symbol projection for the current source text;
- the immutable semantic-hover projection for one source position;
- the immutable compiler-resolved definition projection for one source position;
- the immutable compiler-resolved reference projection for one source position;
- the immutable semantic rename-edit projection for one source position;
- the immutable compiler-visible semantic completion projection for one source position;
- the immutable compiler-resolved signature-help projection for one source position;
- the immutable compiler-classified semantic-token projection for one source;
- compiler-owned machine-applicable diagnostic fixes for semantic checks;
- read-only project/workspace indexing and dependency closure;
- one exact semantic-only source-snapshot check.

These calls do not emit RTL, run formal tools, synthesize a design or mutate a project lock. A failed semantic check is returned as a structured record. An environment or malformed-project failure that prevents a trustworthy record raises `ToolingError`.

23.1 Request-scoped analysis

Tooling queries request only the compiler observations needed for that query; they do not emit RTL or perform formal execution. A bounded compilation session may reuse analysis products after validating the exact project and editor snapshot. Definition, References, rename and semantic tokens use the same compiler-owned identity and occurrence records; no result is inferred from matching source text. See Language Server Protocol and VS Code for editor behavior and the API description above for integration details.

23.2 Diagnostic edits

`TOOLING_DIAGNOSTIC_EDIT_SCHEMA` is currently `1`. A failed `check_snapshot()` record retains the existing prose-only `ToolingDiagnostic.fixes` and may additionally contain immutable `ToolingDiagnosticFix` records. Each machine fix preserves its compiler-owned title and one atomic tuple of `ToolingDiagnosticEdit` records containing the absolute current-source path, exact compiler origin and replacement text.

Only explicit `DiagnosticError.machine_fixes` metadata is projected. Tooling does not parse diagnostic messages, codes, notes or legacy fix prose. The projection requires every edit to match the exact snapshot digest and a valid half-open span; otherwise the entire fix is omitted. The first and currently only producer deletes the later of two imports with identical logical path and alias. Cross-file machine edits are not projected in this schema.

The compiler/tooling records remain protocol neutral. They are not LSP `TextEdit`, `WorkspaceEdit`, `CodeAction` or `Command` values. The LSP layer leaves this schema unchanged and mechanically maps current-source machine fixes to edit-only LSP quick fixes after rechecking the current editor snapshot. The full audit and safety rationale are recorded in Compiler-owned diagnostic edit projection.

23.3 Generated RTL navigation audit

The generated-RTL audit adds no tooling navigation projection. The existing `GeneratedSourceMap` v1 is authoritative for hash-verified generated-line to source-origin attribution, but its bounded builder currently maps only one unique top-level output assignment. It has no generated artifact path or artifact-discovery contract, and no query currently validates a map against the current editor source digest. Recursive `BackendArtifact` bindings describe semantic/physical signals and hierarchy, not generated line locations.

Consequently `zlang.tooling` does not expose raw `BackendArtifact` or source-map objects and does not generate RTL on a tooling request. The exact capability and prerequisite are recorded in Generated source maps.

The public `load_generated_navigation_bundle()` API validates one explicit relocatable publication directory, exact generated/backend-manifest/source-map hashes, lineage, and complete producing source snapshot identities. The returned immutable record can classify current source text or digest as `match`, `stale`, or `unknown_source`. This is artifact infrastructure only: `zlang.tooling` exposes no generated-location query yet, and loading never invokes compilation.

23.4 Document symbols

`TOOLING_DOCUMENT_SYMBOL_SCHEMA` is currently `1`. The `document_symbols(source_text)` function returns a tuple of immutable `ToolingSymbol` records. Each record contains a declaration name, a stable compiler-owned category,

an authoritative `ToolingOrigin` where one is available, an authoritative selection range (currently the same origin when the parser does not retain a narrower name span), and recursively projected children where the parser exposes containment.

This is deliberately a narrow document-structure projection. It does not export AST nodes, typed IR, references, definitions, widths, protocols, or implementation plans. It uses the existing parser through the tooling module; the LSP must not parse or scan source text independently. If parsing fails for an incomplete editor buffer, `document_symbols` returns an empty tuple rather than guessing declarations. The existing `TOOLING_API_SCHEMA` remains `1` because this is an additive, separately versioned projection. Parser records that carry no stable declaration identity are omitted rather than assigned a synthetic symbol name.

23.5 Semantic hover

`TOOLING_HOVER_SCHEMA` is currently `1`. The `hover_at(source, source_text, line, character)` function accepts a stable source path and a zero-based editor position, then returns either an immutable `ToolingHover` record or `None`. The record contains only narrow compiler-owned facts: selected name and kind, canonical type text, width, signedness, fixed-point type text, port direction, an available callable signature, and its authoritative `ToolingOrigin`.

The function invokes the semantic-only compiler snapshot path. It does not invoke optimization, implementation selection, RTL generation, synthesis, or formal execution, and it never returns AST or typed-IR objects. Positions with no semantic entity and malformed/incomplete source return `None`; workspace, resolver, and other environment failures raise `ToolingError` so callers cannot mistake an unavailable project for a valid empty hover.

23.6 Definitions

`TOOLING_DEFINITION_SCHEMA` is currently `1`. The `definition_at(source, source_text, line, character)` function asks the semantic compiler snapshot for its resolved source-definition records and returns either an immutable `ToolingDefinition` or `None`. A definition contains only the resolved name/kind, the locked physical target path, and the authoritative target `ToolingOrigin`.

The projection currently covers compiler-resolved value/port references and ordinary or generic function calls, named types, enum members, registers, immutable locals, and module-instance targets when their declaration origin is retained. Function calls and declarations use parser-owned exact name spans. Imported project and stdlib definitions use the existing workspace lock and physical-input closure to map their logical source unit to the physical `.zhl` path. Unsupported declaration categories return `None` rather than being reconstructed from names or source text. Resolver and workspace failures remain `ToolingError`; the projection never exports AST, resolver, or typed-IR objects.

23.7 References

`TOOLING_REFERENCE_SCHEMA` is currently `1`. The `references_at(source, source_text, line, character, include_declaration)` function reuses the compiler's occurrence-to-target resolution records and returns immutable `ToolingReference` records containing only a locked physical source path and authoritative occurrence origin. Results are grouped by the compiler-owned declaration origin, never by identifier spelling, and are sorted deterministically with exact duplicate locations removed.

When `include_declaration` is true, the authoritative target declaration is included. When false, only resolved usages are returned. Malformed or unresolved source returns an empty tuple; workspace and source-path failures remain `ToolingError`. For a saved declaration in a locked project, the tooling layer considers only project roots whose dependency closure can contain that declaration. A parser-only name prefilter bounds the expensive work, then each candidate root is independently checked and only its compiler-owned occurrence-to-target records may become results. Reported spelling is validated against the exact source span to reject malformed inherited provenance. The prefilter is not a reference search and cannot create a result. Unsaved declaration snapshots remain local so in-memory and disk identities are not mixed. The persistent normalized shard described below only replays these compiler-owned records; no spelling-based result fallback, resolver object, AST, or typed-IR object is exposed.

23.8 Rename

`TOOLING_RENAME_SCHEMA` is currently `1`. The `rename_at(source, source_text, line, character, new_name)` function resolves the target through the compiler-owned definition/reference identity and returns an immutable

tuple of `ToolingRenameEdit` records, or `None` when the category is not rename-safe. Each edit contains a locked local source path, an exact parser-owned identifier `ToolingOrigin`, and the replacement text.

New names are validated with the parser's ordinary binding-name rule. Before returning edits, the tooling layer applies them to the current root snapshot and performs a semantic-only recheck; invalid names, collisions, retargeted references, missing exact spans, and cross-file edits fail closed with `ToolingRenameError`. The initial supported categories are local ports, immutable values, functions, and function parameters. Registers, instances, types, protocol members, and cross-file rename are intentionally unsupported. The LSP converts these records to the standard `WorkspaceEdit.changes` form; the tooling API does not expose AST, IR, resolver objects, or LSP classes.

23.9 Completion

`TOOLING_COMPLETION_SCHEMA` is currently `1`. The `completion_at(source, source_text, line, character)` function consumes compiler-recorded semantic scope snapshots and returns immutable `ToolingCompletion` records. The projection currently covers visible ports, immutable values, function parameters, ordinary/generic functions, and functions imported through the locked workspace. It does not reconstruct scopes, scan source text, add imports, or provide keyword/snippet completion.

Completion uses only the semantic check product. Malformed or incomplete source and positions outside an analyzed expression return an empty tuple; workspace and resolver failures remain `ToolingError`. The API does not expose the semantic scope, AST, resolver, or typed-IR objects themselves.

23.10 Signature help

`TOOLING_SIGNATURE_HELP_SCHEMA` is currently `1`. The `signature_help_at(source, source_text, line, character)` function returns either an immutable `ToolingSignatureHelp` record or `None`. The record contains one compiler-resolved callable label, its parameter labels, the active parameter index, and the authoritative call origin.

The semantic analyzer records a call only after ordinary, generic, or locked imported function resolution succeeds. Argument origins are retained by the compiler and are used to select the active parameter; the tooling and LSP layers do not parse commas, parentheses, or identifier text. Nested calls are resolved by choosing the most-specific compiler call span. Malformed or incomplete source and positions outside a resolved call return `None`, while workspace/environment failures raise `ToolingError`. Signature help demands only the semantic snapshot and does not run optimization, implementation selection, RTL generation, synthesis, formal tools, or external processes.

The projection deliberately exposes no AST, typed IR, resolver state, generic substitution machinery, or implementation metadata. Trigger characters are not advertised by the initial LSP slice because requests are handled identically regardless of how they are triggered.

23.11 Semantic tokens

`TOOLING_SEMANTIC_TOKEN_SCHEMA` is currently `1`. The `semantic_tokens(source, source_text)` function returns immutable `ToolingSemanticToken` records for exact compiler-owned declaration and resolved-occurrence spans in the current root document. Tooling categories are protocol independent: `function`, `parameter`, `property` for ports, and `variable` for immutable values. Declarations carry the `declaration` modifier; usages do not.

The projection consumes the same semantic-only snapshot and exact identifier origins used by definition, references, and rename. It rejects broad or multiline spans, filters imported declarations out of the root document, orders records deterministically, removes exact duplicates, and fails closed on overlap. It does not classify source text, expose a lexer/AST/IR object, or assign numeric LSP legend indices. Malformed or semantically invalid source returns an empty tuple; project/environment failures remain `ToolingError`.

23.12 Reusable tooling sessions

A `ToolingSession` can reuse immutable semantic and symbol results for repeated editor requests. Reuse requires matching compiler/schema, selected root and profile, exact editor text, and the content of the locked dependency closure. Unsaved buffers stay in memory; saved project symbols may be cached on disk. Neither a matching name nor a timestamp alone is validity evidence. Corrupt or stale entries are ignored and checked again by the compiler.

The cache is an optimization, not an alternative language analysis. It never stores source bodies or generated RTL and does not grant unsupported navigation or change diagnostics. The user-facing cache switch is described

under Cache configuration.

24 Language Server Protocol and VS Code

`zlang-lsp` is the single Community ZLang HDL language server. Its protocol surface provides deterministic diagnostics, document symbols, semantic hover, compiler-resolved definition locations, semantic references, safe semantic rename, deterministic semantic completion, and compiler-resolved signature help, full-document semantic tokens, and compiler-owned diagnostic quick fixes. It keeps open document text in memory and queries the compiler-owned tooling projections after full-document opens and changes.

Start it as a standard LSP process:

```
zlang lsp
```

`zlang-lsp` remains an installed compatibility executable for editor settings that accept a program path but cannot express subcommand arguments.

The process reads and writes standard Content-Length framed JSON-RPC messages. It supports local `file ://` URIs for existing `.zhl` files. The server does not write editor buffers to temporary source files; project-backed snapshots must therefore remain compatible with the existing compiler/tooling workspace identity rules. Unsupported remote, virtual, and non-local document URIs are reported explicitly.

24.1 Supported now

- `initialize` / `initialized`;
- `shutdown` / `exit`;
- `textDocument/didOpen`;
- `textDocument/didChange` using full-text synchronization;
- `textDocument/didClose`;
- `textDocument/publishDiagnostics`;
- `textDocument/documentSymbol` for an open document;
- `textDocument/hover` for an open document;
- `textDocument/definition` for an open document;
- `textDocument/references` for an open document;
- `textDocument/rename` for the conservative rename-safe subset of an open document.
- `textDocument/completion` for compiler-visible expression-scope candidates.
- `textDocument/signatureHelp` for one compiler-resolved callable at the requested position.
- `textDocument/semanticTokens/full` for exact compiler-classified identifiers.
- `textDocument/codeAction` for compiler-owned machine-applicable quick fixes.

Diagnostics use the Community `zlang.tooling.check_snapshot()` API, which uses the normal parser, resolver, and semantic checker. They keep the compiler code, message, source origin, and one-based-to-zero-based range conversion. Document symbols use the narrow parser-owned `zlang.tooling.document_symbols()` projection over the current unsaved editor text. Hover uses the narrow semantic `zlang.tooling.hover_at()` projection over the same current text and source path. The LSP does not contain a second parser, type checker, workspace model, backend, formal runner, or optimizer.

Editor observations are demand-driven through one compiler-owned `AnalysisNeeds` mask. Definition, references, rename, and semantic-token requests collect only shared definition records; completion additionally requests scope candidates, and signature help requests resolved-call records. A definition request therefore does not build completion scopes or callable detail strings, while semantic checking and the resulting IR remain unchanged.

Document symbols are structural declarations, not a semantic database. Their names, categories, containment, and available source spans come from the compiler's parser representation. Incomplete or malformed text returns an empty symbol list while diagnostics continue to report the parse error; there is no regex or source-text fallback. A declaration whose AST node has no dedicated source origin uses the enclosing authoritative declaration span.

Hover reports only compiler-owned facts that are already available from the semantic IR: the selected name/kind, canonical type, width, signedness, fixed-point representation, and (for ports) direction. Function declarations may include their compiler-owned signature. It does not run optimization, implementation selection, RTL generation, synthesis, or formal tools. A malformed/incomplete buffer or a position without a semantic entity returns `null`; workspace/environment failures remain explicit protocol errors.

Definitions use `zlang.tooling.definition_at()` and the compiler's actual name/import resolution. The server does not search source text, rebuild scopes, or choose same-named declarations. Project imports are mapped through the locked workspace to the target source URI; unresolved or unsupported categories return `null`.

For a file declaring several modules, Definition and References share one bounded navigation-context selector. A missing default-top occurrence is retried under the module preceding the cursor according to parser-owned module declaration spans. Only that selected module is then semantically analyzed; the compiler's exact occurrence-to-declaration record still decides whether F12 returns a location. A bounded session-local, exact-text selector avoids rechecking an unrelated generic last module on repeated F12; no source AST or physical path is added to persistent symbol shards.

Protocol connection endpoints retain exact parser-owned name spans. Each name is resolved by semantic elaboration, so a connection such as `command -> packet_mapper.command` can navigate independently to the parent port declaration, the instance declaration, and the selected child port. The cache stores only those compiler-produced relations.

VS Code may issue F12 at the exclusive right edge of its selected identifier. Tooling performs the exact half-open-span lookup first, then retries the immediately preceding code point only when the cursor is at an identifier boundary. It does not walk across whitespace or punctuation. Declaration paths outside the root project, including stdlib targets, are returned only when they are members of the compiler-owned locked physical input closure.

Definition coverage includes compiler-resolved module instance targets, named user types (including nested generic components, structs and aliases), nominal enum types, enum members, and local registers. Targets retain their locked source identity, digest, and parser-owned declaration span, so a dependency file need not be open in the editor. Builtin constructors such as `rv` and `bits` have no fabricated source definition. Source-local target/resource declarations also retain exact lexer-token name spans: `provides ResourceName` and `require_resource ResourceName` navigate to one unambiguous `resource ResourceName` declaration and participate in semantic References. Unknown or duplicate source-local names return no location. This does not yet implement general cross-file target-catalog navigation; physical target legality remains with the separate target planner. The server converts VS Code's incoming UTF-16 character offsets to compiler code-point columns before semantic span containment; outbound locations use the inverse conversion.

The extension completes language-client registration before advertising navigation support, so an immediate first F12 request is supported. It does not perform extension-side symbol lookup.

Generic declarations are semantically checked for their concrete specializations. When a generic-only library file is opened directly, the compiler's dedicated `ZL-GENERIC-SPECIALIZATION-REQUIRED` condition is not published as an editor error. A parameter constraint referencing a declared value parameter without a default is likewise deferred only while directly viewing its unspecialized template; concrete false constraints and unrelated runtime references remain editor errors. The production compiler still rejects an unspecialized top, and parse/concrete-specialization errors remain ordinary diagnostics.

A saved project child opened directly is checked using the semantic analyzer's existing child-boundary mode. This avoids treating an enum-bearing internal port as an illegal public top ABI merely because its parent is not the active editor document. The production compiler's default top-level boundary remains fail-closed; all other child semantics and concrete imported types are checked.

References use `zlang.tooling.references_at()` and the same compiler-owned occurrence-to-target relation. Results are grouped by resolved declaration identity, not by spelling, and may include the declaration when the LSP `includeDeclaration` flag is true. For a saved locked-project snapshot, the tooling layer builds an on-demand bounded view of dependency-compatible project roots and accepts only compiler-resolved occurrences with an exact source span. The project sweep reloads the current manifest/lock and reuses each exact root text for parser filtering and semantic checking. It verifies disk bytes before publication; a concurrent edit or more than 128 eligible roots/top pairs reports an error instead of a stale or silently truncated result. A declaration above several modules in one source also checks bounded sibling tops that may use it; this includes encoded-enum uses inside a non-default module. A parser prefilter limits semantic checks but never produces a reference result. Qualified enum owners and members retain separate identifier spans, and incoming LSP UTF-16 cursor columns are converted before compiler lookup. The normalized symbol cache described below reuses compiler-produced shards; it does not create results. There is no text/regular-expression fallback, and an unsaved declaration snapshot is never mixed with disk roots.

Rename uses `zlang.tooling.rename_at()` and the same compiler-owned identity and occurrence records. It edits only exact parser-owned identifier spans, always including the declaration, and rechecks the edited in-memory root source with the semantic checker to reject collisions or retargeted references. Rename supports local ports,

immutable values, functions, and function parameters. Cross-file edits and declarations without exact editable spans are rejected; there is no textual or same-spelling fallback.

Completion uses `zlang.tooling.completion_at()` and semantic scope snapshots recorded by the compiler while checking the current source. It offers only visible ports, immutable values, parameters, ordinary/generic functions, and locked imported functions. Candidates are deterministic, insert the plain identifier, and carry optional type/signature detail. The server does not reconstruct scopes, scan source text, synthesize imports, or provide keyword or snippet completion. Positions outside a compiler-analyzed expression and malformed/incomplete buffers return an empty list.

Signature help uses `zlang.tooling.signature_help_at()` and call records captured while the compiler resolves ordinary, generic, and locked imported function calls. The callable label, parameter labels, return type, call span, and argument spans are compiler-owned. The server selects the most-specific resolved call containing the cursor and determines `activeParameter` from those spans; it never counts commas or parses call text itself. A cursor in malformed/incomplete source, or outside a resolved call, returns `null`. No trigger characters are advertised until parser behavior makes a trigger contract useful.

Semantic tokens use `zlang.tooling.semantic_tokens()` and only the exact identifier spans already retained by compiler definition/reference records. The initial categories are functions, parameters, ports, and immutable values; declarations carry the standard `declaration` modifier and resolved usages do not. The server publishes a fixed standard-token legend, sorts tokens by source position, converts compiler code-point columns to the default LSP UTF-16 code units, and emits standard relative full-document encoding. It does not lex, scan, classify keywords, infer token roles from spelling, or expose hardware resource information. Malformed source produces an empty token data array.

Code actions recheck the exact current in-memory source through `zlang.tooling.check_snapshot()` and map only `ToolingDiagnostic.machine_fixes`. The current action is the compiler-owned deletion of a later completely identical duplicate import. Actions are `quickfix` edit-only `CodeAction` values, filtered by the request range, `context.diagnostics`, and `context.only`; stale diagnostics and fixes are not returned. Compiler-native spans are converted to LSP UTF-16 positions without changing replacement text. Prose-only diagnostic suggestions never become executable actions.

24.2 Cache configuration

The LSP reuses exact validated semantic and symbol results, including after reopening an unchanged saved project file. Changes to editor text, selected module, manifest, lock or any used dependency invalidate affected results; unsaved buffers are memory-only. A cache miss runs normal semantic analysis. Set `ZLANG_LSP_SYMBOL_CACHE=memory` to disable disk caching or `off` to disable symbol shards; `persistent` is the default. Cache corruption or a read-only cache location cannot turn a valid source into a navigation failure.

Existing source maps are hash-bound and exact for their few mapped generated lines, but they do not yet provide complete artifact discovery, current-editor-source staleness validation, or broad RTL coverage. The LSP does not load raw backend manifests, infer mappings from generated names, or generate RTL during navigation. See Generated source maps.

The generated-navigation bundle provides a protocol-neutral validated generated-navigation bundle for explicit published artifact discovery, integrity, lineage, and source-snapshot freshness. No LSP method consumes the bundle yet, and advertised server capabilities are unchanged.

24.3 Not implemented

`codeAction/resolve`, fix-all/source/refactor actions, semantic-token range/delta requests, workspace symbols, generated-RTL navigation, formal/synthesis commands, and other unlisted capabilities are outside the current public scope.

25 Structured diagnostics

Every compiler error retains its human-readable message and may also carry a stable diagnostic code, one primary `SourceOrigin`, notes, and suggested fixes. `SourceOrigin` always has a half-open source span and construct; its logical source unit and lowercase SHA-256 digest are optional in the base record. A *complete* origin has all four fields. Compiler-shipped declarations use logical units such as `std.math.complex`; ordinary CLI inputs use the path supplied to `zlang`.

Text remains the default and is compatible with existing scripts:

```
zlang design.zhl --check
```

The `fixes` values in the structured diagnostic are human-readable suggestions, not source edits. Machine-applicable edits have a separate compiler-owned metadata and tooling projection described in compiler-owned diagnostic edits; consumers must never derive edits from these strings.

Machine consumers select one deterministic JSON object:

```
zlang design.zhl --check --diagnostic-format json
```

The JSON object has `schema`, `severity`, `code`, `message`, `primary`, `notes`, and `fixes`. Older exception-string users continue to receive the same `str(error)`. Categories initially carrying specific codes include parsing, imports, width assignment, timing alignment, domain crossing, protocol ownership/type, top selection, I/O, and backend binding failures. Unmigrated errors use a stable generic category rather than inventing meaning from text.

Backend output can additionally publish a hash-bound generated source map. External-tool attribution is accepted only when the generated text hash matches the sidecar and exactly one entry covers the reported line. Otherwise the original Verilator/Yosys/vendor diagnostic is left unchanged.

26 Generated source maps

ZLang backends can publish a deterministic JSON sidecar alongside an immutable `BackendArtifact`. The sidecar is backend-independent: each published entry connects an inclusive generated line range and semantic identity to a complete typed `SourceOrigin` (source unit, digest, span, and construct). The base origin record permits an absent source unit or digest, but such an incomplete record cannot establish fresh cross-file attribution. The sidecar also records the backend, module, selected-IR identity, and generated artifact SHA-256, so a map cannot silently be applied to different generated text.

The sidecar format is implemented by `zlang.backend.source_map.GeneratedSourceMap`. Direct-SV offers `emit_artifact_with_source_map`; this returns the unchanged production artifact plus its sidecar model. `write_sidecar` writes canonical, sorted JSON. The CLI exposes the same model for one explicit production output:

```
zlang design.zhl --systemverilog Design.sv \
  --source-map Design.sv.zmap.json
```

The sidecar artifact hash must match the generated file before a consumer uses any line attribution.

For published consumers, the versioned validated generated-navigation bundle now provides the missing explicit file/source-snapshot integrity boundary. It does not expand this map's bounded line coverage and does not add an LSP navigation method.

The external-tool helper applies the same rule to Verilator messages: it adds a `ZLang origin:` line only for one exact mapped generated line after hash validation. Changed files, malformed locations, ambiguous entries, and unmapped helper/state-machine lines retain the original tool diagnostic unchanged.

26.1 Exactness boundary

The current generated-source-map implementation maps only a simple top-level output assignment when all of the following are true:

- the typed output expression retains a complete `SourceOrigin`;
- the `BackendArtifact` publishes the matching semantic output binding;
- the emitter's generated assignment statement is uniquely identifiable; and
- the generated text hashes to the artifact identity.

Direct-SV maps a unique `assign <published-token> = ...` statement. Ambiguous, sequential, multi-output, hierarchical, protocol, and backend-generated helper lines remain deliberately unmapped. No mapping is inferred from similar source and RTL names. Later emitter refactoring may attach exact origins while fragments are constructed; until then, absence of an entry means "unknown," not "same as the nearest mapped line."

26.2 Generated-RTL navigation boundary

A generated source map gives exact generated **line** ranges, not columns. Only a unique top-level output assignment with a retained complete compiler `SourceOrigin` is currently mapped; the final output assignment of an eligible pipeline may also qualify. Module declarations, internal state, helpers, protocol bridges and hierarchy are otherwise unmapped. An origin's span and construct are always present; its logical source unit and digest are optional in the base record, but a complete mapped origin has all four.

A consumer must first verify the generated text against the artifact SHA-256. Zero matching entries means "unmapped"; multiple distinct complete origins are ambiguous and must not be guessed from names or proximity. The bundle can validate source-snapshot freshness, but there is no supported source-to-generated query or LSP navigation method. Generated coordinates are one-based lines; source spans use compiler-native code-point columns, converted to UTF-16 only by an LSP client boundary. See the generated-navigation bundle for publication and integrity details.

27 Generated navigation bundles

ZLang can publish a small, relocatable Community artifact bundle that binds one already-generated Direct-SV file to its exact generated source map, backend manifest, and producing source snapshots. The bundle is an integrity and provenance boundary for tooling. **It does not provide an LSP navigation method.**

Publish it during the normal compiler invocation:

```
zlang design.zhl --top Design \
  --systemverilog build/Design.sv \
  --source-map build/Design.source-map.json \
  --build-manifest build/Design.build.json \
  --generated-navigation-bundle build/Design.navigation
```

The source-map and whole-build outputs remain independently optional. The bundle always contains its own exact generated file, generated source map, and serialized `BackendArtifact`; it never invokes a second backend render.

27.1 Why generated navigation uses a separate bundle

Before this contract, `BackendArtifact`, `BackendBuildRecord`, and `WholeBuildManifest` already carried backend/module/selected-IR identities, artifact hashes, dependency identities, and relocatable logical product paths. `GeneratedSourceMap` carried the same artifact lineage and exact mapped source origins. Those records did not provide a single persisted path from which a consumer could locate the arbitrarily placed physical RTL and sidecar, and the whole-build root publication did not retain the compiler logical source unit.

The navigation bundle is a narrow publication contract that avoids changing unrelated whole-build consumers. It reuses the canonical serialized `BackendArtifact` as the lineage owner instead of duplicating its hierarchy, implementation, timing, module-signature, and dependency structures.

27.2 Layout and schema

All persisted paths are normalized relative paths below the bundle root:

```
manifest.json
generated/design.sv
generated/source-map.json
manifest/backend-artifact.json
```

`manifest.json` identifies its own format for strict compatibility checks. It contains:

- `bundle_identity`: SHA-256 of the canonical identity payload;
- `generated_artifact`: a `PublishedFile` logical path, SHA-256, kind, and size;
- `source_map`: the same record for canonical `GeneratedSourceMap` JSON;
- `backend_manifest`: the same record for canonical `BackendArtifact` JSON;
- `sources`: an ordered list of `{role, source_unit, digest}` records;
- `schema` and `schema_version`.

Exactly one source has role `root`. Every compiler-owned dependency from the backend artifact's locked dependency closure and resolved library dependency set is recorded with role `dependency`. Absolute checkout/build paths are not serialized. Package/revision identities remain in the hashed backend manifest; source snapshot matching uses the exact logical source unit plus SHA-256.

The root record is mandatory even when `GeneratedSourceMap.entries` is empty. No fake source-map entry is created. Consequently an empty map can still report the root source as matching or stale.

27.3 Publication and validation

`publish_generated_navigation_bundle()` accepts an already-emitted `BackendArtifact`, its `GeneratedSourceMap`, and the already-written generated file. It verifies the physical generated bytes against `artifact_hash`, checks backend/module/selected-IR/map lineage, derives source snapshots from existing compiler dependency metadata, and publishes the four relative files with the existing no-follow atomic publication utility.

`load_generated_navigation_bundle(path)` starts from one explicit bundle directory. It does not scan for `.sv` or JSON files and does not infer filenames from a module. Loading validates:

- bundle schema and canonical identity;
- safe relative paths and no-follow containment;
- presence, regular-file status, exact sizes, and SHA-256 of all child files;
- canonical backend-manifest and source-map serialization;
- generated bytes against the backend artifact hash;
- Direct-SV backend, module, selected-IR, and source-map lineage;
- exact source snapshot agreement with backend dependency provenance;
- complete `source_unit` and digest identity for every mapped source origin.

Any mismatch is an error. Missing files, stale/tampered contents, sidecars from another artifact, unsupported schemas, path traversal, and symlinked bundle components fail closed. Loading performs only metadata parsing, file reads, and SHA-256 checks; it never parses ZLang, compiles, emits RTL, runs formal tools, or starts an external process.

27.4 Source freshness

The immutable loaded record provides two protocol-neutral comparisons:

- `source_status(source_unit, current_text)` hashes current UTF-8 text;
- `source_digest_status(source_unit, current_digest)` compares a supplied SHA-256 directly.

Both return exactly `match`, `stale`, or `unknown_source`. Unknown sources are never treated as matching. Bundle facts are validated at load time; current editor text is checked separately because it can change after loading.

27.5 Relocatability and limitations

The complete directory can be moved or copied and loaded at its new location. No original absolute path is required. Current bundles are Direct-SV only and do not embed producing source contents. They validate caller-supplied source text/digests but do not locate workspaces or manage unsaved overlays.

Source-map coverage remains the bounded line-only coverage documented in Generated source maps. generated-navigation bundle adds no generated columns, recursive line mappings, virtual documents, artifact index, implicit compilation, or source-to-generated query. **There is no LSP generated-RTL navigation yet.**

28 Whole-build manifests and evidence

ZLang can describe one reproducible build as a deterministic, versioned record. The whole-build manifest joins the compiler stages, selected implementation policy, backend products, companion files, actual tool executions, generated reports, and typed evidence without making any of those records the source of language semantics.

The first schema is `zlang-whole-build-manifest-v1`. It is complementary to, not a replacement for, the per-backend `BackendArtifact` manifest.

28.1 Compiler and implementation identities

A build records two distinct canonical compiler identities:

- `high-level:<sha256>` identifies the fully typed, backend-independent high-level canonical IR;
- `selected:<sha256>` identifies the selected canonical architecture IR.

The identities use the versioned canonical-identity schema and exclude source spans and other attribution-only metadata. An optional canonical content hash may accompany either reference. The high-level and selected identities are not backend artifacts, RTL hashes, or synthesis-plan identities.

The production Direct-SV backend is planned from the selected-IR identity. Its record keeps these identities separate:

- the normalized backend plan identity;
- the backend `build_identity`;
- the exact emitted artifact hash and BackendArtifact version;
- an optional physical implementation-graph identity;
- an optional source-map hash.

This distinction prevents a physical Direct-SV resource plan from being mistaken for the selected semantic design. Backend states are explicit: `selected`, `generic_fallback`, `unsupported`, `failed`, or `not_requested`. A selected or fallback build must publish an artifact, its manifest version, and at least one content-addressed output. Unsupported, failed, and unrequested plans cannot silently publish successful products.

28.2 Published files and companions

The manifest records the exact compiled root-source bytes, the complete locked project and compiler-shipped `std.*` dependency closure, backend outputs, and backend companion bundles as `PublishedFile` values. Every value contains:

- a normalized relative POSIX path;
- a SHA-256 content hash;
- a file kind;
- the exact byte size for build products;
- optional ZLang source attribution.

Absolute paths, `..`, non-normal paths, and duplicate publication paths are rejected. Directory-root validation additionally rejects symlink escapes. The CLI's explicit logical-path-to-physical-`Path` map checks every build product's type, size, and content hash before publishing the manifest. Locked dependency records are the exception: the project workspace has already validated their content hashes before semantic analysis, so their physical cache/check-out paths are intentionally absent from the public manifest validation map.

Before any product is written, the CLI also checks a private, non-semantic map of every physical compiler input: the root snapshot, project manifest, lock, root and dependency modules, dependency manifests, and every consulted stdlib candidate. Explicit file sinks, generated companions, and compiler-owned output or cache directories may not alias or contain any of those inputs. File sinks may not alias one another, and owned directories may not overlap. Physical paths protect the workspace from overwrite; they never enter semantic, artifact, cache, or whole-build identity.

Companions remain associated with the backend product that needs them. For example, an initialized ROM image is not hidden inside the RTL identity: the image is a separate content-addressed companion validated beside Direct-SV `$readmemb` output.

The CLI acquires the root source once as raw UTF-8 bytes. Semantic compilation and manifest hashing use that same snapshot; CRLF is preserved, and a physical source change before final publication rejects the manifest. Generated RTL and ROM companions are published through one relative-path publisher: every directory and leaf is opened without following symlinks, the payload is fsynced in a same-directory exclusive temporary, renamed atomically, and revalidated by content. Symlinked or non-regular destinations fail without writing through them. Every published companion retains one semantic identity and is hash-validated independently.

Generated source maps and rendered reports are content-validated in the same way. A report's stable semantic identity is derived from its report ID, kind, and ordered evidence IDs. Its rendering format, content hash, and optional publication path remain serialized and tamper-checked, but intentionally do not alter the whole-build identity. A report cannot refer to unknown evidence, and a published report path cannot carry different bytes.

28.3 Exact evidence meanings

Evidence is adapted only from typed compiler/formal result objects. Log strings and ad-hoc result dictionaries are not evidence inputs. The stable statuses mean:

Status	Exact meaning
`typed_legal`	Semantic analysis produced a well-typed module. This is not a timing or proof result.
`timing_validated`	Every public scalar output was checked against the exact typed module timing contract. This is not a proof result.
`bounded_pass`	BMC found no counterexample through the recorded positive `depth`. It requires mode `bmc` and is never described as unbounded proof.
`proven`	An unbounded `prove` execution succeeded. Only this status denotes an unbounded proof.
`failed`	The executed check found a failure; typed counterexample attribution may be attached.
`unknown`	Execution completed without a pass, proof, or counterexample classification.
`skipped`	The route was explicitly unavailable or inapplicable. It is not success.
`not_run`	No proof execution occurred. It is not success.

`bounded_pass` must include BMC mode and depth. `proven` must include prove mode. Semantic and timing validation cannot carry proof mode/depth. An unexecuted record is rejected if its claim says that something was proved.

The typed adapters preserve the applicable property ID, candidate identity, backend and artifact hash, reference hash, engine, solver, relation, proof route, mode, and depth:

Positive formal-aware selection evidence requires a connected backend and artifact. The report layer does not infer execution from cache metadata, registered rewrite rules, generated properties, an SBY file, or a solver executable being installed.

28.4 Harness generation is not proof execution

`--formal-harness` and `--formal-sby` generate formal inputs. They do not run a solver and therefore cannot produce `bounded_pass` or `proven`. When no typed execution result exists, the corresponding property is reported as `not_run` and may explain why execution was not performed, but it must not imply success. Artifact generation itself belongs in the backend/publication records, not in a second evidence status.

The first-class verification UX adds an immutable bundle boundary rather than changing that rule. `--verification-bundle` publishes hash-validated structured verification IR, implementation/source-map inputs, and separate safety/cover jobs. Exact ROM images are immutable `companion` inputs to every job that uses them. `zlang verify` or `zlang --verify` creates a run report only after real execution. Solver, engine, depth, timeout, logs, tool versions, witnesses, and counterexamples are run data and are not folded into the bundle's source identity. A bounded cover miss is `bounded_unreached`, never proof of unreachability.

The bundle describes immutable inputs. A separate run report records `run_identity`, exact execution configuration, retained bounded-stage results, tool versions, and a work directory for each safety/cover job. Generated SBY configuration, stdout/stderr, timeout diagnostics, and VCDs live outside the bundle below `--work-dir/--verification-work-dir`. A retained VCD may be decoded through the immutable binding table into semantic signal/value pairs; paths and raw logs remain outside `run_identity`.

A joint `--verify` plus non-`off` formal-policy run publishes a compiler report that links the raw execution report to the exact compiler execution plan and separately typed selected-candidate semantic-reference equivalence reports. Candidate reports retain deterministic per-route work roots and the discovered tool snapshot when execution needed tool discovery; exact in-session formal-aware selection reuse also carries its recorded work root. Physical paths remain operational metadata outside every semantic, run, evidence, and cache identity. Persistent formal-aware selection cache payloads omit them rather than claiming that an old workspace is still available.

Each safety/cover job is tied to one exact clock/reset pair. Multiple supported synchronous domains may therefore appear as independent jobs in one bundle; an unsupported domain skips only goals that name it. This is not a cross-domain equivalence or temporal proof relation.

A requested prove run is staged behind BMC. Proof starts only when every safety job is `bounded_pass`. Covers run once during BMC and are not rerun; an unrelated cover result does not block safety proof, while an unwitnessed feasibility cover first makes its dependent safety result vacuous/unknown and therefore blocks proof.

The merged report retains both the bounded cover evidence and any proved safety results. This sequencing is part of execution, not bundle identity.

A safety counterexample or an executed joint semantic-reference equivalence counterexample is a verification failure. Missing/unknown/vacuous safety verification/source evidence or an unsatisfied requested proof is incomplete. A bounded cover miss is non-failing, and unavailable advisory candidate evidence neither changes formal-aware selection eligibility nor makes an otherwise complete joint run incomplete.

Likewise, merely detecting Verilator, Yosys, SymbiYosys, or Z3 does not create a tool-execution record. A tool record represents an actual normalized invocation and includes its role, exact reported version, path-free command shape, result status and exit code, logical outputs, and proof mode/depth when applicable. Tool output paths must resolve to files already published by the build.

28.5 Counterexample and source attribution

Source origins remain attribution, not semantics: they include the logical source unit, source digest, span, and construct, survive manifest JSON round-trips, and are excluded from deterministic evidence and build identities. Moving an unchanged locked project therefore does not change its build identity.

28.6 CLI publication

The intended publication forms are:

```
zlang design.zhl --systemverilog build/design.sv \
  --evidence-report build/evidence.json \
  --evidence-format json \
  --build-manifest build/zlang-build.json
```

`--evidence-format text|json` selects deterministic human-readable or JSON evidence. `--evidence-report` publishes that report. `--build-manifest` publishes the whole-build join after requested outputs have been written and their hashes validated. The manifest is fsynced to a same-directory temporary, renamed atomically, its physical publications are validated again, and the parent directory is fsynced; a detected intervening output mutation removes the manifest. A whole-build manifest requires a real backend product; report-only or check-only commands do not fabricate a successful build.

The build identity includes locked source/dependency hashes, both canonical IR identities, normalized implementation request/policy and optional profile, independent backend records, actual tool executions, reports, evidence, and stable metadata. Ordering is canonical. Host paths, timestamps, wall time, cache hit order, Python insertion order, and other volatile runtime state are not part of it.

28.7 Current limitations

These boundaries preserve the existing formal-infrastructure freeze while making build and evidence claims reproducible, attributable, and mechanically auditable.

29 Storage-owning instance arrays

This bounded composition slice extends one-dimensional compile-time instance arrays to same-domain scalar-wire children that own one storage resource. It does not add source syntax: the existing `inst lane[N]`, indexed bindings, and indexed child-output references elaborate into the existing typed hierarchy.

29.1 Supported boundary

A child specialization may contain ordinary registers/rules without storage, or own at most one of:

- a FIFO;
- a synchronous memory; or
- an initialized ROM.

The legacy globally controlled storage profile remains storage-only: it does not combine that resource with user registers, next-state assignments, or rules. The separately validated scheduled-FIFO profile does permit ordinary register/rule state. FIFO actions and register writes are selected atomically by the same existing

ResolvedTransition; the array backend does not create a second scheduler. Unsupported ownership combinations fail during semantic analysis rather than reaching a backend that could emit only part of the child.

The child must retain the existing storage-array contract: scalar wire ports, one inherited clock/reset domain, and no nested instances. Multiple scalar outputs are allowed. Each array element has a distinct physical instance identity and storage state; all elements share the one deterministic specialization identity. The Direct-SV backend consumes ElaboratedInstance bindings directly and emits one reusable component plus one application/instance per physical element.

CSR, other protocol/storage mixtures, multiple storage resources, arbitrary nested storage hierarchy, multiple clock domains, CDC, runtime-selected inputs or protocol endpoints, and formal observation extensions remain rejected. Runtime selection of a bit-packable scalar-wire output is supported as a read-only projection: lane [select].value lowers to a typed runtime index over all statically elaborated lane[i].value references. Every child continues to execute and retain independent state; the selector is only an output mux and never gates a child or denotes a dynamic physical instance.

The one bounded protocol/storage exception is a storage-only child whose external ports are exclusively primitive ready/valid and which owns exactly one globally controlled FIFO. It uses the same typed HierarchicalConnection, closed component ABI, and FIFO state as a non-array child. Ready/valid arrays with synchronous memory or initialized ROM remain rejected because they do not yet have a frozen protocol/storage contract.

The two-lane synchronous-memory witness checks independent cells and addresses, write-first same-address collision behavior, registered read output, and reset clearing both cells and read state. It passes strict Direct-SV/Verilator behavior.

The ZtpuBankedMemory witness elaborates two four-element arrays into eight independently identified physical memory children with one shared specialization. Runtime output projection selects the addressed bank separately for each read port; decoded writes are broadcast to the matching bank in both replicas. The semantic simulator and Direct-SV/Verilator agree on masked writes, independent reads, read_first collisions, reset suppression/preservation, and post-reset contents. The versioned BackendArtifact records every leaf path and no fictitious dynamic instance. Hidden cells gain no new formal observation family.

30 Source and product identity

The public product name is **ZLang HDL**. Its distribution and repository are zlang-hdl; the compiler command is zlang, the source suffix is .zhl, the VS Code language ID is zlang-hdl, and the MIME type is text/x-zlang-hdl.

The .zl suffix is not a compatibility alias. Physical compiler inputs using it fail with diagnostic ZL-SOURCE-EXTENSION; use .zhl.

Logical imports do not contain a source suffix. For example, import std.math.fixed resolves to stdlib/math/fixed.zhl, while project and locked-package imports use their logical module identities.

Older dependency locks are rejected by the current versioned lock schema; regenerate them with zlang lock update. Source, build, and proof caches are bound to dependency and source-unit identities, and stale inputs miss safely.

The short prose name **ZLang** remains valid after the product has been introduced. The Python package zlang , zlang.toml, zlang.lock, the compiler-owned .zlang/ state directory, environment variables prefixed ZLANG_, generated HDL identifiers, and the TextMate scope source.zlang are intentional product identities.

31 Language support matrix

Canonical implementation-selection markers: implement, choice.

Implementation selection uses the canonical implement form for compiler-discovered candidates and choice for user-supplied alternatives. The retired scalar pipeline(auto), architecture(auto), and explore spellings are migration diagnostics; only protocol transform pipeline(auto, ...) remains source syntax.

zlang.public_capabilities.CAPABILITY_REGISTRY is the machine-readable source for public capability context, semantic status, backend-independent IR, simulator/Direct-SV/formal coverage, executable witness, and current limitations. The advertised witnesses are compiled by the public capability gate. examples/all_syntax.zhl is a representative language tour, not a substitute for that phase-specific capability record.

The following language/backend view is generated from that registry. `bounded` means the detailed limits in this reference and the capability record still apply; "supported" is not a promise for every composition. `Simulation` is the registry's current generic simulator field, not a separate native-JIT guarantee. Native execution fails explicitly for unsupported plans; see Native simulation. Formal entries name the specific eligible relations rather than promising arbitrary proof.

31.1 Language and execution

Capability	Language	Simulation	Direct-SV
`scalar-datapath`	supported	supported	supported
`exact-literals-and-packed-constants`	supported	supported	supported
`fixed-point`	supported	supported	supported
`aggregates`	supported	supported	supported
`characters-strings-tuples`	bounded	supported	supported
`tagged-unions`	bounded	supported	supported
`functional-datapath`	supported	supported	supported
`compile-time-generation`	supported	not applicable	supported after specialization
`concise-exact-lowering`	supported	supported	supported
`typed-static-parameters`	bounded	not applicable	supported after specialization
`generic-rom-and-table-gather`	bounded	supported	supported
`sequential-state`	supported	supported	supported
`registered-outputs-and-drive`	supported	supported	supported
`physical-clock-reset`	bounded	supported	supported
`multi-clock-stateful-logic`	bounded	supported	supported
`encoded-enums-and-fsm`	supported	supported	supported
`vector-state-update`	bounded	supported	supported
`fifo-storage`	supported	supported	supported
`writable-memory`	bounded	supported	supported
`ready-valid`	supported	supported	supported
`credit`	supported	supported	supported
`request-response`	supported	supported	supported
`aggregate-protocols`	bounded	supported	supported
`ahb-lite-stdlib`	bounded	supported	supported
`cdc`	bounded	supported	supported
`csr`	bounded	supported	supported
`contracts`	supported	supported	artifact generation
`exploration`	bounded	not applicable	selected candidates
`bounded-intent-structural-exploration`	bounded	not applicable	selected typed alternatives
`capacity-one-temporal-sharing`	bounded	supported	supported
`elastic-ready-valid-pipeline`	bounded	supported	supported
`per-bit-native-logic-state`	bounded	supported	unchanged
`combinational-instance-arrays`	bounded	supported	supported
`runtime-instance-output-projection`	bounded	supported	supported
`sequential-instance-arrays`	bounded	supported	supported
`storage-instance-arrays`	bounded	supported	supported
`ready-valid-instance-arrays`	bounded	supported	supported
`request-response-instance-arrays`	bounded	supported	supported

31.2 Formal coverage

Capability	Formal coverage
`scalar-datapath`	same-cycle equivalence where eligible
`exact-literals-and-packed-constants`	same-cycle equivalence where eligible
`fixed-point`	fixed-point equivalence within supported widths and forms
`aggregates`	value equivalence where all required fields are observable
`characters-strings-tuples`	scalar and packed-value equivalence where eligible
`tagged-unions`	no dedicated tagged-union proof support
`functional-datapath`	scalar and fixed-point equivalence where eligible
`compile-time-generation`	compile-time only; formal execution not applicable
`concise-exact-lowering`	same formal coverage as the equivalent explicit source form
`typed-static-parameters`	not a runtime formal target; specialized values participate in the elaborated design being checked
`generic-rom-and-table-gather`	storage safety checks only; no dedicated table-gather proof
`sequential-state`	register and rule safety checks where the required state and events are observable
`registered-outputs-and-drive`	registered outputs inherit state safety checks where observable; transient drive actions have no separate proof route
`physical-clock-reset`	reset-aware properties for supported edge, polarity, assertion, and release modes when `power_up` is unspecified
`multi-clock-stateful-logic`	source goals are checked in their declared domains; automatic safety checks are bounded
`encoded-enums-and-fsm`	no dedicated enum or FSM proof support
`vector-state-update`	register safety where the selected element state is observable
`fifo-storage`	bounded FIFO safety checks
`writable-memory`	no dedicated memory-content proof support
`ready-valid`	payload-stability and transfer safety where observable
`credit`	sender and receiver credit safety when the exact counter is observable
`request-response`	outstanding-transaction and directional-buffer safety where observable
`aggregate-protocols`	no general aggregate-protocol proof support
`ahb-lite-stdlib`	register-state and ready/valid safety where signals are observable
`cdc`	no CDC correctness proof
`csr`	CSR state and access safety checks where observable
`contracts`	safety checking and bounded cover execution when supported
`exploration`	candidate equivalence can gate required selection when a supported equivalence check is available
`bounded-intent-structural-exploration`	exact-value alternatives retain equivalence provenance before implementation evidence is evaluated
`capacity-one-temporal-sharing`	capacity-one transaction-stream bounded model checking; unbounded proof is not claimed
`elastic-ready-valid-pipeline`	ready/valid stability safety only; end-to-end equivalence is not supported
`per-bit-native-logic-state`	simulation-only diagnostic semantics; not a formal proof relation
`combinational-instance-arrays`	hierarchical equivalence is not supported
`runtime-instance-output-projection`	hierarchical equivalence is not supported
`sequential-instance-arrays`	bounded safety for exposed child state and events; no general recursive hierarchy proof
`storage-instance-arrays`	no formal access to unexposed internal storage contents
`ready-valid-instance-arrays`	ready/valid safety where the required signals are observable
`request-response-instance-arrays`	request/response safety where the required signals are observable

32 Known limitations

ZLang `0.1.0a23` is an experimental alpha candidate. The compiler deliberately fails closed when a design falls outside a validated language/backend intersection: it must not publish RTL after silently dropping an IR entity.

32.1 Language and backend boundaries

- ZLang has no runtime procedural `if`, mutable local variables, implicit numeric casts, implicit fixed-point rescaling, or general HLS scheduler.
- Runtime-selected instance inputs/protocols, general cross-module atomic scheduling, full AXI4, automatic CDC insertion, and arbitrary stateful elastic pipelines are outside the alpha contract.
- Stateful objects inside one module may belong to independent explicit clock domains. A bounded `async_mem` is the sole storage exception: its one writer and one registered reader have distinct owners and form an explicit semantic boundary. Asynchronous 2RW memory, mixed widths, automatic banking, cross-clock atomic rules, derived/gated clocks, and target-aware scheduling across a CDC boundary are not implemented. Ordinary global memory control is still not composable with unrelated user register/rule state in the same module; use hierarchy until that unified-state slice is implemented.
- The Direct-SV production intersection is authoritative. Unsupported combinations must produce a structured diagnostic rather than partial RTL.
- The Python API is provisional. The command-line interface and versioned artifact/lock/bundle schemas are the intended integration surfaces.
- Simulation-only architectural state access is currently a generic Direct-SV/ Verilator facility for one exact clock/reset domain. It does not expose backend-created FIFO, CSR, protocol, CDC, or target-mapped state and must not be confused with synthesizable memory initialization.

32.2 Verification boundaries

The machine-readable capability registry and syntax support matrix are the detailed authorities for individual constructs. Report any accepted design that emits invalid RTL as a correctness defect.

A ZLang HDL concise source-authoring reference

This page is a concise guide to current executable ZLang HDL source, not a historical proposal. The compiler's typed IR defines semantics; generated SystemVerilog is not a second language specification.

For an unfamiliar construct, consult the syntax support matrix and known limitations. For exact width tables and backend limits, follow the linked topic guide.

A.1 Non-negotiable rules

- Source files use `.zhl`. The old `.zl` suffix is rejected.
- Run `zlang SOURCE --check`; highlighting is lexical and is not validation.
- Native simulation needs a matching platform wheel and fails explicitly when it is unavailable; there is no second Python execution engine.
- Hardware assignments are concurrent. `=` drives the current cycle and `<-` schedules next state at the active clock edge.
- Assignment types are exact. There is no implicit resize, signedness change, fixed-point rescale, protocol adapter, or clock-domain crossing.
- Do not invent `let`, `const`, `var`, `return`, semicolons, C-style `input`/`output`, or runtime procedural `if`.
- `if` is compile-time elaboration only. Use `?:`, `mux`, or `switch` for pure runtime values; use `when`/`else when`/`else`, `priority`, or `fsm` for state actions.
- Scalar `pipeline(auto)`, `architecture(auto)`, and `explore` are retired. Compiler-selected scalar implementation uses `implement`.
- `pipeline(N) { expression }` is exact N-cycle hardware, not an optimization request. Protocol `transform` `pipeline(auto, ...)` is a separate bounded ready/valid construct and remains supported.
- Unsupported combinations must fail closed. Never work around a diagnostic by guessing backend signal names or duplicating compiler semantics in source.

A.2 Smallest valid modules

Combinational logic:

```
module Add {
    in a, b : u8
    out y : u9 = a + b
}
```

Sequential logic:

```
module Accumulator {
    clock clk reset rst
    in enable : bit
    in value : u8
    out total : u9

    reg state : u9 = 0
    update: when enable {
        state <- truncate<9>(state + extend<9>(value))
    }
    total = state
}
```

All operands are read from one pre-edge snapshot. An accepted action group commits atomically; reset has priority. Multiple conflicting actions require an explicit `priority` relation or are rejected.

Multiple domains use the same declarations with one `@clock` owner:

```
module DualClock {
    clock a reset ra @a
    clock b reset rb @b
    reg left : u32 @a = 0
    reg right : u32 @b = 0
    rule TickLeft @a when 1 { left <- truncate<32>(left + 1) }
    rule TickRight @b when 1 { right <- truncate<32>(right + 2) }
}
```

An omitted owner is accepted only when exactly one domain can be inferred. Dynamic provenance follows through local expressions; an explicit `sync_level`, `pulse_toggle`, `handshake`, or `async_fifo(N)` is the only way to cross domains.

A.3 Types, literals, and exact conversions

Common scalar types are `bit`, `u8`, `s16`, `uint<N>`, `sint<N>`, and `bits<N>`. `char` is canonical `u8`; `string<N>` is canonical `vec<N,u8>`. Fixed point uses `fixed<W,F>`, `ufixed<W,F>`, or concise forms such as `SF8.8` and `SF_Sat8.8`.

- Unsuffixed integer literals use minimum exact width unless they occur directly at a concrete typed boundary.
- `a + b` and `a - b` follow the documented exact result-width rules; multiplication returns the full product width. Shifts preserve left width.
- Use `extend<N>` and `truncate<N>` for width changes.
- Use `quantize<T>` for explicit fixed-point rounding/overflow. Do not move a quantization boundary into products or reductions.
- Compile-time `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt` generate constants only; non-integral results require explicit `quantize<T>`.
- Use `bitcast<T>` only for equal-width representation changes. `pack`/`unpack` are supported low-level compatibility forms.
- `concat(a,b,...)` places the first operand at the MSB. Destination context never changes operand widths.
- Packed bit index zero is the LSB. `x[MSB:LSB]` uses an inclusive static slice; vector ranges `values[first..past_last]` are half-open.
- Indexed aggregates are packed LSB-first: `vec`/`string` element zero and tuple `item0` occupy the least-significant component recursively. First-declared struct fields and tagged-union tags remain at the MSB.

See types and numerics before changing a width or a fixed-point type.

A.4 Aggregates and functions

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Function bodies are pure and combinational. Bare body bindings are immutable, inferred, declaration-ordered locals; the final expression is the result. Return types may be explicit or inferred. Functions cannot capture module ports or contain state, protocols, delays, or pipelines.

Supported aggregates include `vec<N,T>`, nominal `struct`, nominal `enum`, tagged `union`, and structural tuples `(T,U)`. Prefer inferred locals when the type is exact and obvious. Keep explicit types at public ports, state, raw/enum boundaries, narrowing, and fixed-point quantization.

Generics are monomorphized. Type/value parameters and exact compile-time constraints are source declarations, not runtime hardware:

```
module Queue<type T,D=4> where D >= 2 && is_power_of_two(D) {
    // ...
}
```

Imports are logical. For example `import std.bus.reg` resolves compiler-shipped `stdlib/bus/reg.zhl`; `import std.math.complex as cx` adds a source-local qualifier but does not create a runtime namespace.

A.5 State, hierarchy, and protocols

- Declare one default synchronous domain with `clock clk reset rst`.
- `async reset arst @clk` means asynchronous assertion and fixed two-edge synchronized release. Do not emulate reset modes in user logic.
- Add `release externally_synchronized` only when deassertion is already synchronized outside; no internal 2FF is emitted.
- `reg q:T = VALUE` resets to `VALUE`; `reg q:T` has no reset branch.
- `out q:T` is a combinational output wire.
- `out reg q:T [@clock] [= VALUE]` is real architectural state: it holds when not updated and follows register clock/reset scheduling. It is not Verilog's historical `output reg` declaration category.
- `drive q = expression` is a transient rule/FSM-owned logical wire effect, zero when no owning action fires. It is not tri-state, high-impedance, electrical-strength, pad, analog, or multiple-driver-resolution behavior.

- `reg`, `fifo`, `memory`, and `rom` retain explicit typed state/storage semantics. Memory latency, collision, mask, and reset policies are not inferred.
- A concise child declaration is `child : ChildModule`; `inst child : ChildModule` remains accepted. Bind only typed ports/endpoints.
- `rv<T>`, `credit<T,N>`, and `request_response` properties such as `.transfer` are typed observations, not ordinary struct fields.
- Use explicit `connect`/`source -> sink`, adapters, and CDC crossings. The compiler never inserts these silently.
- Top-level structs and tuples become recursively named leaf ports; vectors use multidimensional packed arrays in the production Direct-SV ABI.

Read sequential state/storage and hierarchy/protocols before composing stateful children.

A.6 Functional datapath and implementation intent

`generate`, `map`, `reduce`, `sum`, and `dot` are statically bounded functional hardware expressions. They are not runtime loops.

Use exactly one `implement` region when the value is fixed and the compiler may choose among legal implementations:

```
y = implement {
    dot(samples, coefficients)
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        fmax >= 100
        minimize lut
    }
}
```

An Fmax constraint or objective compares candidates using their recorded evidence and provenance. It is not a guarantee of achieved device or silicon frequency; structural estimates, synthesis measurements, and routed measurements remain distinct.

The expression is the hardware meaning. `intent` contains hard constraints and one supported objective; it does not name optimization passes. For an eligible pure value, the compiler retains the source plus a bounded, structurally distinct set of checked exact egglog alternatives, then applies the same intent to the resulting provider candidates. This is deterministic bounded exploration, not exhaustive optimality. Positive-latency candidates require a valid clock/reset context. `choice` is reserved for user-authored equivalent alternatives. Egglog supplies only frozen pure scalar value rewrites; it does not schedule pipelines, resources, protocols, or state. Selected project profiles may set bounded `intent-exploration` count limits; there is no source-level timeout. Exact roots can reuse checked in-memory session/workspace recipes, while constraints, ranking, target selection and formal evidence are always evaluated again.

See optimization and formal before adding or changing an implementation policy.

A.7 Verification

```
contract arithmetic @ clk {
    require legal_inputs { (a < 8) & (b < 8) }
    ensure exact_output { y == a + b }
    cover maximum { y == 14 }
}
```

Named `assert`/`ensure` are same-cycle safety goals. `require` is an environment-owned precondition scoped to its contract. `cover` is bounded reachability; a missed cover is not an unreachability proof. BMC produces `bounded_pass`, never `proven`. Verification syntax does not legalize invalid hardware or feed optimizer range inference.

Do not add liveness, arbitrary SVA/SMT, temporal sequences, new observation families or source-level semantic-reference equivalence controls. Use the existing formal examples and compiler-owned routes.

A.8 Commands and completion check

```
zlang source.zhl --check
zlang source.zhl --top Top --systemverilog build/Top.sv
zlang source.zhl --top Top --target sky130-fd-sc-hd --systemverilog build/Top.sv
```

```
zlang source.zhl --top Top --verify
zlang sim source.zhl --top Top --clock clk --cycles 100
zlang sim source.zhl --top Top --logic-state --clock clk --cycles 100
zlang sim source.zhl --top Top --strict-uninitialized --clock clk --cycles 100
zlang verify build/verify --mode bmc --depth 20 --work-dir build/verify-work
zlang lock update --project zlang.toml
zlang lsp
```

In a `--events` JSONL schedule, `"repeat": N` repeats one event lazily for a bounded positive number of steps (at most 1,000,000 expanded events) without a project-specific generator.

Ordinary native simulation retains its deterministic two-state zero seed. `--logic-state` enables per-bit `0`/`1`/`U`/`X`: unreset state starts as `U`, an accepted unresolved write becomes `X`, and controlling values such as `U | 1` still resolve exactly. `--initial-reg` overrides selected initial registers and `--trace-signal` selects VCD signals; companion `__zlang_meta/*_u_mask` traces distinguish `U` from VCD's shared `x` spelling. `--strict-uninitialized` implies this mode and rejects observed `U` or `X`. Neither mode models `Z`, electrical resolution, or Verilog event regions. CLI VCD output is an event-index visualization: physical clock/reset levels match each compiled domain, but the JSONL schedule does not define analog time.